



XCPC Templates

常用 XCPC 模板

Competitive Programming

Data Structures • Algorithms • Mathematics

Kendieer

2026 年 9 月 3 日

前言

这份手册整理了在算法竞赛中常用的知识体系与代码模板，涵盖数据结构、图论、数学、字符串等专题，所有代码均以 C++23 为准。

使用说明

- 每个专题独立成章，可直接按目录跳转查阅；
- 代码块中的模板均经过评测验证，复杂度以注释标注；

约定

- 除特别说明外，数组下标从 1 开始；
- n, m 分别表示点数与边数（或序列长度与操作次数）；
- 所有取模运算的模数在代码中以 Z 表示，取模类见 `ModInt`。
- 以下是常用的类型宏定义

```
1 using i64 = long long;
2 using u64 = unsigned long long;
3 using p32 = std::array<int, 2>;
4 using p64 = std::array<i64, 2>;
```

- 部分函数传参带 s/beg 通常代表起始下标。
- 所有用到的 STL 库内容均会以 `using std::xxx` 或直接使用 `std::xxx`

Kendieer
2026 年 9 月 3 日

目录

第一章	基础算法	1
1.1	快速幂	1
1.1.1	整数快速幂	1
1.1.2	大模数防溢出	1
1.2	三分	2
1.2.1	实数版本	2
1.2.2	整数版本	2
1.3	离散化	3
1.4	格雷码	3
1.4.1	按格雷码序枚举子集	4
第二章	数论	5
2.1	质数筛	5
2.2	Miller–Rabin 素性测试	6
2.3	质因数分解 (Pollard–Rho)	7
2.4	拓展欧几里得算法 (exgcd)	9
2.5	中国剩余定理及其拓展 (CRT & exCRT)	9
2.6	BSGS 及其拓展	10
2.6.1	BSGS 算法	11
2.6.2	exBSGS 算法	11
2.7	欧拉函数	12
2.7.1	求单一数字 $\varphi(x)$	12
2.7.2	欧拉筛求 $\varphi(x)$	13
2.7.3	欧拉定理 (欧拉降幂)	13
2.8	原根	14
2.9	二次剩余 (Cipolla 算法)	15
第三章	生成函数与多项式卷积	17
3.1	快速傅里叶变换 (FFT)	17

3.2 快速沃尔什变换 (FWT) 以及位运算卷积全家桶	18
3.2.1 基础位运算卷积	19
3.2.2 子集卷积	21
3.3 快速数论变换 (NTT) 以及多项式全家桶	22
3.3.1 DFT 与 IDFT 函数	22
3.3.2 多项式杂项函数	24
3.3.3 多项式乘法	24
3.3.4 多项式乘法逆	25
3.3.5 多项式除法与多项式求余	26
3.3.6 多项式对数 (自然底数)	27
3.3.7 多项式指数函数 ($\exp$)	27
3.3.8 多项式开根	28
3.3.9 多项式快速幂	28
3.3.10 多项式多点求值	29
3.3.11 多项式快速插值	30
第四章 数值计算	32
4.1 拉格朗日插值	32
4.2 牛顿迭代法	34
第五章 组合数学	35
5.1 组合数学基础	35
5.1.1 排列组合基础	35
5.1.2 卡特兰数	37
5.1.3 卢卡斯定理	37
5.2 二项式定理与二项式反演	37
5.2.1 二项式定理	37
5.2.2 二项式反演	38
第六章 线性代数	39
6.1 矩阵	39
6.2 高斯消元	42
6.2.1 求解线性方程组	42
6.2.2 计算行列式	43
6.2.3 求矩阵逆	44
6.3 线性基	46
第七章 计算几何	48
7.1 二维计算几何	48
7.1.1 点	48
7.1.2 线	50
7.1.3 简单多边形	51

7.1.4	凸包	51
7.1.5	多边形面积与凸多边形面积交	52
7.1.6	曼哈顿与切比雪夫距离	54
7.1.7	极角与三角函数	55
第八章	数据结构	57
8.1	并查集	57
8.1.1	普通版本	57
8.1.2	可撤销版本	58
8.2	ST 表	59
8.3	树状数组	60
8.3.1	普通树状数组	60
8.4	线段树	61
8.4.1	静态线段树	61
8.4.2	开点线段树	64
8.4.3	可持久化线段树	67
8.5	平衡树	71
8.5.1	笛卡尔树	71
8.5.2	Splay 树	72
8.5.3	FHQ Treap	76
8.6	莫队算法	84
8.6.1	普通莫队	84
8.6.2	带修莫队	84
第九章	树上问题	86
9.1	最近公共祖先	86
9.1.1	$\mathcal{O}(1)$ LCA	86
9.1.2	树链剖分求 LCA	87
9.2	虚树	88
9.3	长链剖分	89
9.3.1	长链剖分优化 DP	89
9.4	Prufer 序列	90
9.5	Link Cut Tree	92
第十章	图论	97
10.1	强连通分量与缩点	97
10.2	边双连通分量	99
10.3	割点与割边	100
10.4	欧拉路径与欧拉回路	102
10.5	最短路	102
10.5.1	Dijkstra (堆优化, 非负权)	102

10.5.2 Johnson 全源最短路（允许负权、无负环）	103
10.6 二分图最大权匹配（KM）	105
10.7 网络流	106
10.7.1 网络最大流	106
10.7.2 网络费用流	114
10.7.3 上下界网络流	119
10.7.4 最小割树（Gomory-Hu）	121
10.8 Matrix-Tree 定理	122
10.8.1 生成树计数	122
10.8.2 BEST 定理	123
第十一章 字符串	125
11.1 KMP（border 数组）	125
11.2 Manacher	126
11.3 Z 函数（exKMP）	127
11.4 Lyndon 分解	128
11.5 后缀数组	128
11.6 AC 自动机（Aho-Corasick）	130
11.7 广义后缀自动机（GSAM）	132
11.8 回文树（PAM / Eertree）	134
11.9 Thue-Morse 序列	135
第十二章 杂项	137
12.1 STL 常用库	137
12.1.1 int128	137
12.1.2 pb_ds 常用容器	138
12.1.3 可并堆（左偏树）	140
12.2 取模类	141
12.2.1 取模类（静态模数）	141
12.2.2 取模类（全局模数）	144
12.3 复数	147
12.4 高精度	148
12.4.1 高精度整数	148
12.5 分数	153
12.6 时空优化	155
12.6.1 读入优化	155
12.6.2 指令集优化	157

基础算法

1.1 快速幂

`qpow(a,b,mod)` 计算 $a^b \bmod mod$, 复杂度 $\mathcal{O}(\log b)$ 。

1.1.1 整数快速幂

```
1 long long qpow(long long a, long long b, long long mod) {
2     long long res = 1;
3     for (; b; b >>= 1) {
4         if (b & 1) res = res * a % mod;
5         a = a * a % mod;
6     }
7     return res;
8 }
```

1.1.2 大模数防溢出

当 mod 接近 10^{18} 量级时 `res * a` 会溢出 `long long`, 改用 `mulmod(a,b,mod)` (借助 `__int128` 的不溢出模乘); `qpow` 用法同上, 仅内部换成 `mulmod`。

```
1 long long mulmod(long long a, long long b, long long mod) {
2     return (__int128)a * b % mod;
3 }
4 long long qpow(long long a, long long b, long long mod) {
5     long long res = 1;
6     for (; b; b >>= 1) {
7         if (b & 1) res = mulmod(res, a, mod);
```

```
8     a = mulmod(a, a, mod);
9 }
10 return res;
11 }
```

1.2 三分

三分查找求单峰函数（先增后减 / 先减后增）的极值。搜索区间端点为 l, r ，函数为 f 。实数域约需 $\mathcal{O}(\log_{3/2} n)$ 轮（ n 为区间长度），但下例固定迭代 100 次已足够。

1.2.1 实数版本

```
1 // 求凹函数（先增后减）最大值，f 在 [l,r] 单峰
2 double l = 0, r = 1e9;
3 for (int it = 0; it < 100; it++) {
4     double m1 = (l * 2 + r) / 3;
5     double m2 = (l + r * 2) / 3;
6     if (f(m1) < f(m2)) l = m1;
7     else r = m2;
8 }
9 double ans = f((l + r) / 2);
```

求最小值时对 $-f$ 求最大即可。实数域迭代 100 次能达到 `double` 精度。

1.2.2 整数版本

整数定义域收敛不到单点，故区间缩小到 ≤ 3 时暴力枚举区间内所有点取最优。

```
1 // 求凹函数（先增后减）最大值，整数域 [l,r]
2 int l = 0, r = n;
3 while (r - l > 3) {
4     int m1 = l + (r - l) / 3;
5     int m2 = r - (r - l) / 3;
6     if (f(m1) < f(m2)) l = m1;
7     else r = m2;
8 }
9 int ans = f(l);
10 for (int i = l + 1; i <= r; i++) ans = std::max(ans, f(i));
```

1.3 离散化

当数值范围大但实际出现的值个数有限时，把参与的值映射到连续的 $1..m$ ，适用于树状数组、扫描线等以下标为键的问题。 a 为 1 起始的数组，离散化后 $a[i]$ 变为其在去重排序后的名次（1 起始）。复杂度 $\mathcal{O}(n \log n)$ 。

```

1 // 将 a (1-indexed) 原地离散化, a[i] 变为排序去重后的名次
2 std::vector<int> xs;
3 for (int i = 1; i <= n; i++) xs.push_back(a[i]);
4 std::sort(xs.begin(), xs.end());
5 xs.erase(std::unique(xs.begin(), xs.end()), xs.end());
6 for (int i = 1; i <= n; i++)
7     a[i] = std::lower_bound(xs.begin(), xs.end(), a[i]) -
        ↪ xs.begin() + 1;

```

需要还原原值时，直接取 $xs[a[i] - 1]$ 即可；若有多个数组共用同一映射，则只建一次 xs 。

1.4 格雷码

n 的（反射）格雷码为 $g = n \oplus (n \gg 1)$ ，它把 $0..2^k - 1$ 一一映到自身，且相邻两个编码恰好有一位不同，首尾也仅一位不同（ k 维超立方体的一条哈密顿回路）。适合需要“增量式”枚举子集的问题：按格雷码顺序遍历所有子集，每步只需加入或移除一个元素，就能在 $\mathcal{O}(1)$ 内维护当前集合的聚合信息，避免每次重新计算。

`toGray(n)` 返回 n 的格雷码；`fromGray(g)` 还原为原数（互为逆映射）。`toGray` 为 $\mathcal{O}(1)$ ，`fromGray` 需按位宽做若干次异或。

```

1 template<typename T>
2 T toGray(T n){
3     return n ^ (n >> 1);
4 }
5 template<typename T>
6 T fromGray(T g){
7     T n = g;
8     for (T x = g >> 1; x; x >>= 1) n ^= x;
9     return n;
10 }

```

1.4.1 按格雷码序枚举子集

```
1 int s = 0;
2 for (int i = 1; i < (1 << k); i++){
3     int c = __builtin_ctz((unsigned)i);
4     s ^= 1 << c;
5 }
```

s 依次取遍 $\{0..k-1\}$ 的全部子集，且相邻两步的 s 只差一个元素：第 i （1 起始）步翻转的元素下标即 i 的二进制最低位位置（`__builtin_ctz(i)`）。想增量维护答案时，先判断第 c 位翻转后是加入还是移除，再据此 $\mathcal{O}(1)$ 更新。

数论

2.1 质数筛

`Sieve()` 预处理 $[0, N)$ 内每个数是否为质数，`np[i]` 为 `true` 表示 i 是合数； N 在编译期确定，一般取 10^7 量级。本机运行 50 次的平均耗时（毫秒）：

数据规模	线性筛 (ms)	埃氏筛 (ms)
2×10^6	8.354	8.812
1×10^7	41.562	44.417
2×10^7	83.388	115.250
5×10^7	205.396	308.354
1×10^8	422.438	673.562

埃氏筛实现简单，适合 N 较小（约 10^7 ）的场合：

```

1 constexpr int N = 1e7 + 3;
2 std::bitset<N> np;
3 void Sieve(){
4     np[0] = np[1] = 1;
5     for (int i = 2; i < N; i++){
6         if (np[i]) continue;
7         for (int j = i << 1; j < N; j += i){
8             np[j] = 1;
9         }
10    }
11 }
```

欧拉筛（线性筛）常数更优，并顺带得到质数表 `primes`：

```

1  constexpr int N = 1e7 + 3;
2  std::bitset<N> np;
3  std::vector<int> primes;
4  void Sieve(){
5      np[0] = np[1] = 1;
6      for (int i = 2; i < N; i++){
7          if (!np[i]) primes.push_back(i);
8          for (const int& x : primes){
9              if (x * i >= N) break;
10             np[x * i] = 1;
11             if (i % x == 0) break;
12         }
13     }
14 }

```

2.2 Miller–Rabin 素性测试

isPrime(n) 判断 64 位整数 n 是否为质数，固定基底、确定性结果。辅助函数：

- quickPower(a,p,m)：取模快速幂。
- mul(x,y,m)：借助 __int128 的不溢出取模乘法。

```

1  using i64 = long long;
2  i64 mul(const i64& x, const i64& y, const i64& m){
3      return static_cast<__int128>(x) * y % m;
4  }
5
6  i64 quickPower(i64 a, i64 p, i64 m){
7      i64 r = 1;
8      for(a %= m; p; p >>= 1, a = mul(a, a, m)) {
9          if (p & 1) r = mul(r, a, m);
10     }
11     return r;
12 }
13
14 bool isPrime(i64 n){

```

```

15     if (n == 2 || n == 3) return 1;
16     if (n % 2 == 0 || n % 3 == 0 || n < 2) return 0;
17     int p2 = __builtin_ctzll(n-1);
18     constexpr std::array<int,7> P =
        ↪ {2,325,9375,28178,450775,9780504,1795265022};
19     for (const auto& p : P){
20         if (p % n == 0) break;
21         i64 x = quickPower(p, (n-1) >> p2, n);
22         if (x == 1 || x == n-1) continue;
23         for (int i = 1; i <= p2; i++){
24             i64 y(x);
25             x = mul(x,x,n);
26             if (x == 1) {
27                 if (y != n-1) return 0;
28                 break;
29             }
30         }
31         if (x != 1) return 0;
32     }
33     return 1;
34 }

```

2.3 质因数分解 (Pollard-Rho)

前置模板: **Miller--Rabin** 素性测试。

`factorize(v)` 返回 v 的全部质因子 (可重复、升序排列), 要求 $v \geq 1$, 期望复杂度 $\mathcal{O}(v^{\frac{1}{4}})$ 。

```

1  using i64 = long long;
2
3  std::vector<i64> factorize(i64 v) {
4      static constexpr int C = 1e4;
5      std::vector<i64> p;
6      while ((v & 1) == 0) {
7          p.push_back(2);
8          v >>= 1;
9      }

```

```
10     std::function<void(i64)> PollardRho = [&](i64 n)->void {
11         if (isPrime(n)) {
12             p.push_back(n);
13             return;
14         }
15         if (n <= C) {
16             for (int i = 2; i * i <= n; i++){
17                 while (n % i == 0) {
18                     p.push_back(i);
19                     n /= i;
20                 }
21             }
22             if (n > 1) p.push_back(n);
23             return;
24         }
25         static std::mt19937_64 rng(time(0));
26         std::uniform_int_distribution<i64> uf(1,n-1);
27         i64 c = uf(rng);
28         auto f = [&](i64 x)->i64 {
29             return (static_cast<__int128>(x) * x + c) % n;
30         };
31         while (1) {
32             i64 x = 0, y = 0;
33             i64 d = 1, prod = 1;
34             int lmt = 1, cnt = 0;
35             while (d == 1) {
36                 y = f(y);
37                 prod = mul(prod, std::abs(x-y), n);
38                 ++cnt;
39                 if (cnt % 127 == 0) {
40                     d = std::gcd(prod, n);
41                     prod = 1;
42                 }
43                 if (lmt == cnt) {
44                     x = y;
45                     cnt = 0;
46                     lmt <= 1;
47                     d = std::gcd(prod, n);
```

```

48         prod = 1;
49     }
50 }
51 if (d != n) {
52     PollardRho(n/d);
53     PollardRho(d);
54     return;
55 }
56 c = uf(rng);
57 }
58 };
59 PollardRho(v);
60 std::sort(p.begin(), p.end());
61 return p;
62 }

```

2.4 拓展欧几里得算法 (exgcd)

$\text{exgcd}(a, b, x, y)$ 求 $ax + by = \gcd(a, b)$ 的一组整数解并返回 $\gcd(a, b)$ ，解写入 x, y ；模板参数可换为 `__int128` 以处理大数。

```

1  template <typename T = long long>
2  T exgcd(T a, T b, T &x, T &y) {
3      if (b == 0) return x = 1, y = 0, a;
4      T g = exgcd(b, a % b, x, y);
5      std::swap(x, y);
6      y -= a / b * x;
7      return g;
8  }

```

2.5 中国剩余定理及其拓展 (CRT & exCRT)

前置模板：拓展欧几里得算法。

$\text{crt}(n, m, p)$ 求满足 $x \equiv p_i \pmod{m_i}$ 的最小非负整数解，要求 m_i 两两互质； $\text{excrt}(n, m, p)$ 用于 m_i 不互质的情形，无解时返回 -1 。 m 为模数数组、 p 为余数数组，内部用 `__int128` 防止乘积溢出。

```

1  using i128 = __int128;
2  using i64 = long long;
3  template<typename T = long long>
4  T crt(int n, const std::vector<T> &m, const std::vector<T> &p)
    ↪ { // x%m=p
5      i128 LCM = 1, ans = 0;
6      for (int i = 0; i < n; i++) LCM *= m[i];
7      assert(LCM);
8      for (int i = 0; i < n; i++) {
9          i64 cm = LCM / m[i], x, y;
10         exgcd(cm % m[i], m[i], x, y);
11         x = (x % m[i] + m[i]) % m[i];
12         ans += p[i] * cm % LCM * x % LCM;
13         ans %= LCM;
14     }
15     return ans;
16 }
17 template<typename T>
18 T excrt(int n, const std::vector<T> &m, const std::vector<T>
    ↪ &p) {
19     i128 LCM = 1, ans = 0;
20     for (int i = 0; i < n; i++) {
21         i128 x, y, g = exgcd<i128>(LCM, m[i], x, y), c =
            ↪ ((p[i] - ans) % m[i] + m[i]) % m[i];
22         if (c % g) return -1;
23         i128 z = m[i] / g;
24         x = (x * c / g % z + z) % z; // x0
25         ans += x * LCM;
26         LCM *= z;
27     }
28     return ans;
29 }

```

2.6 BSGS 及其拓展

2.6.1 BSGS 算法

BSGS(a, b, p) 求 $a^x \equiv b \pmod{p}$ 的最小非负整数解, 无解返回 -1 , 复杂度 $\mathcal{O}(\sqrt{p})$ 。
要求 $\gcd(a, p) = 1$ (p 为素数时自然满足); a, p 不互质或 p 为合数时改用 exBSGS。

```

1  using i64 = long long;
2  i64 BSGS(i64 a, i64 b, const i64& p){ // a ^ x = b \pmod p
3      a %= p, b %= p;
4      if (b == 1) return 0;
5      const i64 t = sqrt(p);
6      std::unordered_map<long long, long long> mp;
7      i64 ap = 1;
8      for (i64 i = 0, res = b; i < t; i++){
9          mp[res] = i;
10         res = (i64)((__int128)res * a % p);
11         ap = (i64)((__int128)ap * a % p);
12     }
13     for (i64 i = t, v = ap; i < p + t; i += t, v =
14         ↪ (i64)((__int128)v * ap % p)){
15         if (mp.contains(v)) {
16             return i - mp[v];
17         }
18     }
19     return -1;

```

2.6.2 exBSGS 算法

exBSGS(a, b, p) 求 $a^x \equiv b \pmod{p}$ 的最小非负整数解, 无解返回 -1 , 复杂度约为 $\mathcal{O}(\sqrt{p})$ 。适用于 a, p 不互质、 p 为合数等 BSGS 失效的情形。

前置模板: 拓展欧几里得算法与上式 **BSGS** (递归基直接调用)。

```

1  using i64 = long long;
2  i64 exBSGS(i64 a, i64 b, const i64& p){
3      a %= p, b %= p;
4      if (b % p == 1 % p) return 0;
5      i64 d = std::gcd(a, p);
6      if (b % d != 0) return -1;
7      if (d > 1) {

```

```

8      i64 x,y;
9      i64 np = p / d;
10     exgcd(a / d,np,x,y);
11     i64 nb = (i64)(((__int128)(b / d) * x % np + np) %
    ↪ np);
12     auto ans = exBSGS(a,nb,np);
13     return ans == -1 ? -1 : ans + 1;
14 }
15 return BSGS(a,b,p);
16 }

```

2.7 欧拉函数

$\varphi(x)$ 表示 $1 \sim x$ 中与 x 互质的数的个数。

2.7.1 求单一数字 $\varphi(x)$

getPhi(x) 求单一数字的 $\varphi(x)$ ，复杂度 $\mathcal{O}(\sqrt{x})$ ：

```

1  int getPhi(int x){
2      int res = 1;
3      for (int i = 2;i <= x / i;i++){
4          if (x % i == 0){
5              res *= i-1;
6              x /= i;
7              while (x % i == 0){
8                  res *= i;
9                  x /= i;
10             }
11         }
12     }
13     if (x > 1) res *= x-1;
14     return res;
15 }

```

欧拉筛线性求 φ 数组，phi[i] 为 $\varphi(i)$ ，需先调用 Sieve()：

2.7.2 欧拉筛求 $\varphi(x)$

```

1  constexpr int N = 1E7 + 3;
2  std::bitset<N> np;
3  std::vector<int> phi, primes;
4  void Sieve(){
5      np[0] = np[1] = 1;
6      phi.resize(N, 1);
7      for (int i = 2; i < N; i++){
8          if (!np[i]) {
9              phi[i] = i-1;
10             primes.push_back(i);
11         }
12         for (const int& x : primes){
13             if (x * i >= N) break;
14             np[x * i] = 1;
15             if (i % x == 0) {
16                 phi[i * x] = phi[i] * x;
17                 break;
18             }
19             phi[x * i] *= (x - 1) * phi[i];
20         }
21     }
22 }

```

欧拉函数常用于化简 gcd 之和。由 $n = \sum_{d|n} \varphi(d)$ 取 $n = \gcd(a, b)$ 得

$$\gcd(a, b) = \sum_{d|a, d|b} \varphi(d).$$

对 i 求和并用 $\sum_{i=1}^n [d | i] = \lfloor \frac{n}{d} \rfloor$, 得到

$$\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d).$$

2.7.3 欧拉定理（欧拉降幂）

若 $\gcd(a, m) = 1$, 则 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。指数为超大数（字符串给出）时按下式降幂：

$$a^b \bmod m = \begin{cases} a^{b \bmod \varphi(m)} \bmod m, & \gcd(a, m) = 1, \\ a^{b \bmod \varphi(m) + \varphi(m)} \bmod m, & \gcd(a, m) \neq 1, b \geq \varphi(m). \end{cases}$$

2.8 原根

仅参考，实际可预先处理出常用模数的原根直接使用。

模数 n 存在原根当且仅当 $n \in \{2, 4, p^k, 2p^k\}$ (p 为奇质数)。设 g 为原根，则 g 的阶为 $\varphi(n)$ ，即最小的 d 使 $g^d \equiv 1 \pmod{n}$ 恰为 $d = \varphi(n)$ ；原根共有 $\varphi(\varphi(n))$ 个，且 g^k 也为原根当且仅当 $\gcd(k, \varphi(n)) = 1$ 。

`getPrimitiveRoot(MOD)` 返回模 MOD 的最小原根， MOD 无原根时返回 0。要求 $MOD < 2^{31}$ （内部以 `int` 处理中间量）。前置模板：取模类（全局模数）与欧拉函数。

```

1  bool hasRoot(long long MOD){
2      if (MOD == 2 || MOD == 4) return 1;
3      if (MOD % 2 == 0){
4          MOD >>= 1;
5          if (MOD % 2 == 0) return 0;
6      }
7      if (MOD == 1) return 0;
8      for (long long i = 3; i * i <= MOD; i += 2){
9          if (MOD % i == 0){
10             while (MOD % i == 0) MOD /= i;
11             return MOD == 1;
12         }
13     }
14     return 1;
15 }
16 int getPrimitiveRoot(long long MOD){
17     if (MOD == 2 || MOD == 4) return MOD - 1;
18     if (!hasRoot(MOD)) return 0;
19     arbModInt::setMod(MOD);
20     long long Phi = getPhi((int)MOD);
21     std::vector<long long> fac;
22     long long t = Phi;
23     for (long long i = 2; i * i <= t; i++){
24         if (t % i == 0){
25             fac.push_back(i);
26             while (t % i == 0) t /= i;
27         }
28     }

```

```

29     if (t > 1) fac.push_back(t);
30     for (long long g = 2; g < MOD; g++){
31         if (arbModInt::Pow(g, Phi)() != 1) continue;
32         bool ok = 1;
33         for (const long long& q : fac)
34             if (arbModInt::Pow(g, Phi/q)() == 1){ ok = 0;
35                 ↪ break; }
36         if (ok) return (int)g;
37     }
38     return 0;
39 }

```

常见质数模数的最小原根：

mod	998244353	$10^9 + 7$	$10^9 + 9$	2147483647	999999937	167772161
g	3	5	13	7	11	3

2.9 二次剩余 (Cipolla 算法)

前置模板：取模类（静态模数）与 复数。

求 $x^2 \equiv n \pmod{MOD}$ 的一个解，Quad_Restdue(n) 返回该解（ $n = 0$ 时返回 0）；其中 MOD 为 Z 对应模数 998244353，要求为奇质数且 n 是二次剩余，另一解为 $MOD - x$ 。

```

1  Z i2; // i²
2  template<>
3  struct Complex<Z>{
4      Z real, imag;
5      constexpr Complex operator*=(const Complex& x){
6          Z rl = real * x.real + imag * x.imag * i2;
7          imag = x.imag * real + x.real * imag;
8          real = rl;
9          return *this;
10     }
11     constexpr friend Complex operator*(const Complex&
12     ↪ x, const Complex y){
13         Complex r = x;
14         return (r *= y);
15     }

```

```
15     constexpr static Complex Pow(Complex x, long long p){
16         Complex res = {1,0};
17         for (;p;p >>= 1,x *= x){
18             if (p&1) res *= x;
19         }
20         return res;
21     }
22 };
23 std::mt19937_64 rnd(time(0));
24
25 Z Quad_Restdue(Z n){
26     if (n() == 0) return 0;
27     long long p = 998244353;
28     assert(Z::Pow(n,(p-1)/2)() == 1);
29     Z a = rnd() % (p-1)+1;
30     while (Z::Pow(a,(p-1)/2)() != 1 ||
31           ↪ Z::Pow(a*a-n,(p-1)/2)() == 1){
32         a = rnd() % (p-1)+1;
33     }
34     i2 = a*a-n;
35     return Complex<Z>::Pow({a,1},(p+1)/2).real;
36 }
```

生成函数与多项式卷积

3.1 快速傅里叶变换 (FFT)

需要配合 `Complex` 类使用。

计算两个整数系数多项式 A, B 的卷积（多项式乘法），复杂度 $\mathcal{O}(n \log n)$ 。

`work(A,B)` 返回 $A \cdot B$ 的系数向量（类型 `long long`），长度为 $|A| + |B| - 1$ ；对外只需调用 `work`。

注意：基于 `double`，单个系数绝对值小于约 2^{53} 时结果才精确，多次卷积还会累积浮点误差；需取模运算时改用 NTT。

```

1 namespace FFT{
2     using Comp = Complex<double>;
3     using Poly = std::vector<Comp>;
4     using i64 = long long;
5
6     std::vector<int> revl;
7     const double PI = std::acos(-1);
8     inline void FFT(const int len, Poly& arr, const short
        ↪ sign){
9         for (int i = 0; i < len; i ++){
10             if (i < revl[i]) std::swap(arr[i], arr[revl[i]]);
11         }
12         for (int p = 2; p <= len; p <= 1){
13             int mid = p >> 1;
14             Comp Wn(std::cos(PI/mid), sign*std::sin(PI/mid));
15             for (int l = 0; l < len; l += p){
16                 Comp w(1), tmp;
17                 for (int i = l, j = 0; j < mid; i ++, j ++, w *=
                    ↪ Wn){

```

```

17         tmp = arr[i+mid] * w;
18         arr[i+mid] = arr[i] - tmp;
19         arr[i] += tmp;
20     }
21 }
22 }
23 }
24 std::vector<i64> work(const std::vector<i64>& A,const
↪ std::vector<i64>& B){
25     int limit = 1,limBit = 0;
26     while (limit <= (A.size() + B.size())) limit <=<=
↪ 1,limBit ++;
27     // 蝴蝶变换
28     revl.resize(limit);
29     for (int i = 1;i < limit;i ++)
30         revl[i] = (revl[i>>1]>>1) | ((i&1)<<(limBit-1));
31     Poly poly(limit);
32     for (int i = 0;i < A.size();i ++) poly[i].real =
↪ A[i];
33     for (int i = 0;i < B.size();i ++) poly[i].imag =
↪ B[i];
34     FFT(limit,poly,1);
35     for (int i = 0;i < limit;i ++) poly[i] *= poly[i];
36     FFT(limit,poly,-1);
37     std::vector<i64> ans(A.size()+B.size()-1);
38     for (int i = 0;i < ans.size();i++) ans[i] =
↪ std::round(poly[i].imag/2/limit);
39     return ans;
40 }
41 }
42 using namespace FFT;

```

3.2 快速沃尔什变换 (FWT) 以及位运算卷积全家桶

需要 ModInt 取模类，模数固定为 998244353。

3.2.1 基础位运算卷积

适合计算以下情况：

- $F[k] = \sum_{i \oplus j = k} A_i B_j$ 。
- $F[k] = \sum_{i \cup j = k} A_i B_j$ 。
- $F[k] = \sum_{i \cap j = k} A_i B_j$ 。

时间复杂度为 $\mathcal{O}(n \log n)$ 。序列长度自动补零至 2 的幂； $|=$ 、 $\&=$ 、 $=^{\wedge}$ 分别计算 $\cup$ 、 $\cap$ 、 $\oplus$ 卷积。

```

1  struct FWT{
2      using Z = ModInt<998244353>;
3      using Poly = std::vector<Z>;
4      Poly ele;
5      int siz = 0;
6      FWT(int n){
7          while (n & (n - 1)) n += n & -n;
8          siz = n;
9          ele.resize(n);
10     }
11     Z& operator[](int idx){return ele[idx];}
12     void OR(int sign = 1){
13         for (int len = 2; len <= siz; len <<= 1){
14             int mid = len >> 1;
15             for (int l = 0; l < siz; l += len){
16                 for (int i = l, j = l + mid; j < l +
17                     ↪ len; i++, j++){
18                     ele[j] += ele[i] * sign;
19                 }
20             }
21         }
22     void AND(int sign = 1){
23         for (int len = 2; len <= siz; len <<= 1){
24             int mid = len >> 1;
25             for (int l = 0; l < siz; l += len){
26                 for (int i = l, j = l + mid; j < l +
27                     ↪ len; i++, j++){

```

```

27         ele[i] += ele[j] * sign;
28     }
29 }
30 }
31 }
32 void XOR(int sign = 1){
33     Z inv2 = ~Z(2);
34     for (int len = 2; len <= siz; len <= 1){
35         int mid = len >> 1;
36         for (int l = 0; l < siz; l += len){
37             for (int i = l, j = l + mid; j < l + len; i
38                 ↪ ++, j++){
39                 Z x = ele[i], y = ele[j];
40                 ele[i] = x+y;
41                 ele[j] = x-y;
42                 if (sign == -1) ele[i] *= inv2, ele[j] *=
43                     ↪ inv2;
44             }
45         }
46     }
47 }
48 FWT& operator|=(FWT rhs){
49     if (rhs.siz > siz) {
50         ele.resize(rhs.siz);
51         siz = rhs.siz;
52     } else if (siz > rhs.siz) {
53         rhs.ele.resize(siz);
54         rhs.siz = siz;
55     }
56     OR(1), rhs.OR(1);
57     for (int i = 0; i < siz; i++) ele[i] *= rhs[i];
58     OR(-1);
59     return *this;
60 }
61 FWT& operator&=(FWT rhs){
62     if (rhs.siz > siz) {
63         ele.resize(rhs.siz);
64         siz = rhs.siz;

```

```

63         } else if (siz > rhs.siz) {
64             rhs.ele.resize(siz);
65             rhs.siz = siz;
66         }
67         AND(1),rhs.AND(1);
68         for (int i = 0;i < siz;i++) ele[i] *= rhs[i];
69         AND(-1);
70         return *this;
71     }
72     FWT& operator^=(FWT rhs){
73         if (rhs.siz > siz) {
74             ele.resize(rhs.siz);
75             siz = rhs.siz;
76         } else if (siz > rhs.siz) {
77             rhs.ele.resize(siz);
78             rhs.siz = siz;
79         }
80         XOR(1),rhs.XOR(1);
81         for (int i = 0;i < siz;i++) ele[i] *= rhs[i];
82         XOR(-1);
83         return *this;
84     }
85     friend FWT operator|(FWT lhs,const FWT& rhs){
86         return lhs |= rhs;
87     }
88     friend FWT operator&(FWT lhs,const FWT& rhs){
89         return lhs &= rhs;
90     }
91     friend FWT operator^(FWT lhs,const FWT& rhs){
92         return lhs ^= rhs;
93     }
94 };

```

3.2.2 子集卷积

前置模板：基础位运算卷积。

用于计算 $C[k] = \sum_{i \cup j = k, i \cap j = \emptyset} A_i B_j$ ，即两子集不相交。

两序列长度可不同，时间复杂度为 $\mathcal{O}(n \log^2 n)$ 。

```

1  #define popcnt(i) __builtin_popcount(i)
2  FWT subset(FWT a,FWT b){
3      int lmt = std::__lg(std::max(a.siz,b.siz));
4      int n = 1 << lmt;
5      std::vector<FWT> PA(lmt+1,FWT(n));
6      auto PB(PA),PC(PA);
7      for (int i = 0;i < a.siz;i++) PA[popcnt(i)][i] = a[i];
8      for (int i = 0;i < b.siz;i++) PB[popcnt(i)][i] = b[i];
9      for (int i = 0;i <= lmt;i++){
10         PA[i].OR(1),PB[i].OR(1);
11     }
12     for (int i = 0;i <= lmt;i++){
13         for (int j = 0;j <= i;j++){
14             for (int k = 0;k < n;k++){
15                 PC[i][k] += PA[j][k] * PB[i-j][k];
16             }
17         }
18     }
19     for (int i = 0;i <= lmt;i++) PC[i].OR(-1);
20     FWT c(n);
21     for (int i = 0;i < n;i++) c[i] = PC[popcnt(i)][i];
22     return c;
23 }
24 #undef popcnt

```

3.3 快速数论变换 (NTT) 以及多项式全家桶

需要 `ModInt` 取模类以及对应的原根。

NTT 是 FFT 的模意义版本：用模数原根 g 代替复数单位根，全程整数运算、无浮点误差；模数 $998244353 = 119 \times 2^{23} + 1$ ，原根 $g = 3$ ，支持长度不超过 2^{23} 的卷积。以下模板全部复用 `dft` 做正/逆变换。

注意：更大长度需用不同模数的 NTT + CRT 还原。

3.3.1 DFT 与 IDFT 函数

以下所有模板均依赖 `dft` 做正/逆变换。

`dft(len,a,sign)`: 对长度 `len` (`2` 的幂) 的系数向量 `a` 做正/逆变换, `sign=1` 正、`-1` 逆, 逆变换末尾已自动除以 `len`。

```

1  constexpr int PolyMod = 998244353;
2  using MInt = ModInt<PolyMod>;
3  const MInt g = 3, gi = ~g;
4  using Poly = std::vector<MInt>;
5  using std::min;
6  using std::max;
7
8  void dft(const int& len, Poly& arr, int sign){ // sign = -1 ->
    ↪ IDFT
9      int lg2 = std::__lg(len);
10     std::vector<int> revi(len);
11     for (int i = 1; i < len; i++)
12         revi[i] = (revi[i>>1]>>1) | ((i&1)<<(lg2-1));
13     for (int i = 0; i < len; i++)
14         if (i < revi[i]) std::swap(arr[i], arr[revi[i]]);
15     for (int p = 2; p <= len; p <= 1){
16         int mid = p >> 1;
17         MInt Wn = (sign == 1 ? g : gi);
18         Wn = MInt::Pow(Wn, (PolyMod-1)/p);
19         for (int l = 0; l < len; l += p){
20             MInt w = 1, tmp;
21             for (int i = l, j = 0; j < mid; i++, j++, w *= Wn){
22                 tmp = arr[i+mid] * w;
23                 arr[i+mid] = arr[i].val - tmp;
24                 arr[i] += tmp;
25             }
26         }
27     }
28     if (sign == -1){
29         MInt inv = ~MInt(len);
30         for (int i = 0; i < arr.size(); i++) arr[i] *= inv;
31     }
32 }

```

3.3.2 多项式杂项函数

多项式求导和多项式积分

求导: $(\sum a_i x^i)' = \sum (i+1)a_{i+1}x^i$, 末项置零; 积分是其逆运算, 每项除以 i , 积分常数取 0。二者是 $\ln/\exp$ 的基础。

```

1 Poly polyDeriv(Poly f){
2     assert(f.size());
3     for (int i = 0; i < f.size()-1; i++) f[i] = f[i+1] *
        ↪ (i+1);
4     f.back() = 0;
5     return f;
6 }
7
8 Poly polyInteg(Poly f){
9     assert(f.size());
10    for (int i = f.size()-1; i >= 1; i--) f[i] = f[i-1] / i;
11    f.front() = 0;
12    return f;
13 }

```

3.3.3 多项式乘法

系数表示 $\rightarrow$ 点值逐项相乘 $\rightarrow$ 逆变换回系数, 复杂度 $\mathcal{O}(n \log n)$; limit 取大于 $|A| + |B| - 1$ 的 2 的幂。

```

1 Poly& operator*=(Poly& A, Poly B){
2     int limit = 1, siz = A.size() + B.size() - 1;
3     while (limit <= siz) limit <=< 1;
4     A.resize(limit, 0);
5     B.resize(limit, 0);
6     dft(limit, A, 1);
7     dft(limit, B, 1);
8     for (int i = 0; i < limit; i++) A[i] *= B[i];
9     dft(limit, A, -1);
10    A.resize(siz);
11    return A;
12 }

```

```

13
14 Poly polyMul(Poly A, Poly B){
15     return A *= B;
16 }

```

多项式转置乘法

`mulT(A,B)` 返回长度 $|A|$ 的系数向量，供多点求值与快速插值内部使用，一般不单独调用。依赖 多项式乘法。

```

1 Poly mulT(Poly A, Poly B){
2     const int& n = A.size(), m = B.size();
3     std::reverse(B.begin(), B.end());
4     auto F = polyMul(A, B);
5     for (int i = 0; i < n; i++) A[i] = F[i + m - 1];
6     return A;
7 }

```

3.3.4 多项式乘法逆

前置模板：多项式乘法

Newton 迭代倍增：当前逆 G （长度 i ）更新为 $G(2 - FG) \bmod x^{2i}$ 直到覆盖 n ，复杂度 $\mathcal{O}(n \log n)$ ，要求 $F[0] \neq 0$ 。

```

1 Poly polyInv(const Poly& F){
2     Poly G(1, 1 / F[0]);
3     const int& n = F.size();
4     for (int i = 1; i < n << 1; i <= 1){
5         const int& len = i << 1;
6         G.resize(len);
7         Poly F2(F.begin(), F.begin() + std::min(i, n));
8         F2.resize(len, 0);
9         dft(len, F2, 1);
10        dft(len, G, 1);
11        for (int j = 0; j < len; j++){
12            G[j] = (2 - G[j] * F2[j]) * G[j];
13        }
14        dft(len, G, -1);
15        std::fill(G.begin() + i, G.end(), 0);

```

```

16     }
17     G.resize(n);
18     return G;
19 }

```

3.3.5 多项式除法与多项式求余

前置模板：多项式乘法，多项式乘法逆。

$A = BQ + R$ ($\deg R < \deg B$): `polyDiv(A,B)` 返回商 Q , `polyMod(A,B)` 返回余 R 。要求 $\deg A \geq \deg B$, 否则商为 0、余为 A 。复杂度 $\mathcal{O}(n \log n)$ 。

```

1  Poly& operator/=(Poly& A, Poly B){
2      const int& n = A.size(), m = B.size();
3      if (n < m) { A.clear(); return A; }
4      const int& k = n - m + 1;
5      std::reverse(A.begin(), A.end());
6      std::reverse(B.begin(), B.end());
7      A.resize(k);
8      B.resize(k);
9      A *= polyInv(B);
10     A.resize(k);
11     std::reverse(A.begin(), A.end());
12     return A;
13 }
14
15 Poly polyDiv(Poly A, Poly B){
16     return A / B;
17 }
18
19 Poly& operator%=(Poly& A, Poly B){
20     const int& n = A.size(), m = B.size();
21     if (n < m) return A;
22     Poly C = polyDiv(A, B);
23     B *= C;
24     for (int i = 0; i < n; i++){
25         A[i] -= B[i];
26     }
27     A.resize(m-1);

```

```

28     return A;
29 }
30
31 Poly polyMod(Poly A, Poly B){
32     return A %= B;
33 }

```

3.3.6 多项式对数（自然底数）

需要前置模板：多项式杂项，多项式乘法，多项式乘法逆。

$\ln F = \int F'/F$ ：先求导、乘逆、再积分，要求 $F[0] = 1$ 。

```

1 Poly polyLn(const Poly& F){
2     Poly dF = polyMul(polyDeriv(F), polyInv(F));
3     dF = polyInteg(dF);
4     dF.resize(F.size()); // 截断到与输入等长：ln F 只需次数低于 |F|
5     return dF;
6 }

```

3.3.7 多项式指数函数（exp）

需要前置模板：多项式杂项，多项式乘法，多项式乘法逆，多项式对数。

Newton 迭代： $G \leftarrow G(1 - \ln G + F) \bmod x^{2i}$ ，长度倍增，要求 $F[0] = 0$ 。

```

1 Poly polyExp(const Poly& F){
2     Poly G(1,1);
3     const int& n = F.size();
4     for (int i = 2; i < n << 1; i <=& 1){
5         G.resize(i);
6         Poly F2(polyLn(G));
7         for (int j = 0; j < min(i,n); j++){
8             F2[j] = F[j] - F2[j];
9         }
10        ++F2[0];
11        G *= F2;
12    }
13    G.resize(n);
14    return G;

```

15 }

3.3.8 多项式开根

需要前置模板：多项式乘法逆。

Newton 迭代： $G \leftarrow \frac{1}{2}(G + F/G) \bmod x^{2^i}$ ， $1/G$ 用乘法逆求，长度倍增；要求 $F[0]$ 平方根非零（ $F[0] \neq 1$ 时先用 Cipolla 求二次剩余）。

```

1 Poly polySqrt(const Poly& F){
2     Poly G(1,1); // 如果 F[0] 不为 1 需要求 F[0] 的二次剩余
3     const int& n = F.size();
4     const MInt& inv2 = ~MInt(2);
5     for (int i = 2; i < n << 1; i <= 1){
6         G.resize(i);
7         Poly F2(F.begin(), F.begin()+min(i,n)), G2(G);
8         for (int j = 0; j < i; j++) G2[j] += G2[j];
9         G2 = polyInv(G2);
10        G2 *= F2;
11        G2.resize(i);
12        for (int j = 0; j < i; j++){
13            G[j] = G2[j] + G[j] * inv2;
14        }
15    }
16    G.resize(n);
17    return G;
18 }
```

3.3.9 多项式快速幂

前置模板：多项式杂项，多项式乘法，多项式乘法逆，多项式对数，多项式指数函数。

$F^k = \exp(k \ln F)$ ：先取对数乘 k 再指数还原，复杂度 $\mathcal{O}(n \log n)$ ；要求 $F[0] = 1$ （否则先提取常数项与最低次项）。

```

1 Poly polyPow(Poly F, const int& k){
2     const int& n = F.size();
3     F = polyLn(F);
4     for (int i = 0; i < n; i++){
5         F[i] *= k;

```

```

6     }
7     return polyExp(F);
8 }

```

3.3.10 多项式多点求值

前置模板：多项式转置乘法，多项式乘法，多项式乘法逆。

`mutipointsEval(F,Q)` 返回 $(F(Q_0), \dots, F(Q_{m-1}))$ ，其中 $m = |Q|$ ，即多项式 F 在多点 Q 处的取值； $|F|$ 与 m 可不同。复杂度 $\mathcal{O}(n \log^2 n)$ 。

```

1 std::vector<MInt> mutipointsEval(Poly F, std::vector<MInt> Q){
2     int n = F.size(), m = Q.size();
3     n = max(n, m);
4     F.resize(n), Q.resize(n);
5     std::vector<Poly> G(n << 2);
6     auto buildG = [&](this auto&& self, int i, int l, int
    ↪ r)->void {
7         if (l == r) {
8             G[i] = Poly(2, 1);
9             G[i][1] = -Q[l];
10            return;
11        }
12        const int& mid = l + r >> 1;
13        self(i << 1, l, mid);
14        self(i << 1 | 1, mid+1, r);
15        G[i] = polyMul(G[i << 1], G[i << 1 | 1]);
16    };
17    buildG(1, 0, n-1);
18    std::vector<MInt> ans(n);
19    auto conquer = [&](this auto&& self, int i, int l, int
    ↪ r, Poly f)->void {
20        f.resize(r-l+1);
21        if (l == r) {
22            ans[l] = f[0];
23            return;
24        }
25        const int& mid = l + r >> 1;
26        self(i << 1, l, mid, mulT(f, G[i << 1 | 1]));

```

```

27         self(i << 1 | 1, mid+1, r, mulT(f, G[i << 1]));
28     };
29     conquer(1, 0, n-1, mulT(F, polyInv(G[1])));
30     ans.resize(m);
31     return ans;
32 }

```

3.3.11 多项式快速插值

前置模板：多项式转置乘法，多项式乘法，多项式乘法逆。

`mutipointsInterp(X, Y)`: 给定互异节点 X_i 与对应取值 Y_i ($|X| = |Y| = n$)，返回过全部 (X_i, Y_i) 点、次数 $< n$ 的多项式系数向量。复杂度 $\mathcal{O}(n \log^2 n)$ 。

```

1  Poly mutipointsInterp(const std::vector<MInt>& X, const
   ↪ std::vector<MInt>& Y){
2      const int& n = X.size();
3      std::vector<Poly> G(n << 2);
4      auto buildG = [&](this auto&& self, int i, int l, int
   ↪ r)->void {
5          if (l == r) {
6              G[i] = {MInt(1), -X[l]};
7              return;
8          }
9          const int& mid = l + r >> 1;
10         self(i << 1, l, mid);
11         self(i << 1 | 1, mid+1, r);
12         G[i] = polyMul(G[i << 1], G[i << 1 | 1]);
13     };
14     buildG(1, 0, n-1);
15     Poly GInv = polyInv(G[1]);
16     std::reverse(G[1].begin(), G[1].end());
17     Poly F = polyDeriv(G[1]);
18     std::vector<MInt> dy(n);
19     auto eval = [&](this auto&& self, int i, int l, int r, Poly
   ↪ f)->void {
20         f.resize(r-l+1);
21         if (l == r) {
22             dy[l] = f[0];

```

```

23         return;
24     }
25     const int& mid = l + r >> 1;
26     self(i << 1, l, mid, mulT(f, G[i << 1 | 1]));
27     self(i << 1 | 1, mid+1, r, mulT(f, G[i << 1]));
28     std::reverse(G[i << 1].begin(), G[i << 1].end());
29     std::reverse(G[i << 1 | 1].begin(), G[i << 1 |
    ↪ 1].end());
30 };
31 eval(1, 0, n-1, mulT(F, GInv));
32 auto conquer = [&](this auto&& self, int i, int l, int
    ↪ r)->Poly {
33     if (l == r) {
34         return {Y[l] * MInt::Pow(dy[l], PolyMod-2)};
35     }
36     const int& mid = l + r >> 1;
37     Poly L = self(i << 1, l, mid);
38     Poly R = self(i << 1 | 1, mid+1, r);
39     L *= G[i << 1 | 1];
40     R *= G[i << 1];
41     if (L.size() < R.size()) L.swap(R);
42     for (int i = 0; i < R.size(); i++) L[i] += R[i];
43     return L;
44 };
45 return conquer(1, 0, n-1);
46 }

```

数值计算

4.1 拉格朗日插值

插值是一种通过已知的、离散的数据点推算一定范围内的新数据点的方法。插值法常用于函数拟合中。

对已知的 n 个点 $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$, 求次数不超过 $n-1$ 的多项式 $f(x)$ 满足 $f(x_i) = y_i$ ($i = 0, \dots, n-1$)。

Lagrange 插值的形式为:

$$f(x) = \sum_{i=1}^n y_i \cdot \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

朴素实现的时间复杂度为 $\mathcal{O}(n^2)$, 可以优化到 $\mathcal{O}(n \log^2 n)$

普通版本

```

1  template<typename T>
2  std::vector<T> Lagrange_Interpolation(std::vector<T>&
   ↪ x, std::vector<T>& y){
3      int n = x.size();
4      std::vector<T> f(n), g(n+1), px(n, 1);
5      g[0] = 1;
6      for (int i = 0; i < n; i++){
7          for (int j = i; j >= 0; j--){
8              g[j+1] += g[j];
9              g[j] *= -x[i];
10         }
11     }
12     for (int i = 0; i < n; i++){
13         for (int j = 0; j < n; j++){

```

```

14         if (i == j) continue;
15         px[i] *= x[i] - x[j];
16     }
17 }
18 for (int i = 0; i < n; i++){
19     T c = y[i] / px[i], h = g[n];
20     for (int j = n-1; j >= 0; j--){
21         f[j] += h * c;
22         h = g[j] + h * x[i];
23     }
24 }
25 return f;
26 }

```

x 是连续整数版本

优化上述式子，最终其插值式为：

$$\begin{aligned}
 f(x) &= \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x-j}{i-j} \\
 &= \sum_{i=1}^n y_i (-1)^{n-i} \frac{pre_{i-1} \cdot suf_{i+1}}{(i-1)!(n-i)!}
 \end{aligned} \tag{4.1}$$

```

1  template<typename T>
2  T Lagrange_Interpolation(std::vector<T> y, T x){ // For
    ↪ x=[1..n]
3      int n = y.size();
4      std::vector<T> fac(n+1, 1), pre(n+2), suf(n+2);
5      for (int i = 1; i <= n; i++) fac[i] = fac[i-1] * i;
6      pre[0] = suf[n+1] = 1;
7      for (int i = 1; i <= n; i++) pre[i] = pre[i-1] * (x - i);
8      for (int i = n; i >= 1; i--) suf[i] = suf[i+1] * (x - i);
9      T res = 0;
10     for (int i = 1; i <= n; i++){
11         res += y[i-1] * pre[i-1] * suf[i+1] / (fac[i-1] *
            ↪ fac[n-i]) * ((n-i) % 2 == 0 ? 1 : -1);
12     }
13     return res;

```

14 }

4.2 牛顿迭代法

核心公式为 $x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$ ，可用于计算高精度开根等。

组合数学

5.1 组合数学基础

5.1.1 排列组合基础

需要 **ModInt** 模板。

组合计数取模类, **MOD** 需为素数: $C(n, m)$ 返回 $\binom{n}{m} \bmod MOD$, n, m 可任意大 (自动 Lucas 分解)。

$A(n, m)$ 、 $fact(n)$ 、 $inv(n)$ 分别为排列数 $n!/(n-m)!$ 、 $n!$ 、 $(n!)^{-1}$, 一般要求 $n < MOD$ (更大时返回模意义下结果); 首次查询按需预计算阶乘, 单次 $\mathcal{O}(1)$ 。

```

1  template <int MOD>
2  struct Combination{
3      using Z = ModInt<MOD>;
4  private:
5      int siz = 0;
6      std::vector<Z> _fact, _inv;
7      void expend(int size){
8          if (siz >= size) return;
9          _fact.resize(size), _inv.resize(size);
10         for (int i = siz; i < size; i++) _fact[i] = _fact[i-1]
            ↪ * i;
11         _inv[size-1] = 1 / _fact[size-1];
12         for (int i = size - 2; i >= siz; i--) _inv[i] =
            ↪ _inv[i+1] * (i+1);
13         siz = size;
14     }
15     void check(int size){
16         if (siz >= size) return;

```

```

17     int n = siz;
18     while(n < size) n <= 1;
19     n = std::min<unsigned int>(n,MOD);
20     expend(n);
21 }
22 public:
23     Combination():siz(1),_fact(1,1),_inv(1,1){}
24     Z fact(int n){
25         if (n < 0) return inv(-n);
26         if (n >= MOD) return 0;
27         check(n+1);
28         return _fact[n];
29     }
30     Z inv(int n){
31         if (n < 0) return fact(-n);
32         if (n >= MOD) return 0;
33         check(n+1);
34         return _inv[n];
35     }
36     Z A(int n,int m){
37         if (n < 0 || m < 0 || n < m) return 0;
38         if (m >= MOD) return 0;
39         if (n < MOD) {
40             check(n+1);
41             return _fact[n]*_inv[n-m];
42         }
43         check(m+1);
44         return C(n,m)*_fact[m];
45     }
46     Z C(int n,int m){
47         if (n < 0 || m < 0 || n < m) return 0;
48         if (n < MOD && m < MOD) {
49             check(n+1);
50             return _fact[n] * _inv[m] * _inv[n - m];
51         } else return C(n/MOD,m/MOD)*C(n%MOD,m%MOD);
52     }
53 };

```

5.1.2 卡特兰数

通常情况下有四种形式：

1.

$$C_n = \begin{cases} 1 & , n = 0 \\ \sum_{i=0}^{n-1} C_i C_{n-i-1} & , n > 0 \end{cases}$$

2.

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{n!(n+1)!}$$

3.

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}$$

4.

$$C_n = \frac{4n-2}{n+1} C_{n-1}, n > 0, C_0 = 1$$

代码为片段：Z 为全局取模类型，C(n,m) 为组合数查询函数（如 Combination 对象的 C 成员）。

```

1 Z Catalan(int n){
2     if (n < 0) return 0;
3     if (n == 0) return 1;
4     return C(2*n,n) - C(2*n,n-1);
5 }

```

5.1.3 卢卡斯定理

对于素数 p ，有 $\binom{n}{k} \equiv \binom{\lfloor n/p \rfloor}{\lfloor k/p \rfloor} \binom{n \bmod p}{k \bmod p} \pmod{p}$ 。其中，当 $n < k$ 时，二项式系数 $\binom{n}{k}$ 规定为 0。

5.2 二项式定理与二项式反演

5.2.1 二项式定理

$$(a+b)^n = \sum_{i=0}^n \binom{n}{i} a^i b^{n-i}$$

5.2.2 二项式反演

通常情况下，二项式反演有以下四种模板演化。

$$g(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} f(i) \iff f(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} g(i)$$

$$g(n) = \sum_{i=0}^n \binom{n}{i} f(i) \iff f(n) = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} g(i)$$

$$g(n) = \sum_{i=n}^N (-1)^i \binom{i}{n} f(i) \iff f(n) = \sum_{i=n}^N (-1)^i \binom{i}{n} g(i)$$

$$g(n) = \sum_{i=n}^N \binom{i}{n} f(i) \iff f(n) = \sum_{i=n}^N (-1)^{i-n} \binom{i}{n} g(i)$$

线性代数

6.1 矩阵

用于矩阵运算与矩阵快速幂，常见于线性递推求项、图上路径计数等需要大量矩阵乘法的场景。`Pow(a,k)` (a^k) 要求 a 为方阵且指数 $k \geq 0$; `normalize()` 将矩阵置为单位阵。

```

1  using std::vector;
2  template<typename T,typename Mul =
    ↪ std::multiplies<T>,typename Add = std::plus<T>>
3  struct Matrix{
4      int N,M;
5      vector<vector<T>> value;
6      Matrix(int n,int m):N(n),M(m),value(vector<vector<T>>(n,
    ↪ vector<T>(m))){}
7      Matrix(int n):N(n),M(n),value(vector<vector<T>>(n,vector
    ↪ <T>(n))){}
8      Matrix(const vector<T>&
    ↪ v):N(1),M(v.size()),value(vector<vector<T>>(1,v)){}
9      Matrix(const vector<vector<T>>&
    ↪ v):value(v),N(v.size()),M(v[0].size()){}
10 void normalize(){
11     assert(N==M);
12     for (int i = 0;i < N;i++){
13         for (int j = 0;j < N;j++){
14             value[i][j] = T(i == j);
15         }
16     }
17 }
```

```

18     void fill(const T& val){
19         std::fill(value.begin(),value.end(),vector<T>(M,val),
20             ↪ );
21     }
22     vector<T>& operator[](int idx){
23         return value[idx];
24     }
25     Matrix& operator+=(const Matrix &x){
26         assert(N==x.N&&M==x.M);
27         for (int i = 0; i < N; ++i){
28             for (int j = 0; j < M; ++j){
29                 value[i][j] =
30                     ↪ Add()(value[i][j],x.value[i][j]);
31             }
32         }
33         return *this;
34     }
35     friend Matrix operator+(Matrix x,const Matrix &y){
36         return x += y;
37     }
38     friend Matrix operator*(const Matrix& a,const Matrix& b){
39         assert(a.M == b.N);
40         Matrix r(a.N,b.M);
41         for (int i = 0; i < a.N; ++i){
42             for (int k = 0; k < a.M; ++k){
43                 for (int j = 0; j < b.M; ++j){
44                     r.value[i][j] = Add()(r.value[i][j],Mul(
45                         ↪ )(a.value[i][k],b.value[k][j]));
46                 }
47             }
48         }
49         return r;
50     }
51     Matrix& operator *=(const Matrix &x){
52         Matrix r = *this*x;
53         std::swap(value,r.value);
54         return *this;

```

```

53     }
54     static Matrix Pow(const Matrix& a, long long p){
55         assert(a.N == a.M);
56         assert(p >= 0);
57         Matrix r(a.N), tmp = a;
58         r.normalize();
59         for (; p; p >>= 1, tmp *= tmp) if (p&1) r *= tmp;
60         return r;
61     }
62     Matrix operator^(long long p){
63         return Pow(*this, p);
64     }
65     Matrix& operator^=(long long p){
66         *this = Pow(*this, p);
67         return *this;
68     }
69
70     friend std::istream& operator>>(std::istream& is, Matrix&
    ↪ m){
71         for (int i = 0; i < m.N; ++i){
72             for (int j = 0; j < m.M; ++j){
73                 is >> m.value[i][j];
74             }
75         }
76         return is;
77     }
78     friend std::ostream& operator<<(std::ostream& os, const
    ↪ Matrix& m){
79         for (int i = 0; i < m.N; ++i){
80             for (int j = 0; j < m.M; ++j){
81                 os << m.value[i][j] << " \n"[j==m.M-1];
82             }
83         }
84         return os;
85     }
86 };

```

6.2 高斯消元

6.2.1 求解线性方程组

`Gauss::work(e)` 求解线性方程组：传入 n 行 $n+1$ 列的增广矩阵 `e`（最后一列为常数项），函数会修改该矩阵。

返回 `0` 表示有唯一解，此时 `e[i][n]` 即为 x_i ；`1` 表示无穷多解；`-1` 表示无解。

```

1 namespace Gauss{
2     using f64 = long double;
3     using std::vector;
4     constexpr f64 eps = 1E-6;
5     int work(vector<vector<f64>> &e){
6         assert(e.size()+1 == e[0].size());
7         int n = e.size(), m = 0;
8         for (int i = 0; i < n; i++){
9             int mx = m;
10            for (int j = m+1; j < n; j++){
11                if (fabsl(e[j][i]) > fabsl(e[mx][i])) mx = j;
12            }
13            if (fabsl(e[mx][i]) <= eps) continue;
14            else swap(e[mx], e[m]);
15            f64 d = 1 / e[m][i];
16            for (int j = 0; j <= n; j++) e[m][j] *= d;
17            assert(fabsl(e[m][i]-1) <= eps);
18            for (int j = 0; j < n; j++) {
19                if (j == m || fabsl(e[j][i]) <= eps)
20                    ⇨ continue;
21                f64 k = e[j][i];
22                for (int p = 0; p <= n; p++){
23                    e[j][p] -= k * e[m][p];
24                }
25                m++;
26            }
27            if (m == n) return 0;
28            else {
29                while(m < n){

```

```

30         if (fabsl(e[m++][n]) > eps) return -1;
31     }
32     return 1;
33 }
34 }
35 };

```

6.2.2 计算行列式

`Guass::det(R)` 求 n 阶方阵 R 的行列式并返回；取模版对模数无素数性要求，浮点版 $O(n^3)$ 且需注意数值误差。需要取模类 `ModInt (Z)`。

```

1 namespace Guass {
2     using MInt = Z;
3     using std::vector;
4     MInt det(vector<vector<MInt>> R){
5         const int n = R.size();
6         MInt w = 1;
7         for (int i = 0; i < n; i++){
8             assert(R[i].size() == n);
9             for (int j = i + 1; j < n; j++){
10                 while (R[i][i] != 0) {
11                     int div = R[j][i]() / R[i][i]();
12                     for (int k = 0; k < n; k++) R[j][k] -=
13                         ↪ R[i][k] * div;
14                     swap(R[j], R[i]);
15                     w = -w;
16                 }
17                 swap(R[i], R[j]);
18                 w = -w;
19             }
20         }
21         MInt ans = 1;
22         for (int i = 0; i < n; i++) ans *= R[i][i];
23         return ans * w;
24     }
25     constexpr double eps = 1e-8;
26     double det(vector<vector<double>> R){

```

```

26     const int n = R.size();
27     double w = 1;
28     for (int i = 0; i < n; i++){
29         assert(R[i].size() == n);
30         for (int j = i+1; j < n; j++){
31             if (abs(R[j][i]) > abs(R[i][i])) {
32                 swap(R[i], R[j]);
33                 w = -w;
34             }
35         }
36         if (abs(R[i][i]) <= eps) return 0;
37         double d = R[i][i];
38         w *= d;
39         for (int j = i; j < n; j++) R[i][j] /= d;
40         for (int j = i+1; j < n; j++){
41             if (abs(R[i][i]) <= eps) return 0;
42             double t = R[j][i];
43             for (int k = i; k < n; k++){
44                 R[j][k] -= t * R[i][k];
45             }
46         }
47     }
48     double ans = 1;
49     for (int i = 0; i < n; i++) ans *= R[i][i];
50     return ans * w;
51 }
52 };

```

6.2.3 求矩阵逆

需要取模类 `ModInt (Z)`。

`Guass::inverse(R)` 求方阵 `R` 的逆矩阵并返回，不可逆时返回空 `vector`。

```

1 namespace Guass {
2     using MInt = Z;
3     using std::vector;
4     vector<vector<MInt>> inverse(const vector<vector<MInt>>&
    ↪ R){

```

```
5      const int n = R.size();
6      vector<vector<MInt>> A(n,vector<MInt>(2*n));
7      for (int i = 0;i < n;i++){
8          assert(R[i].size() == n);
9          for (int j = 0;j < n;j++){
10             A[i][j] = R[i][j];
11         }
12         A[i][i+n] = 1;
13     }
14     vector<vector<MInt>> I;
15     for (int i = 0;i < n;i++){
16         for (int j = i;j < n;j++){
17             if (A[j][i] != 0) {
18                 swap(A[i],A[j]);
19                 break;
20             }
21         }
22         if (A[i][i] == 0) return I;
23         MInt inv = 1 / A[i][i];
24         for (int k = 0;k < 2*n;k++) A[i][k] *= inv;
25         for (int j = 0;j < n;j++){
26             if (i == j) continue;
27             MInt x = A[j][i];
28             for (int k = 0;k < 2*n;k++){
29                 A[j][k] -= A[i][k] * x;
30             }
31         }
32     }
33     I.assign(n,vector<MInt>(n));
34     for (int i = 0;i < n;i++){
35         for (int j = 0;j < n;j++){
36             I[i][j] = A[i][j+n];
37         }
38     }
39     return I;
40 }
41 };
```

6.3 线性基

用于维护一组数的线性基，可求子集最大异或和、判断某数能否由集合内异或表示、求基的秩等。`template<int N>` 为最高可处理的二进制位（默认 30，可支持 $\leq 2^{31}-1$ 的整数）。

`insert(x)` 插入 `x`，返回 `true` 表示新增了独立基向量，`false` 表示 `x` 可由现有基表示。`operator[](i)` 取第 `i` 位的基向量，`count()` 返回基的秩，`operator==` 判断两个基是否等价（可表示的异或空间相同）。

```

1  template<int N = 30>
2  struct Basis{
3      using i64 = long long;
4  private:
5      std::array<i64,N+1> b{};
6      int cnt = 0;
7  public:
8      Basis() {
9          std::fill(b.begin(),b.end(),0);
10     }
11     bool insert(i64 x) {
12         if (!x) return 0;
13         for (int i = N;i >= 0;i--){
14             if (!(x >> i & 1)) continue;
15             if (!b[i]) {
16                 b[i] = x;
17                 for (int j = N;j >= 0;j--){
18                     if (i == j || !b[j]) continue;
19                     if (b[j] >> i & 1) b[j] ^= x;
20                 }
21                 ++cnt;
22                 return 1;
23             } else x ^= b[i];
24         }
25         return 0;
26     }
27     i64 operator[](const int& i){return b[i];}
28     i64 operator[](const int& i) const {return b[i];}
29     bool operator==(const Basis& rhs) const {

```

```
30         if (cnt  $\neq$  rhs.cnt) return 0;
31         Basis t = *this;
32         for (int i = N; i >= 0; i--)
33             if (rhs.b[i]) t.insert(rhs.b[i]);
34         return t.cnt == cnt;
35     }
36     int count() const {
37         return cnt;
38     }
39 };
```

计算几何

7.1 二维计算几何

7.1.1 点

`Point<T>`: 二维点, T 常取 `double`。重载加减 (向量)、数乘、比较 (需 C++20, 按 x 后 y 排序) 与流式 IO; `dot` 点积、`cross` 叉积、`square` 长度平方、`length` 模长、`normalize` 单位向量。`normalize` 建议仅用于浮点 T ; 对零向量调用 (或 `operator/` 除数为 0) 会触发 `assert`。

```

1  template<typename T = double>
2  struct Point{
3      T x,y;
4      Point(T X = 0,T Y = 0):x(X),y(Y){}
5      constexpr auto operator<=>(const Point& b) const { //
6          ↪ require C++20
7              if (x == b.x) return y <= b.y;
8              return x <= b.x;
9      }
10     constexpr bool operator==(const Point& b) const =
11         ↪ default;
12     constexpr Point& operator+=(const Point& b){
13         x += b.x,y += b.y;
14         return *this;
15     }
16     constexpr Point& operator-=(const Point& b){
17         x -= b.x,y -= b.y;
18         return *this;
19     }

```

```

18     constexpr Point& operator*=(const T lambda){
19         x *= lambda,y *= lambda;
20         return *this;
21     }
22     constexpr Point& operator/=(const T lambda){
23         x /= lambda,y /= lambda;
24         return *this;
25     }
26     constexpr Point operator-(){
27         return Point(-x,-y);
28     }
29     friend Point operator+(const Point& a,const Point& b){
30         return Point(a.x + b.x,a.y + b.y);
31     }
32     friend Point operator-(const Point& a,const Point& b){
33         return Point(a.x - b.x,a.y - b.y);
34     }
35     friend Point operator*(const Point& a,const T& lambda){
36         return Point(a.x * lambda,a.y * lambda);
37     }
38     friend Point operator*(const T& lambda,const Point& a){
39         return a * lambda;
40     }
41     friend Point operator/(const Point& a,const T& lambda){
42         assert(lambda != 0);
43         return Point(a.x / lambda,a.y / lambda);
44     }
45     // Vector Operations
46     static T dot(const Point& a,const Point& b){
47         return a.x * b.x + a.y * b.y;
48     }
49     static T cross(const Point& a,const Point& b){
50         return a.x * b.y - a.y * b.x;
51     }
52     static T square(const Point& a){
53         return dot(a,a);
54     }
55     double length() const { // Euclidean norm

```

```

56         return std::sqrt(dot(*this,*this));
57     }
58     Point normalize() const { // unit vector
59         return *this / length();
60     }
61     // STL
62     friend std::istream& operator>>(std::istream& is,Point
    ↪ &p){
63         return is >> p.x >> p.y;
64     }
65     friend std::ostream& operator<<(std::ostream& os,const
    ↪ Point &p){
66         return os << p.x << ' ' << p.y;
67     }
68 };

```

7.1.2 线

前置模板：点。Col(a,b,c)：判断三点共线；onSeg(a,b,p)：判断 p 是否在线段 ab 上（含端点）。二者均用精确 = 判定、无 eps：整型坐标可靠，浮点坐标建议按需自行加容差。均为 $O(1)$ 。

```

1  template<typename T = double>
2  bool Col(const Point<T> &a,const Point<T> &b,const Point<T>
    ↪ &c){//三点共线
3      return Point<T>::cross( b - a , c - a ) == 0 ;
4  }
5
6  template<typename T = double>
7  bool onSeg(const Point<T>& a, const Point<T>& b, const
    ↪ Point<T>& p) {//点在线段上
8      return Point<T>::cross(b - a, p - a) == 0 &&
9          std::min(a.x,b.x) <= p.x && p.x <= std::max(a.x,b.x) &&
10         std::min(a.y,b.y) <= p.y && p.y <= std::max(a.y,b.y);
11 }

```

7.1.3 简单多边形

判断点是否在多边形内

时间复杂度为 $\mathcal{O}(n)$ 。

前置模板：点、线。`inPolygon(a,p)`： a 为按边界顺序给出的简单多边形（可凹、不可自交）， p 在内部或边界上时返回 `true`。判定用精确比较，浮点坐标靠近边界可能误判，建议使用整型坐标。

```

1  template<typename T = double>
2  bool inPolygon(const std::vector<Point<T>>& a, const
   ↪ Point<T>& p) {
3      int n = (int)a.size();
4      bool ans = 0;
5      for (int i = 0, j = n - 1; i < n; j = i++) {
6          if (onSeg(a[i],a[j],p)) return true;
7          if ((a[i].y > p.y) != (a[j].y > p.y)) {
8              Point<T> del1 = a[j] - a[i];
9              Point<T> del2 = p - a[i];
10             T l = del1.y * del2.x;
11             T r = del1.x * del2.y;
12             if ((del1.y > 0 && l < r) || (del1.y < 0 && l >
   ↪ r)) {
13                 ans = !ans;
14             }
15         }
16     }
17     return ans;
18 }
```

7.1.4 凸包

前置模板：点。`Hull::Andrew(points)`：对点集求凸包，返回 (hi, lo) ， hi 、 lo 分别为上、下凸壳从左到右的顶点链，均只含凸包顶点（重复点与共线点已剔除）。最左、最右 x 各恰有一个点时两链共享两 endpoints；否则（存在竖直边）端点分属该竖直边的上、下端。拼出凸包： lo 正向接反向 hi 并去掉重复端点。复杂度 $\mathcal{O}(n \log n)$ 。

```

1  namespace Hull{
2      template<typename T>
```

```

3     auto Andrew(std::vector<Point<T>> points,int s = 0){
4         using P = Point<T>;
5         const int n = points.size() - 1;
6         std::vector<P> hi,lo;
7         std::sort(points.begin() + s,points.end());
8         for (int i = s;i <= n;i ++){
9             P p = points[i];
10            while (hi.size() > 1 && P::cross(hi.back() -
    ↪ hi[hi.size() - 2],p - hi.back()) >= 0){
11                hi.pop_back();
12            }
13            if (hi.size() && hi.back().x == p.x){
14                hi.pop_back();
15            }
16            hi.push_back(p);
17            while (lo.size() > 1 && P::cross(lo.back() -
    ↪ lo[lo.size() - 2],p - lo.back()) <= 0){
18                lo.pop_back();
19            }
20            if (!lo.size() || lo.back().x < p.x)
    ↪ lo.push_back(p);
21        }
22        return std::make_pair(hi,lo); // high hull and low
    ↪ hull
23    }
24 };

```

7.1.5 多边形面积与凸多边形面积交

前置模板：点。**polygonArea(a)**： a 为按边界顺序给出的任意简单多边形（可凹、不自交），返回其面积， $\mathcal{O}(n)$ 。**polygonIntersectionArea(A,B)**：两凸多边形交的面积； A,B 顶点顺序任意（内部统一转为逆时针），面积以 **long double** 返回；空交或仅边界/顶点接触时返回 0。复杂度 $\mathcal{O}(|A| \cdot |B|)$ 。

```

1  template<typename T>
2  long double polygonArea(const std::vector<Point<T>>& a){
3      long double s = 0;

```

```

4     for (int i = 0, j = (int)a.size() - 1; i <
      ↪ (int)a.size(); j = i++)
5         s += (long double)a[j].x * a[i].y - (long
      ↪ double)a[j].y * a[i].x;
6     return std::abs(s) / 2;
7 }
8
9 template<typename T>
10 long double polygonIntersectionArea(std::vector<Point<T>> A,
  ↪ std::vector<Point<T>> B){
11     using F = long double;
12     struct Pt { F x, y; };
13     auto conv = [](const auto& p){
14         std::vector<Pt> r;
15         for (const auto& q : p) r.push_back({(F)q.x,
      ↪ (F)q.y});
16         return r;
17     };
18     auto signedA = [](std::vector<Pt>& p){
19         F s = 0;
20         for (int i = 0; i < (int)p.size(); i++){
21             Pt a = p[i], b = p[(i + 1) % (int)p.size()];
22             s += a.x * b.y - a.y * b.x;
23         }
24         return s;
25     };
26     std::vector<Pt> C = conv(A), D = conv(B);
27     if ((int)C.size() < 3 || (int)D.size() < 3) return 0;
28     if (signedA(C) < 0) std::reverse(C.begin(), C.end());
29     if (signedA(D) < 0) std::reverse(D.begin(), D.end());
30     auto inter = [](Pt u, Pt v, Pt a, Pt b){
31         F dx = b.x - a.x, dy = b.y - a.y;
32         F s = ((a.x - u.x) * dy - (a.y - u.y) * dx) /
33             ((v.x - u.x) * dy - (v.y - u.y) * dx);
34         return Pt{u.x + s * (v.x - u.x), u.y + s * (v.y -
      ↪ u.y)};
35     };
36     std::vector<Pt> cur = C;

```

```

37     for (int i = 0; i < (int)D.size() && !cur.empty(); i++){
38         Pt a = D[i], b = D[(i + 1) % (int)D.size()];
39         std::vector<Pt> nxt;
40         for (int k = 0; k < (int)cur.size(); k++){
41             Pt u = cur[k], v = cur[(k + 1) %
42                 ↪ (int)cur.size()];
43             F inU = (b.x - a.x) * (u.y - a.y) - (b.y - a.y)
44                 ↪ * (u.x - a.x);
45             F inV = (b.x - a.x) * (v.y - a.y) - (b.y - a.y)
46                 ↪ * (v.x - a.x);
47             if (inU * inV < 0) nxt.push_back(inter(u, v, a,
48                 ↪ b));
49             if (inV >= 0) nxt.push_back(v);
50         }
51         cur = nxt;
52     }
53     return cur.size() < 3 ? 0 : std::abs(signedA(cur)) / 2;
54 }

```

7.1.6 曼哈顿与切比雪夫距离

前置模板：点。曼哈顿距离 $M = |x_1 - x_2| + |y_1 - y_2|$ ，切比雪夫距离 $C = \max(|x_1 - x_2|, |y_1 - y_2|)$ 。由 $|a| + |b| = \max(|a + b|, |a - b|)$ ，把点 (x, y) 视作 $(x + y, x - y)$ 后，原曼哈顿距离恰为新坐标的切比雪夫距离；反向还原为 $(\frac{u+v}{2}, \frac{u-v}{2})$ 。常用于把斜线可达性（切比雪夫）问题转成矩形/曼哈顿形式，整数场景下 $(x + y, x - y)$ 的奇偶性一致，结果整体除以 2 仍为整数。

`manhattan(a, b)`、`chebyshev(a, b)`：两点间距离。`maxManhattan(a)`：点集内最大点对曼哈顿距离；`maxChebyshev(a)`：点集内最大点对切比雪夫距离。均 $\mathcal{O}(n)$ ， a 为空或单点时返回 0。

```

1  using i64 = long long;
2  template<typename T>
3  i64 manhattan(const Point<T>& a, const Point<T>& b){
4      return std::abs((i64)a.x - b.x) + std::abs((i64)a.y -
5          ↪ b.y);
6  }
7  template<typename T>
8  i64 chebyshev(const Point<T>& a, const Point<T>& b){

```

```

8     i64 dx = std::abs((i64)a.x - b.x);
9     i64 dy = std::abs((i64)a.y - b.y);
10    return std::max(dx, dy);
11 }
12 template<typename T>
13 i64 maxManhattan(const std::vector<Point<T>>& p){
14     if ((int)p.size() < 2) return 0;
15     const i64 INF = (i64)4e18;
16     i64 a1 = INF, a2 = -INF, b1 = INF, b2 = -INF;
17     for (const auto& q : p){
18         i64 u = (i64)q.x + q.y;
19         i64 v = (i64)q.x - q.y;
20         a1 = std::min(a1, u); a2 = std::max(a2, u);
21         b1 = std::min(b1, v); b2 = std::max(b2, v);
22     }
23     return std::max(a2 - a1, b2 - b1);
24 }
25 template<typename T>
26 i64 maxChebyshev(const std::vector<Point<T>>& p){
27     if ((int)p.size() < 2) return 0;
28     const i64 INF = (i64)4e18;
29     i64 a1 = INF, a2 = -INF, b1 = INF, b2 = -INF;
30     for (const auto& q : p){
31         a1 = std::min(a1, (i64)q.x); a2 = std::max(a2,
32             ↪ (i64)q.x);
33         b1 = std::min(b1, (i64)q.y); b2 = std::max(b2,
34             ↪ (i64)q.y);
35     }
36     return std::max(a2 - a1, b2 - b1);
37 }

```

7.1.7 极角与三角函数

弧度制工具。**Polar(x,y)** 坐标转极角, 值域 $[0, 2\pi)$ (原点会 **assert**)。toRadian 把角度制转为弧度制。**getAngle(rad1,rad2)**: 求从 $rad1$ 逆时针转到 $rad2$ 的夹角 (弧度, $[0, 2\pi)$ 内), 两参数需先化为 $[0, 2\pi)$ 。

arcLength(r,a)、**sectorArea(r,a)** 分别返回半径 r 、角 a 的圆弧长与扇形面积, a 缺省为整圆 2π 。

```
1 namespace Trigonometric{ // 三角函数
2     using f64 = long double;
3     const f64 PI = std::acos(-1.0);
4     f64 toRadian(f64 angle){ // 角度制转弧度制
5         return PI*angle/180;
6     }
7     f64 Polar(f64 x,f64 y){ // 坐标转极角 (弧度制), 值域 [0,2PI)
8         assert(x || y); // 坐标原点不存在极角
9         if (x == 0) return (y > 0 ? 0.5 : 1.5) * PI;
10        if (y == 0) return (x > 0 ? 0.0 : 1.0) * PI;
11        f64 ATAN = std::atan(y/x);
12        if (ATAN < 0) ATAN += PI;
13        return (y > 0 ? 0 : PI) + ATAN;
14    }
15    f64 getAngle(f64 rad1,f64 rad2){ // x1->x2 求逆时针夹角弧度
16        return rad2 - rad1 + (rad1 > rad2 ? 2*PI : 0);
17    }
18    f64 arcLength(f64 radius,f64 angle = 2*PI){ //
19        ↪ 求指定角度(弧度制)和半径的弧长
20        return angle*radius;
21    }
22    f64 sectorArea(f64 radius,f64 angle = 2*PI){ //
23        ↪ 扇形面积(弧度制)
24        return arcLength(radius,angle)*radius/2;
25    }
26 }
```

数据结构

8.1 并查集

8.1.1 普通版本

单次操作均摊时间复杂度为 $\alpha(n)$ ，其中 α 为反阿克曼函数，在 10^6 量级下值大约为 6。

`Disjoint` 用整数 $0 \dots n$ 各建独立集合（各元素为代表）。调用 `d(u,v)`（重载 `operator()`）合并 u 、 v 所在集合，合并后新集合的代表取第一个参数一侧的原代表；`d[x]`（重载 `operator[]`）返回 x 所在集合的代表值，`size(x)` 返回其大小，`init(n)` 可重建。适合需要“代表值”而不仅是判连通的合并场景（如按起点方向合并相邻连通块后询问块首）。

```

1  struct Disjoint{
2      std::vector<int> fa,siz,top;
3      int cnt=0;
4      Disjoint(int n = 0){
5          init(n);
6      }
7      void init(int n = 0){
8          fa.resize(n+1);
9          top.resize(n+1);
10         siz.assign(n+1,1);
11         std::iota(fa.begin(),fa.end(),0);
12         std::iota(top.begin(),top.end(),0);
13         cnt = n;
14     }
15     void operator()(int head,int ele){ // merge
16         head = root(head),ele = root(ele);

```

```

17     if (head == ele) return;
18     top[ele] = top[head];
19     if (siz[head] < siz[ele]) std::swap(head,ele);
20     siz[head] += siz[ele];
21     fa[ele] = head;
22 }
23 int operator[](int index){return top[root(index)];}
24 int size(int x){return siz[root(x)];}
25 private:
26     int root(int x){
27         return fa[x] = fa[x] == x ? x : root(fa[x]);
28     }
29 };

```

8.1.2 可撤销版本

支持按顺序撤销合并的版本（离线分治、动态图连通性等场景）。 $d(x,y)$ 合并并返回是否真的发生了合并； $d[x]$ 返回 x 所在集合的根； $undo(cnt)$ 撤销最近 cnt 次确实生效的合并。按需回滚而不做路径压缩、只按大小合并，单次合并 $\mathcal{O}(\log n)$ ， $undo$ 代价 $\mathcal{O}(cnt)$ 。

```

1 struct DisjointWithUndo{
2     std::vector<int> siz,fa;
3     std::vector<std::array<int,2>> tm; // fa,son
4     DisjointWithUndo(int n = 0){init(n);}
5     void init(int n){
6         siz = std::vector<int>(n+1,1), fa =
7         ↪ std::vector<int>(n+1), tm.clear();
8         std::iota(fa.begin(),fa.end(),0);
9     }
10    bool operator()(int x,int y){ // 合并
11        x = root(x), y = root(y);
12        if (x == y) return false;
13        if (siz[x] < siz[y]) std::swap(x,y);
14        tm.push_back({x,y});
15        siz[x] += siz[y], fa[y] = x;
16        return true;
17    }

```

```

17     int operator[](int index){return root(index);}
18     void undo(int cnt){ // 撤销
19         while(cnt--){
20             if (tm.empty()) break;
21             auto [x,y] = tm.back();
22             tm.pop_back();
23             siz[x] -= siz[y];
24             fa[y] = x;
25         }
26     }
27 private:
28     int root(int x){
29         while(x≠fa[x]) x=fa[x];
30         return x;
31     }
32 };

```

8.2 ST 表

对静态序列的区间查询， $\odot$ 需满足结合律且可重复贡献（幂等， $A \odot A = A$ ），如 $\min / \max / \gcd$ ；区间和等不幂等的运算不可用（两块区间覆盖部分会被重复贡献）。预处理 $\mathcal{O}(n \log n)$ ，单次查询 $\mathcal{O}(1)$ 。

```

1  template<typename T,typename Opt>
2  struct SparseTable {
3  private:
4      std::vector<std::vector<T>> st;
5      Opt op;
6  public:
7      SparseTable() = default;
8      SparseTable(const std::vector<T>& ele, int beg, Opt _op)
9          : op(std::move(_op)) {
10         build(ele, beg);
11     }
12     void build(const std::vector<T>& ele, int beg) {
13         int n = ele.size() - 1;
14         int len = n - beg + 1;

```

```

14     if (len <= 0) return;
15     int p = __lg(len);
16     st.assign(n + 1, std::vector<T>(p + 1));
17     for (int i = beg; i <= n; i++) st[i][0] = ele[i];
18     for (int j = 1; j <= p; j++) {
19         for (int i = beg; i + (1 << j) - 1 <= n; i++) {
20             st[i][j] = op(st[i][j - 1], st[i + (1 << (j
                ↪ - 1))][j - 1]);
21         }
22     }
23 }
24 T operator()(int l, int r) const {
25     int p = __lg(r - l + 1);
26     return op(st[l][p], st[r - (1 << p) + 1][p]);
27 }
28 };

```

类 `SparseTable` 以 $[beg, n]$ ($n = \text{数组长度} - 1, beg$ 前下标不参与) 为有效区间建表: 构造函数接收元素数组、 beg 与结合运算 `op` (如 `std::min`、`std::max`、`std::gcd`), 构造即建表; 对同一 `op` 更换数组可再调 `build` 重建。用对象 t 做 $t(l, r)$ 查询 (重载了 `operator()`) 返回 $[l, r]$ 的运算结果, 要求 $beg \leq l \leq r \leq n$ 。

8.3 树状数组

8.3.1 普通树状数组

对下标 $1..n$ 的数组做单点修改与前缀和查询 (区间和由两次前缀和相减得到)。构造 `Fenwick(n)` 得到全零数组。

`add(i, x)` 在位置 i 加 x , `remove(i, x)` 在位置 i 减 x ; `presum(i)` (重载的 `[i]`) 返回前缀 $[1, i]$ 之和。单次 $\mathcal{O}(\log n)$, 值类型 T 需支持加减与默认零值。

```

1  template<typename T>
2  struct Fenwick{
3      std::vector<T> value;
4      int size = 0;
5      Fenwick(int n = 0):size(n+1){
6          value.assign(size, T(0));
7      }

```

```

8     void add(int i,T x){
9         for (;i < size;i += i&(-i)) value[i] += x;
10    }
11    void remove(int i,T x){
12        for (;i < size;i += i&(-i)) value[i] -= x;
13    }
14    T presum(int i) const {
15        T ans = 0;
16        for (;i;i -= i & (-i)) ans += value[i];
17        return ans;
18    }
19    T operator[](const int i) const {
20        return presum(i);
21    }
22 };

```

8.4 线段树

8.4.1 静态线段树

支持区间修改、区间查询。Info 与 Tag 需按题目自实现，区间端点 l, r 由树自动维护。

Info 存区间聚合值，需重载 `operator+` 合并两个相邻子区间、提供 `apply(Tag)` 作整段修改；Tag 为懒标记，需提供 `apply(Tag)` 叠加标记，默认构造表示无待下传标记。

构造 `SegmentTree(n)` 得全零树，或传下标 $1..n$ （大小 $n+1$ ）的初值数组建树。`modify(l,r,tag)` 区间整体修改；`seg(l,r)` 返回 $[l, r]$ 的聚合值，越界下标自动夹到 $[1, n]$ 。单次 $\mathcal{O}(\log n)$ 。

```

1  template<typename Info,typename Tag>
2  struct SegmentTree{
3  private:
4      int _n = 0;
5      std::vector<Info> _info;
6      std::vector<Tag> _tag;
7  public:
8      SegmentTree(int n = 0){
9          rebuild(n);

```

```

10     }
11     SegmentTree(const std::vector<Info>& v){
12         assert(v.size() >= 1);
13         rebuild(v.size() - 1,v);
14     }
15     void rebuild(int n,const std::vector<Info>& v =
16         ↪ std::vector<Info>(0)){
17         _n = n;
18         _info.assign((_n + 2) << 2,Info());
19         _tag.assign((_n + 2) << 2,Tag());
20         if (n > 0) build(1,1,n,v);
21     }
22 private:
23     void build(int i,int l,int r,const std::vector<Info>& v){
24         if(l == r){
25             if (v.size() > l) _info[i] = v[l];
26             resize(_info[i],l,r);
27             return;
28         }
29         int mid = l + r >> 1;
30         build(i << 1,l,mid,v);
31         build(i << 1 | 1,mid + 1,r,v);
32         push_up(i,l,r);
33     }
34     void resize(Info& info,int l,int r){
35         info.l = l;
36         info.r = r;
37     }
38     void push_down(int i){
39         _info[i << 1].apply(_tag[i]);
40         _info[i << 1 | 1].apply(_tag[i]);
41         _tag[i << 1].apply(_tag[i]);
42         _tag[i << 1 | 1].apply(_tag[i]);
43         _tag[i] = Tag();
44     }
45     void push_up(int i,int l,int r){
46         _info[i] = _info[i << 1] + _info[i << 1 | 1];
47         resize(_info[i],l,r);

```

```

47     }
48     void modify(int i,const Tag& val,int l,int r,int ql,int
    ↪ qr){
49         if (ql <= l && r <= qr) {
50             _info[i].apply(val);
51             _tag[i].apply(val);
52             return;
53         }
54         push_down(i);
55         int mid = l + r >> 1;
56         if (ql <= mid) modify(i << 1, val, l, mid, ql, qr);
57         if (qr > mid) modify(i << 1 | 1, val, mid+1, r, ql,
    ↪ qr);
58         push_up(i,l,r);
59     }
60     Info ask(int i,int l,int r,int ql,int qr){
61         if (ql <= l && r <= qr) {
62             return _info[i];
63         }
64         push_down(i);
65         int mid = l + r >> 1;
66         if (qr <= mid) return ask(i << 1, l, mid, ql, qr);
67         else if (ql > mid) return ask(i << 1 | 1, mid+1, r,
    ↪ ql, qr);
68         Info res = ask(i << 1, l, mid, ql, qr);
69         res = res + ask(i << 1 | 1, mid+1, r, ql, qr);
70         resize(res,std::max(ql,l),std::min(qr,r));
71         return res;
72     }
73 public:
74     void modify(int l,int r,const Tag& val){
75         if (l > r) return;
76         modify(1,val,1,_n,std::max(1,l),std::min(r,_n));
77     }
78     Info operator()(int l,int r){
79         if (l > r) return Info();
80         return ask(1,1,_n,std::max(1,l),std::min(r,_n));
81     }

```

```

82 };
83
84 struct Tag{
85     Tag(){}
86     void apply(const Tag& t){
87
88     }
89 };
90
91 struct Info{
92     int l = 1, r = 0;
93
94     Info(){}
95     void apply(const Tag& t){
96
97     }
98
99     friend Info operator+(const Info& lhs, const Info& rhs){
100         Info res;
101         return res;
102     }
103 };

```

8.4.2 开点线段树

值域 $[1, n]$ 可极大（如 10^{18} ）或整体稀疏时使用，只给被访问到的区间分配结点，单次修改/查询都只新建 $\mathcal{O}(\log n)$ 个结点。Info、Tag 的实现要求与静态版相同；构造 `SegmentTree<Info, Tag>(n)` 后，`modify(l, r, tag)` 与 `seg(l, r)` 的语义同静态版， n 的类型为 `long long`。

```

1  template<typename Info, typename Tag>
2  struct SegmentTree{
3  private:
4      long long _n = 0;
5      int _cur = 0, _root = 0;
6      std::vector<Info> _info;
7      std::vector<Tag> _tag;
8      std::vector<int> _lc, _rc;

```

```

9
10 void resize(Info& info, long long l, long long r){
11     info.l = l;
12     info.r = r;
13 }
14 int new_node(long long l, long long r){
15     _cur++;
16     _info.push_back(Info());
17     _tag.push_back(Tag());
18     _lc.push_back(-1);
19     _rc.push_back(-1);
20     resize(_info[_cur], l, r);
21     return _cur;
22 }
23 void push_down(int u, long long l, long long r, long long
    ↪ mid) {
24     if(_lc[u]<0){
25         _lc[u] = new_node(l, mid);
26     }
27     if(_rc[u]<0){
28         _rc[u] = new_node(mid + 1, r);
29     }
30     _info[_lc[u]].apply(_tag[u]);
31     _tag[_lc[u]].apply(_tag[u]);
32     _info[_rc[u]].apply(_tag[u]);
33     _tag[_rc[u]].apply(_tag[u]);
34     _tag[u] = Tag();
35 }
36 void push_up(int u, long long l, long long r){
37     _info[u] = _info[_lc[u]] + _info[_rc[u]];
38     resize(_info[u], l, r);
39 }
40 void modify(int u, const Tag& val, long long l, long long
    ↪ r, long long ql, long long qr){
41     if (u<0) u = new_node(l, r);
42     if (ql <= l && r <= qr) {
43         _info[u].apply(val);
44         _tag[u].apply(val);

```

```

45         return;
46     }
47     long long mid = l + r >> 1;
48     push_down(u,l,r,mid);
49     if (ql <= mid) modify(_lc[u], val, l, mid, ql, qr);
50     if (qr > mid) modify(_rc[u], val, mid+1, r,ql, qr);
51     push_up(u,l,r);
52 }
53 Info ask(int u,long long l,long long r,long long ql,long
    ↪ long qr){
54     if (u<0) u = new_node(l,r);
55     if (ql <= l && r <= qr) {
56         return _info[u];
57     }
58     long long mid = l + r >> 1;
59     push_down(u,l,r,mid);
60     if (qr <= mid) return ask(_lc[u], l, mid, ql, qr);
61     else if (ql > mid) return ask(_rc[u], mid+1, r, ql,
        ↪ qr);
62     Info res = ask(_lc[u], l, mid, ql, qr);
63     res = res + ask(_rc[u], mid+1, r, ql, qr);
64     resize(res,std::max(ql,l),std::min(qr,r));
65     return res;
66 }
67 public:
68     SegmentTree(long long n){
69         _cur = _root = -1;
70         _n = n;
71         _root = new_node(1,_n);
72     }
73     void modify(long long l,long long r,const Tag& val){
74         if (l > r) return;
75         modify(_root,val,1,_n,l,r);
76     }
77     Info operator()(long long l,long long r){
78         if (l > r) return Info();
79         return ask(_root,1,_n,l,r);
80     }

```

```

81 };
82
83 struct Tag{
84
85     Tag(){}
86     void apply(const Tag& t){
87
88     }
89 };
90
91 struct Info{
92     long long l = 1, r = 0;
93
94     Info(){}
95
96     void apply(const Tag& t){
97
98     }
99
100     friend Info operator+(const Info& lhs, const Info& rhs){
101         Info res;
102
103         return res;
104     }
105 };

```

8.4.3 可持久化线段树

支持保留历史版本, 各版本可独立查询、继续修改。0 号版本为 `rebuild(n)` 建立的初始版本; `modify(ver, l, r, tag)` 以第 `ver` 个版本为基础修改并返回新版本编号 (旧版本不变), 之后用返回的编号继续操作即形成版本链。 `seg(ver, l, r) (operator())` 查询第 `ver` 个版本的 $[l, r]$ 。该实现不做标记永久化, 查询时沿路径累计标记。时间与空间复杂度均为 $\mathcal{O}((n + q) \log n)$ 。

```

1  template<typename Info, typename Tag>
2  struct PersistentSegmentTree {
3      int _n = 0;
4      int _cur = 0;

```

```

5     std::vector<int> roots;
6     std::vector<Info> _info;
7     std::vector<Tag> _tag;
8     std::vector<int> _lc;
9     std::vector<int> _rc;
10    void resize(Info& info,int l,int r){
11        info.l = l;
12        info.r = r;
13    }
14    int new_node(int u,int l=-1,int r=-1){
15        int v = _cur++;
16        _info.emplace_back(_info[u]);
17        _tag.emplace_back(_tag[u]);
18        _lc.push_back(u>0?_lc[u]:0);
19        _rc.push_back(u>0?_rc[u]:0);
20        if(u==0)resize(_info[v],l,r);
21        return v;
22    }
23    void push_down(int u,int l,int r,int mid) {
24        _lc[u] = new_node(_lc[u],l,mid);
25        _rc[u] = new_node(_rc[u],mid+1,r);
26        _info[_lc[u]].apply(_tag[u]);
27        _tag[_lc[u]].apply(_tag[u]);
28        _info[_rc[u]].apply(_tag[u]);
29        _tag[_rc[u]].apply(_tag[u]);
30        _tag[u] = Tag();
31    }
32    void push_up(int u){
33        _info[u] = _info[_lc[u]] + _info[_rc[u]];
34    }
35    void modify(int u,const Tag& val,int l,int r,int ql,int
    ⇐  qr){
36        int v = u;
37        if (ql <= l && r <= qr) {
38            _info[v].apply(val);
39            _tag[v].apply(val);
40            return;
41        }

```

```

42         int mid = l + r >> 1;
43         push_down(v,l,r,mid);
44         if (ql <= mid)modify(_lc[v],val,l,mid,ql,q);
45         if (qr > mid)modify(_rc[v],val,mid+1,r,ql,q);
46         push_up(v);
47         resize(_info[v],l,r);
48         return;
49     }
50     Info ask(int u,Tag val,int l,int r,int ql,int qr){
51         if (u == 0) {
52             Info res;
53             resize(res,std::max(ql,l),std::min(qr,r));
54             res.apply(val);
55             return res;
56         }
57         if (ql <= l && r <= qr) {
58             Info res = _info[u];
59             res.apply(val);
60             return res;
61         }
62         val.apply(_tag[u]);
63         int mid = l + r >> 1;
64         if (qr <= mid) {
65             return ask(_lc[u],val,l,mid,ql,q);
66         } else if (ql > mid) {
67             return ask(_rc[u],val,mid+1,r,ql,q);
68         } else {
69             Info left = ask(_lc[u],val,l,mid,ql,q);
70             Info right = ask(_rc[u],val,mid+1,r,ql,q);
71             Info res = left + right;
72             resize(res,std::max(ql,l),std::min(qr,r));
73             return res;
74         }
75     }
76 public:
77     PersistentSegmentTree(int n = 0){
78         rebuild(n);
79     }

```

```

80     void rebuild(int n){
81         _n = n;
82         _cur = 1;
83         roots.clear();
84         _info.assign(1,Info());
85         _tag.assign(1,Tag());
86         _lc.assign(1,0);
87         _rc.assign(1,0);
88         roots.push_back(new_node(0,1,_n));
89     }
90     int modify(int ver,int l,int r,const Tag& val){
91         if (l > r) return ver;
92         int new_root = new_node(roots[ver]);
93         modify(new_root,val,1,_n,l,r);
94         roots.push_back(new_root);
95         return (int)roots.size()-1;
96     }
97     Info operator()(int ver,int l,int r){
98         if (l > r) return Info();
99         return ask(roots[ver],Tag(),1,_n,l,r);
100    }
101 };
102 struct Tag{
103
104     Tag(){}
105     void apply(const Tag& t){
106
107     }
108 };
109 struct Info{
110     int l = 1,r = 0;
111
112     Info(){}
113     void apply(const Tag& t){
114     }
115     friend Info operator+(const Info& lhs,const Info& rhs){
116         Info res;
117         return res;

```

```

118     }
119 };

```

8.5 平衡树

本模板中所有平衡树均为多重集版本，如果存在多个相同值时，删除操作仅删除其中一个。

8.5.1 笛卡尔树

构建时间复杂度为 $\mathcal{O}(n)$ 。

由下标区间 $[s, n)$ 的值建笛卡尔树：中序遍历恰为下标顺序，同时满足堆性质。堆方向由比较器 `CMP` 决定——默认 `std::less` 建出“大根”树（根为区间内值最大的下标，父值 $\geq$ 子值）；要建满足 $a_{\text{parent}(u)} < a_u$ 的小根笛卡尔树则传入 `std::greater<T>`。

返回 $\text{bst}[u] = \{l, r\}$ ：下标 u 结点的左/右儿子下标，无则 -1 ；左儿子下标总小于 u 、右儿子大于 u （中序即下标序）。整棵树的根是唯一不为他人儿子的下标，即区间的最大（或最小）值所在位置。常见用途：区间 RMQ 转 LCA、笛卡尔树 DP、最大子矩形等。

```

1  using p32 = std::array<int,2>;
2  template<typename T,typename CMP = std::less<T>>
3  std::vector<p32> Cartesian(std::vector<T>& arr,int s = 0,CMP
    ↪ compare = CMP()){
4      std::vector<int> stk;
5      std::vector<p32> bst(arr.size(),{-1,-1});
6      for (int i = s;i < arr.size();i++){
7          while(stk.size() && compare(arr[stk.back()],arr[i])){
8              bst[i][0] = stk.back();
9              stk.pop_back();
10         }
11         if (stk.size()) bst[stk.back()][1] = i;
12         stk.push_back(i);
13     }
14     return bst;
15 }

```

8.5.2 Splay 树

对大小为 n 的 Splay 树做 m 次伸展操作的复杂度是 $\mathcal{O}((n+m)\log n)$ 的，单次均摊复杂度是 $\mathcal{O}(\log n)$ 的。

```

1  template <typename T = long long>
2  struct Splay{
3      using u32 = unsigned int;
4      std::vector<T> key;
5      u32 head = 0;
6      std::vector<u32> fa,siz,h;
7      std::vector<std::array<u32,2>> son;
8      Splay(){
9          newNode();
10         _siz = 0;
11     }
12     u32 rank(T x){
13         u32 rk = 0,u = head,lst = 0;
14         while (u) {
15             lst = u;
16             if (key[u] < x) {
17                 rk += siz[son[u][0]] + 1;
18             }
19             u = son[u][key[u] < x];
20         }
21         splay(lst,0);
22         return rk+1;
23     }
24     void insert(T x){
25         if (head == 0) {
26             u32 v = newNode();
27             head = v;
28             key[v] = x;
29             update(v);
30         } else {
31             u32 lst,i,u = head;
32             while (u) {
33                 lst = u;
34                 i = !(x < key[u]);

```

```

35         u = son[u][i];
36     }
37     u32 v = newNode();
38     son[lst][i] = v;
39     fa[v] = lst;
40     key[v] = x;
41     update(v);
42     update(lst);
43     splay(v,0);
44 }
45 }
46 void remove(T x){
47     u32 rk = rank(x);
48     if (rk == rank(x+1)) return;
49     u32 i = find(rk);
50     splay(i,0);
51     if (son[i][0] == 0 || son[i][1] == 0) {
52         u32 j = son[i][0] == 0;
53         head = son[i][j];
54         delNode(i);
55     } else {
56         u32 j = find(rk+1);
57         assert(son[j][0] == 0); // self check
58         splay(j,i);
59         assert(son[i][1] == j); // self check
60         if (son[i][0]) fa[son[i][0]] = j;
61         son[j][0] = son[i][0];
62         delNode(i);
63         update(j);
64         head = j;
65     }
66     fa[head] = 0;
67 }
68 T index(u32 rk) {
69     u32 i = find(rk);
70     splay(i,0);
71     return key[i];
72 }

```

```
73 // STL-like:
74 u32 size() const {
75     return _siz;
76 }
77 u32 height() const {
78     return h[head];
79 }
80 u32 count(T x) {
81     return rank(x+1) - rank(x);
82 }
83 bool contains(T x) {
84     u32 u = head, lst = 0;
85     bool ans = 0;
86     while (u) {
87         lst = u;
88         if (key[u] == x) {
89             ans = 1;
90             break;
91         }
92         u32 i = key[u] < x;
93         u = son[u][i];
94     }
95     splay(lst, 0);
96     return ans;
97 }
98
99 private:
100 u32 _siz = 0;
101 std::vector<u32> del;
102 u32 newNode(){
103     u32 cur = 0;
104     if (del.size()){
105         cur = del.back();
106         del.pop_back();
107     } else {
108         cur = key.size();
109         key.emplace_back(0);
110         fa.emplace_back(0);
```

```
111         siz.emplace_back(0);
112         h.emplace_back(0);
113         son.emplace_back(std::array<u32,2>({0,0}));
114     }
115     _siz++;
116     return cur;
117 }
118 void delNode(u32 i){
119     del.push_back(i);
120     siz[i] = 0;
121     fa[i] = 0;
122     son[i][0] = son[i][1] = 0;
123     h[i] = 0;
124     _siz--;
125 }
126 u32 find(u32 rank) const {
127     u32 i = head;
128     while (i){
129         if (siz[son[i][0]] + 1 == rank){
130             return i;
131         }
132         if (siz[son[i][0]] + 1 < rank){
133             rank -= siz[son[i][0]] + 1;
134             i = son[i][1];
135         } else {
136             i = son[i][0];
137         }
138     }
139     return 0;
140 }
141 void update(u32 i){
142     if (i <= 0) return;
143     siz[i] = siz[son[i][0]] + siz[son[i][1]] + 1;
144     h[i] = std::max(h[son[i][0]],h[son[i][1]]) + 1;
145 }
146 u32 dir(u32 u) const {
147     return son[fa[u]][1] == u;
148 }
```

```

149 void rotate(u32 u){
150     u32 f = fa[u],gf = fa[f];
151     u32 i = dir(u);
152     son[f][i] = son[u][!i];
153     if (son[f][i]) fa[son[f][i]] = f;
154     son[u][!i] = f;
155     if (gf) {
156         u32 gi = dir(f);
157         son[gf][gi] = u;
158     }
159     fa[f] = u;
160     fa[u] = gf;
161     update(f);
162     update(u);
163 }
164 void splay(u32 u,u32 rt){
165     if (u == 0) return;
166     u32 f = fa[u];
167     while (f != rt) {
168         if (fa[f] != rt) {
169             rotate(dir(f) == dir(u) ? f : u);
170         }
171         rotate(u);
172         f = fa[u];
173     }
174     if (!rt) head = u;
175 }
176 };

```

8.5.3 FHQ Treap

Treap 是一种弱平衡的随机化二叉搜索树，这里给出的是无旋（FHQ）版本。视作平衡多重集使用：元素可重复， T 默认为 `long long`，其余类型需可比较（小于与判等）。

各操作单次期望复杂度 $\mathcal{O}(\log n)$ ：**insert(x)** 插入一个 x ；**remove(x)** 删除一个等于 x 的元素（不存在则无事发生）；**rank(x)** 返回小于 x 的元素个数 +1；**index(k)** 返回第 k 小元素（下标从 1 开始）；**contains(x)**、**size()** 分别判断存在性与元素个数。

普通版本

```

1  std::mt19937 _rnd(std::chrono::high_resolution_clock::now().
    ↪ time_since_epoch().count());
2  template<typename T = long long>
3  struct FHQTreap {
4      using u32 = unsigned int;
5      using _KV = std::pair<T,u32>;
6      u32 head = 0;
7      std::vector<_KV> key;
8      std::vector<u32> siz;
9      std::vector<std::array<u32,2>> son;
10     FHQTreap(){
11         newNode();
12         _siz = 0;
13     }
14     void insert(const T& x){
15         u32 a,b;
16         split(head,{x,0},a,b);
17         u32 u = newNode();
18         key[u].first = x;
19         siz[u] = 1;
20         head = merge(merge(a,u),b);
21     }
22     void remove(const T& x){
23         u32 a,b,c;
24         split(head,{x,0},a,b);
25         split(b,{x,-1},b,c);
26         if (b == 0) { head = merge(a,c); return; }
27         head = merge(merge(a,merge(son[b][0],son[b][1])),c);
28         delNode(b);
29     }
30     u32 rank(const T& x) const {
31         u32 rk = 0,u = head;
32         while (u) {
33             bool flag = key[u].first < x;
34             if (flag) {
35                 rk += siz[son[u][0]] + 1;

```

```

36         }
37         u = son[u][flag];
38     }
39     return rk + 1;
40 }
41 bool contains(const T& x) const {
42     u32 rk = 0, u = head;
43     while (u) {
44         bool flag = key[u].first < x;
45         if (!flag && key[u].first == x) {
46             return 1;
47         }
48         u = son[u][flag];
49     }
50     return 0;
51 }
52 const T& index(u32 rank) const {
53     return key[find(rank)].first;
54 }
55 u32 size() const {
56     return _siz;
57 }
58 private:
59     u32 _siz = 0;
60     std::vector<u32> del;
61     u32 newNode(){
62         u32 cur = 0;
63         if (del.size()) {
64             cur = del.back();
65             del.pop_back();
66         } else {
67             u32 rx = _rnd();
68             while (rx == 0 || rx == -1) rx = _rnd();
69             cur = key.size();
70             key.emplace_back(std::make_pair(T(), rx));
71             siz.emplace_back(0);
72             son.emplace_back(std::array<u32, 2>({0, 0}));
73         }

```

```

74     _siz++;
75     return cur;
76 }
77 void delNode(u32 i){
78     del.push_back(i);
79     siz[i] = 0;
80     son[i][0] = son[i][1] = 0;
81     _siz--;
82 }
83 void update(u32 i){
84     if (i == 0) return;
85     siz[i] = siz[son[i][0]] + siz[son[i][1]] + 1;
86 }
87 void split(u32 u, const _KV& mid, u32& x, u32& y){
88     if (u == 0) x = y = 0;
89     else {
90         if (mid < key[u]) {
91             y = u;
92             split(son[u][0], mid, x, son[y][0]);
93         } else {
94             x = u;
95             split(son[u][1], mid, son[x][1], y);
96         }
97         update(u);
98     }
99 }
100 u32 merge(u32 a, u32 b){
101     if (a == 0 || b == 0) return a | b;
102     if (key[a].second < key[b].second) {
103         son[a][1] = merge(son[a][1], b);
104         update(a);
105         return a;
106     } else {
107         son[b][0] = merge(a, son[b][0]);
108         update(b);
109         return b;
110     }
111 }

```

```

112     u32 find(u32 rk) const {
113         u32 u = head;
114         while (u) {
115             if (siz[son[u][0]] + 1 == rk) return u;
116             if (siz[son[u][0]] + 1 < rk) {
117                 rk -= siz[son[u][0]] + 1;
118                 u = son[u][1];
119             } else {
120                 u = son[u][0];
121             }
122         }
123         return 0;
124     }
125 };

```

可持久化版本

每个版本是独立不可变的快照，旧版本不受影响。第 0 个版本为初始空树；此后每次调用 `insert/remove(x,t)` 以第 t 个版本为基础生成并追加一个新版本（其编号等于调用前已有的版本总数），可随时查询或继续修改。

只读操作 `rank(x,t)`、`index(k,t)`、`contains(x,t)` 均作用于第 t 个版本。`clearVersion()` 令当前最新版本成为新的 0 号版本并丢弃其余历史。单次操作时间与额外空间均期望 $\mathcal{O}(\log n)$ 。

```

1  std::mt19937 _rnd(std::chrono::high_resolution_clock::now().
    ↪ time_since_epoch().count());
2  template<typename T = long long>
3  struct PersistentFHQTreap {
4      using u32 = unsigned int;
5      using _KV = std::pair<T,u32>;
6      std::vector<_KV> key;
7      std::vector<u32> siz,ver;
8      std::vector<std::array<u32,2>> son;
9      PersistentFHQTreap(){
10         newNode(0);
11         _siz = 0;
12         ver.push_back(0);
13     }

```

```
14     u32 copyVersion(u32 t){
15         ver.push_back(ver[t]);
16         return ver.back();
17     }
18     void clearVersion(){
19         ver = {ver.back()};
20     }
21     void insert(const T& x,u32 t){
22         u32 a,b,rt = copyVersion(t);
23         split(rt,{x,0},a,b);
24         u32 u = newNode();
25         key[u].first = x;
26         siz[u] = 1;
27         rt = merge(merge(a,u),b);
28         ver.back() = rt;
29     }
30     void remove(const T& x,u32 t){
31         u32 a,b,c,rt = copyVersion(t);
32         split(rt,{x,0},a,b);
33         split(b,{x,-1},b,c);
34         rt = merge(merge(a,merge(son[b][0],son[b][1])),c);
35         ver.back() = rt;
36     }
37     u32 rank(const T& x,u32 t) const {
38         u32 rk = 0,u = ver[t];
39         while (u) {
40             bool flag = key[u].first < x;
41             if (flag) {
42                 rk += siz[son[u][0]] + 1;
43             }
44             u = son[u][flag];
45         }
46         return rk + 1;
47     }
48     bool contains(const T& x,u32 t) const {
49         u32 rk = 0,u = ver[t];
50         while (u) {
51             bool flag = key[u].first < x;
```

```

52         if (!flag && key[u].first == x) {
53             return 1;
54         }
55         u = son[u][flag];
56     }
57     return 0;
58 }
59 const T& index(u32 rank, u32 t) const {
60     return key[find(rank, t)].first;
61 }
62 private:
63     u32 _siz = 0;
64     u32 newNode(bool tag = 1){
65         u32 cur = 0, rx = 0;
66         while (tag && (rx == 0 || rx == -1)) rx = _rnd();
67         cur = key.size();
68         key.emplace_back(std::make_pair(T(), rx));
69         siz.emplace_back(0);
70         son.emplace_back(std::array<u32, 2>({0, 0}));
71         _siz++;
72         return cur;
73     }
74     u32 copyNode(u32 i){
75         u32 u = newNode(0);
76         key[u] = key[i];
77         siz[u] = siz[i], son[u] = son[i];
78         return u;
79     }
80     void update(u32 i){
81         if (i == 0) return;
82         siz[i] = siz[son[i][0]] + siz[son[i][1]] + 1;
83     }
84     void split(u32 u, const _KV& mid, u32& x, u32& y){
85         if (u == 0) x = y = 0;
86         else {
87             u32 tmp;
88             if (mid < key[u]) {
89                 y = copyNode(u);

```

```

90         split(son[u][0],mid,x,tmp);
91         son[y][0] = tmp;
92         update(y);
93     } else {
94         x = copyNode(u);
95         split(son[u][1],mid,tmp,y);
96         son[x][1] = tmp;
97         update(x);
98     }
99 }
100 }
101 u32 merge(u32 a,u32 b){
102     if (a == 0 || b == 0) return a | b;
103     if (key[a].second < key[b].second) {
104         u32 rt = copyNode(a);
105         son[rt][1] = merge(son[rt][1],b);
106         update(rt);
107         return rt;
108     } else {
109         u32 rt = copyNode(b);
110         son[rt][0] = merge(a,son[rt][0]);
111         update(rt);
112         return rt;
113     }
114 }
115 u32 find(u32 rk,u32 t) const {
116     u32 u = ver[t];
117     while (u) {
118         if (siz[son[u][0]] + 1 == rk) return u;
119         if (siz[son[u][0]] + 1 < rk) {
120             rk -= siz[son[u][0]] + 1;
121             u = son[u][1];
122         } else {
123             u = son[u][0];
124         }
125     }
126     return 0;
127 }

```

128 `};`

8.6 莫队算法

离线处理一类区间询问：若当前区间 $[L, R]$ 的答案能在 $\mathcal{O}(1)$ 内平移到相邻区间（只需把单个位置 p 移入或移出），则把所有询问排序后暴力移动左右指针即可。使用前需自行实现 `add(p)` 与 `del(p)`：分别把位置 p 移入、移出当前区间，并同步维护答案变量 `now`；被 `add` 维护的信息必须能由 `del` 可逆地删除。数组与区间下标均为 1 起始。

8.6.1 普通莫队

询问存于 `qs`，元素为 $\{l, r, id\}$ ($l \leq r$)。按左端点分块、块内右端点单调，并以奇偶块切换右端点的升降方向以减少指针空移。

q 与 n 同阶时总复杂度 $\mathcal{O}((n+q)\sqrt{n})$ 。

```

1  int B = std::max(1, (int)(n / std::sqrt((double)std::max(1,
    ↪ (int)qs.size()))));
2  std::sort(qs.begin(), qs.end(), [&](const auto& x, const
    ↪ auto& y) {
3      int bx = x[0] / B, by = y[0] / B;
4      if (bx != by) return bx < by;
5      return (bx & 1) ? x[1] > y[1] : x[1] < y[1];
6  });
7  std::vector<int> ans(qs.size());
8  int L = 1, R = 0;
9  for (const auto& [l, r, id] : qs){
10     while (L > l) add(--L);
11     while (R < r) add(++R);
12     while (L < l) del(L++);
13     while (R > r) del(R--);
14     ans[id] = now;
15 }
```

8.6.2 带修莫队

额外支持若干次把 $a[p]$ 改为 v 的单点修改。`qs` 元素为 $\{l, r, t, id\}$ ， t 表示“此询问前已应用的修改次数”；`mods` 元素为 $\{p, v\}$ ，第 k (0 起始) 个修改把 $a[p]$ 变

为 v , a 为当前数组。时间指针前进/后退遇到修改时, 若 p 恰在当前区间内, 先 `del` 旧值再 `add` 新值以同步答案 (代码用 `swap` 记录原值以便回退)。

q 与修改次数均同 n 阶、块长取 $n^{2/3}$ 时, 复杂度 $\mathcal{O}(n^{5/3})$ 。

```

1  int L = 1, R = 0, cur = 0;
2  auto moveTime = [&](int& cur, int goal){
3      while (cur < goal){
4          auto& m = mods[cur];
5          if (L <= m[0] && m[0] <= R) del(m[0]);
6          std::swap(a[m[0]], m[1]);
7          if (L <= m[0] && m[0] <= R) add(m[0]);
8          ++cur;
9      }
10     while (cur > goal){
11         --cur;
12         auto& m = mods[cur];
13         if (L <= m[0] && m[0] <= R) del(m[0]);
14         std::swap(a[m[0]], m[1]);
15         if (L <= m[0] && m[0] <= R) add(m[0]);
16     }
17 };
18 int B = std::max(1, (int)std::pow((double)n, 2.0 / 3.0));
19 std::sort(qs.begin(), qs.end(), [&](const auto& x, const
    ↪ auto& y) {
20     if (x[0] / B != y[0] / B) return x[0] / B < y[0] / B;
21     if (x[1] / B != y[1] / B) return x[1] / B < y[1] / B;
22     return x[2] < y[2];
23 });
24 std::vector<int> ans(qs.size());
25 for (const auto& q : qs){
26     moveTime(cur, q[2]);
27     while (L > q[0]) add(--L);
28     while (R < q[1]) add(++R);
29     while (L < q[0]) del(L--);
30     while (R > q[1]) del(R--);
31     ans[q[3]] = now;
32 }

```

树上问题

9.1 最近公共祖先

9.1.1 $\mathcal{O}(1)$ LCA

使用片段：需上下文已有以 1 为根的树邻接表 $G(1..n)$ 与前置模板 SparseTable。段内定义 dfn、dep、fa、seq 与查询函数 lca(x,y)。

预处理 $\mathcal{O}(n \log n)$ ，单次查询 $\mathcal{O}(1)$ 。

```

1  vector<int> dfn(n+1),dep(n+1),fa(n+1),seq(1);
2  int dfnt = 0;
3  auto dfs = [&](this auto&& dfs,int u,int pa) -> void {
4      dfn[u] = ++dfnt;
5      dep[u] = dep[pa] + 1,fa[u] = pa;
6      seq.push_back(u);
7      for (int v : G[u]){
8          if (v == pa) continue;
9          dfs(v,u);
10     }
11 };
12 dfs(1,1);
13 SparseTable st(seq, 1,[&](int x,int y) -> int {
14     return dep[x] <= dep[y] ? x : y;
15 });
16 auto lca = [&](int x,int y) -> int {
17     if (x == y) return x;
18     x = dfn[x],y = dfn[y];
19     if (x > y) swap(x,y);
20     return fa[st(x+1,y)];

```

21 };

9.1.2 树链剖分求 LCA

使用片段：需上下文已有以 1 为根的树邻接表 $G(1..n)$ ；段内定义 dep 、 fa 、 son 、 top 、 dfn 与查询函数 $lca(x,y)$ 。该版本为重链剖分。

预处理 $\mathcal{O}(n)$ ，单次查询时间复杂度为 $\mathcal{O}(\log n)$ 。

```

1  vector<int> siz(n+1,1),dep(n+1),fa(n+1),son(n+1),top(n+1),dfn
   ↪  n(n+1),seq(1);
2  auto dfs1 = [&](this auto&& dfs,int u,int pa)->void {
3      fa[u] = pa;
4      dep[u] = dep[pa] + 1;
5      int mx = 0;
6      for (int v : G[u]){
7          if (v == pa) continue;
8          dfs(v,u);
9          siz[u] += siz[v];
10         if (mx < siz[v]){
11             son[u] = v;
12             mx = siz[v];
13         }
14     }
15 };
16 int tick = 0;
17 auto dfs2 = [&](this auto&& dfs,int u,int tp)->void {
18     dfn[u] = ++tick;
19     seq.push_back(u);
20     top[u] = tp;
21     if (son[u]) {
22         dfs2(son[u],tp);
23     }
24     for (int v : G[u]){
25         if (v == fa[u] || v == son[u]) continue;
26         dfs2(v,v);
27     }
28 };
29 dfs1(1,1);

```

```

30 dfs2(1,1);
31 auto lca = [&](int x,int y) -> int {
32     while (top[x] != top[y]) {
33         if (dep[top[x]] > dep[top[y]]) swap(x,y);
34         y = fa[top[y]];
35     }
36     return dep[x] > dep[y] ? y : x;
37 };

```

9.2 虚树

当若干询问各自只涉及规模为 k ($k \ll n$) 的点集 S 、且要在树上按父子关系做 DP 时，抽取 S 及其两两 LCA 组成虚树，边数不超过 $2k - 1$ ，在虚树上做同样转移即可。

需已有以 1 为根的原树： n 为点数， $dfn[u]$ 为 DFS 进入序（保证子树连续即可）， $dep[u]$ 为深度，并可用 $lca(u,v)$ 求最近公共祖先（即本专题「最近公共祖先」任一版本给出的量）。 $buildVirtualTree(S)$ 返回虚树邻接表 vt （无向存边），根为 S 中点的 LCA；未涉及点为空。每次调用新建邻接表，可重复调用。

排序与建树复杂度 $\mathcal{O}(k \log k)$ ，另含 $\mathcal{O}(k)$ 次 LCA 查询。

```

1  auto buildVirtualTree = [&](std::vector<int> S) ->
    ↪ std::vector<std::vector<int>> {
2      std::sort(S.begin(), S.end(), [&](const auto& x, const
    ↪ auto& y) {
3          return dfn[x] < dfn[y];
4      });
5      S.erase(std::unique(S.begin(), S.end()), S.end());
6      std::vector<std::vector<int>> vt(n + 1);
7      std::vector<int> stk;
8      auto link = [&](int p, int c){
9          vt[p].push_back(c);
10         vt[c].push_back(p);
11     };
12     for (int u : S){
13         if (stk.empty()){ stk.push_back(u); continue; }
14         int w = lca(stk.back(), u);
15         while ((int)stk.size() >= 2 && dep[w] <=
            ↪ dep[stk[stk.size() - 2]]){

```

```

16         link(stk[stk.size() - 2], stk.back());
17         stk.pop_back();
18     }
19     if (stk.back() != w){
20         link(w, stk.back());
21         stk.pop_back();
22         if (stk.empty() || stk.back() != w)
23             ↪ stk.push_back(w);
24     }
25     stk.push_back(u);
26 }
27 while ((int)stk.size() >= 2){
28     link(stk[stk.size() - 2], stk.back());
29     stk.pop_back();
30 }
31 return vt;
32 };

```

9.3 长链剖分

长链剖分与重链剖分类似，但选取的“重儿子”是子树中深度最大的儿子，而不是子树大小最大的儿子。这样剖分后每条长链的长度之和为 $\mathcal{O}(n)$ ，且任意点到其链顶的路径长等于链长。

最常见的应用是以深度为维度的 DP 合并，复杂度 $\mathcal{O}(n)$ 。

9.3.1 长链剖分优化 DP

使用片段：需上下文已有以 1 为根的树邻接表 $G(1..n)$ ；段内定义 `dep`、`son`、`buf`、`dp`、`cur` 与函数 `dfs1/dfs2`。

`dp[u][i]` 表示 u 子树内与 u 距离恰为 i 的统计值。各轻儿子子树 DP 结果合并到链上，整棵树的合并总代价 $\mathcal{O}(n)$ 。由于 u 链上的 `dp` 缓冲随后会被其父结点复用，读取 `dp[u][i]` 必须在 u 并入父链之前完成，不能长期保存 `dp[u]` 的指针。

```

1 // dp[u][i]: u 子树内与 u 距离为 i 的信息；链上结点共享缓冲，
  ↪ 轻儿子单独分配
2 std::vector<int> dep(n + 1), son(n + 1);
3 std::vector<int> buf(n + 1);
4 std::vector<int*> dp(n + 1);

```

```

5  int* cur = buf.data();
6
7  auto dfs1 = [&](this auto&& dfs, int u, int pa) -> void {
8      dep[u] = 1;
9      for (int v : G[u]) if (v ≠ pa){
10         dfs(v, u);
11         if (dep[v] > dep[son[u]]) son[u] = v;
12     }
13     dep[u] = dep[son[u]] + 1;
14 };
15
16 auto dfs2 = [&](this auto&& dfs, int u, int pa) -> void {
17     if (!pa || u ≠ son[pa]) dp[u] = cur, cur += dep[u];
18     dp[u][0] = 1;
19     if (son[u]){
20         dp[son[u]] = dp[u] + 1;
21         dfs(son[u], u);
22     }
23     for (int v : G[u]) if (v ≠ pa && v ≠ son[u]){
24         dfs(v, u);
25         for (int i = 0; i < dep[v]; i++)
26             dp[u][i + 1] += dp[v][i]; // 合并
27     }
28 };
29
30 dfs1(1, 0);
31 cur = buf.data();
32 dfs2(1, 0);

```

9.4 Prufer 序列

一棵 n 个点的带标号无根树与一个长度为 $n - 2$ 的序列一一对应，该序列即 Prufer 序列。常用于无根树的计数、随机生成树等。编码与解码均为 $\mathcal{O}(n)$ 。

```

1  struct Prufer{
2      static std::vector<int>
        ↪ encode(std::vector<std::vector<int>>& G, int n){

```

```
3     std::vector<int> fa = transfromFa(G,n);
4     return encode(fa,n);
5 }
6 static std::vector<int> encode(std::vector<int>& fa,int
    ↪ n){
7     std::vector<int> deg(n+1);
8     for (int i = 1;i < n;i++) deg[fa[i]]++;
9     std::vector<int> res(n-2);
10    for (int i = 1, ptr = 1;i <= n-2;i++){
11        while (deg[ptr] ≠ 0) ptr++;
12        res[i-1] = fa[ptr];
13        deg[ptr]--;
14        deg[fa[ptr]]--;
15        if (deg[fa[ptr]] == 0 && fa[ptr] < ptr) ptr =
            ↪ fa[ptr];
16    }
17    return res;
18 }
19 static std::vector<int> decode(std::vector<int> a,int n){
20     std::vector<int> deg(n+1),fa(n+1);
21     for (int i = 0;i < a.size();i++) deg[a[i]]++;
22     a.push_back(n);
23     for (int cur = 1,i = 0,tmp = -1;i < a.size();i++){
24         int u;
25         if (tmp ≠ -1) {
26             u = tmp;
27             tmp = -1;
28         } else {
29             while (deg[cur] > 0) cur++;
30             u = cur;
31             cur++;
32         }
33         fa[u] = a[i];
34         deg[a[i]]--;
35         if (deg[a[i]] == 0 && cur > a[i]) tmp = a[i];
36     }
37     return fa;
38 }
```

```

39 private:
40     static std::vector<int>
41     ↪ transformFa(std::vector<std::vector<int>>& G,int n){
42         std::vector<int> fa(n+1);
43         auto dfs = [&](this auto&& dfs,int u,int pa)->void {
44             fa[u] = pa;
45             for (int v : G[u]) {
46                 if (v == pa) continue;
47                 dfs(v,u);
48             }
49             dfs(n,n);
50             return fa;
51         }
52     };

```

9.5 Link Cut Tree

动态树（LCT），维护森林，支持加边删边、换根、单点修改、路径异或查询等操作，所有操作均摊 $\mathcal{O}(\log n)$ 。编号从 1 开始。

```

1  struct LinkCutTree {
2      using u32 = unsigned int;
3      LinkCutTree(u32 _n){
4          init(_n);
5      }
6      struct Node {
7          int w = 0,sum = 0;
8      };
9      std::vector<Node> G;
10     std::vector<u32> siz,fa;
11     std::vector<std::array<u32,2>> son;
12     std::vector<bool> rev;
13     u32 size() const {
14         return treeSize;
15     }
16     void pushDown(u32 u){

```

```

17         if (!u) return;
18         if (rev[u]){
19             if (u32 v = son[u][0]) flipNode(v);
20             if (u32 v = son[u][1]) flipNode(v);
21             rev[u] = false;
22         }
23     }
24     void pushUp(u32 u){
25         if (!u) return;
26         G[u].sum = G[u].w;
27         if (son[u][0]) G[u].sum ^= G[son[u][0]].sum;
28         if (son[u][1]) G[u].sum ^= G[son[u][1]].sum;
29         siz[u] = siz[son[u][0]] + siz[son[u][1]] + 1;
30     }
31 private:
32     u32 treeSize = 0;
33     void init(u32 n){
34         G.resize(n+1);
35         fa.resize(n+1);
36         siz.resize(n+1);
37         son.resize(n+1);
38         rev.resize(n+1);
39         treeSize = n;
40         for (u32 i = 0; i <= n; i++) initNode(i);
41     }
42     void initNode(u32 x){
43         son[x] = {0,0};
44         fa[x] = 0;
45         siz[x] = x == 0 ? 0 : 1;
46         rev[x] = 0;
47     }
48     bool isRoot(u32 u) const {
49         return !u || son[fa[u]][0] != u && son[fa[u]][1] !=
           ↪ u;
50     }
51     u32 dir(u32 u){
52         assert(!isRoot(u));
53         return son[fa[u]][1] == u;

```

```
54     }
55     void rotate(u32 u){
56         if (!u || !fa[u] || isRoot(u)) return;
57         u32 f = fa[u], i = dir(u);
58         u32 gf = fa[f];
59         if (!isRoot(f)) son[gf][dir(f)] = u;
60         son[f][i] = son[u][!i];
61         son[u][!i] = f;
62         if (son[f][i]) fa[son[f][i]] = f;
63         fa[u] = gf;
64         fa[f] = u;
65         pushUp(f);
66         pushUp(u);
67     }
68     void flipNode(u32 u){
69         std::swap(son[u][0], son[u][1]);
70         rev[u] = !rev[u];
71     }
72     void reverse(u32 u){
73         if (!u) return;
74         splay(u);
75         flipNode(u);
76     }
77 public:
78     void splay(u32 u){
79         std::vector<u32> stk;
80         stk.emplace_back(u);
81         u32 x = u;
82         while (!isRoot(x)){
83             x = fa[x];
84             stk.emplace_back(x);
85         }
86         while (!stk.empty()) {
87             pushDown(stk.back());
88             stk.pop_back();
89         }
90         u32 f = fa[u];
91         while (!isRoot(u)){
```

```
92         if (!isRoot(f)){
93             rotate(dir(f) == dir(u) ? f : u);
94         }
95         rotate(u);
96         f = fa[u];
97     }
98     pushUp(u);
99 }
100 void access(u32 u){
101     u32 lst = 0;
102     for (u32 x = u; x; x = fa[x]){
103         splay(x);
104         son[x][1] = lst;
105         pushUp(x);
106         lst = x;
107     }
108     splay(u);
109 }
110 void make_root(u32 u){
111     access(u);
112     reverse(u);
113 }
114 void split(u32 x, u32 y){
115     make_root(x);
116     access(y);
117     splay(y);
118 }
119 void link(u32 x, u32 y){
120     if (find_root(x) == find_root(y)) return;
121     make_root(x);
122     fa[x] = y;
123 }
124 void cut(u32 x, u32 y){
125     split(x, y);
126     if (son[x][1] || son[y][0] != x) return;
127     fa[x] = son[y][0] = 0;
128     pushUp(y);
129 }
```

```
130     u32 find_root(u32 u){
131         access(u),splay(u);
132         while(son[u][0]) u = son[u][0];
133         return u;
134     }
135     void modify(u32 x,int val){
136         splay(x);
137         G[x].w = val;
138         pushUp(x);
139     }
140 };
```

图论

10.1 强连通分量与缩点

Tarjan 求有向图强连通分量 (SCC) 并缩点，缩点后得到 DAG。该版本为带点权合并版本。

复杂度 $\mathcal{O}(n + m)$ 。

```

1 struct SCC{
2     int size = 0, t = 0;
3     std::vector<std::vector<int>> G1, G2;
4     std::vector<int> low, dfn, W, stk, scc, W2;
5     std::vector<bool> instack;
6     SCC(int siz){
7         size = siz+1;
8         const std::vector<std::vector<int>*> tmp =
9             ↳ {&low, &dfn, &W, &scc};
10        G1.resize(size), instack.resize(size);
11        G2.resize(1), W2.resize(1);
12        for (const auto &ptr : tmp) ptr->resize(size);
13    }
14    void connect(int u, int v){
15        G1[u].push_back(v);
16    }
17    std::vector<int>& operator[](int i){return G2[i];}
18    void tarjan(int u){
19        low[u] = dfn[u] = ++t;
20        instack[u] = 1, stk.push_back(u);
21        for (int v : G1[u]){

```

```

21         if (!dfn[v]) {
22             tarjan(v);
23             low[u] = std::min(low[u], low[v]);
24         } else if (instack[v]) {
25             low[u] = std::min(low[u], dfn[v]);
26         }
27     }
28     if (low[u] == dfn[u]) {
29         int p = -1;
30         G2.push_back({});
31         W2.push_back(0);
32         do {
33             p = stk.back();
34             stk.pop_back();
35             scc[p] = G2.size() - 1;
36             W2[scc[p]] += W[p];
37             instack[p] = 0;
38         } while (p != u);
39     }
40 }
41 void work(){
42     G2.assign(1, {});
43     for (int i = 1; i < size; i++)
44         if (!dfn[i]) tarjan(i);
45     for (int i = 1; i < size; i++){
46         int u = scc[i];
47         for (int v : G1[i]){
48             if (u == scc[v]) continue;
49             G2[u].push_back(scc[v]);
50         }
51     }
52     for (int i = 1; i < G2.size(); i++){
53         std::sort(G2[i].begin(), G2[i].end());
54         G2[i].erase(std::unique(G2[i].begin(), G2[i].end()
55             ↪ ), G2[i].end());
56     }
57     size = G2.size();
58 }

```

58 `};`

10.2 边双连通分量

Tarjan 求无向图边双连通分量 (EBCC) 并缩点, 缩点后的 G_2 是一棵树 (桥树), 树上边即割边。

复杂度 $\mathcal{O}(n + m)$ 。

```

1  struct EBCC{
2      int size = 0,t = 0,eid = 0;
3      std::vector<std::vector<std::pair<int,int>>> G1;//
        ↪ E:to,id
4      std::vector<std::vector<int>> G2;
5      std::vector<int> low,dfn,stk,ebcc;
6      EBCC(int siz){
7          size = siz + 1;
8          std::vector<std::vector<int>*> ptr =
        ↪ {&low,&dfn,&ebcc};
9          for (auto &v : ptr) v->assign(size,0);
10         G1.assign(size,{});
11         G2.assign(1,{});
12     }
13     void connect(int u,int v){
14         eid++;
15         G1[u].push_back({v,eid});
16         G1[v].push_back({u,eid});
17     }
18     auto& operator[](int i){return G2[i];}
19     void tarjan(int u,int e){
20         low[u] = dfn[u] = ++t;
21         stk.push_back(u);
22         for (const auto &[v,id] : G1[u]){
23             if (id == e) continue;
24             if (!dfn[v]) {
25                 tarjan(v,id);
26                 low[u] = std::min(low[u],low[v]);
27             } else low[u] = std::min(low[u],dfn[v]);

```

```

28     }
29     if (low[u] == dfn[u]){
30         int p = -1;
31         G2.push_back({});
32         do {
33             p = stk.back();
34             stk.pop_back();
35             ebcc[p] = G2.size() - 1;
36         } while(p != u);
37     }
38 }
39 void work(){
40     for (int u = 1; u < size; u++)
41         if (!dfn[u]) tarjan(u, -1);
42     for (int i = 1; i < size; i++){
43         int u = ebcc[i];
44         for (auto [v, id] : G1[i]){
45             if (ebcc[v] == u) continue;
46             G2[u].push_back(ebcc[v]);
47             G2[ebcc[v]].push_back(u);
48         }
49     }
50     size = G2.size();
51     for (int u = 1; u < size; u++){
52         std::sort(G2[u].begin(), G2[u].end());
53         G2[u].erase(std::unique(G2[u].begin(), G2[u].end())
54                     ↪ ), G2[u].end());
55     }
56 };

```

10.3 割点与割边

Tarjan 求割点。割边模板见 EBCC（所有边双分量之间的边均为割边）。

复杂度 $\mathcal{O}(n + m)$ 。

```

1 struct CutVertex{
2     int siz, tim = 0;

```

```
3     std::vector<std::vector<int>>> G;
4     std::vector<int> dfn,low;
5     std::vector<bool> vis,isCut;
6     CutVertex(int n = 1):siz(n),G(n+1),dfn(n+1),low(n+1),vis_
    ↪ (n+1),isCut(n+1){}
7     void connect(int u,int v){
8         assert(u <= siz && v <= siz);
9         G[u].push_back(v);
10        G[v].push_back(u);
11    }
12    void dfs(int u,int pa,bool isRoot = 0){
13        vis[u] = 1;
14        dfn[u] = ++tim,low[u] = dfn[u];
15        int deg = 0;
16        for (int v : G[u]){
17            if (v == pa) continue;
18            if (vis[v]) {
19                low[u] = std::min(low[u],dfn[v]);
20            } else {
21                deg++;
22                dfs(v,u);
23                low[u] = std::min(low[u],low[v]);
24                if (!isRoot && low[v] >= dfn[u]) isCut[u] =
    ↪ 1;
25            }
26        }
27        if (isRoot && deg >= 2) isCut[u] = 1;
28    }
29    void work(){
30        for (int i = 1;i <= siz;i++) {
31            if (vis[i]) continue;
32            dfs(i,i,1);
33        }
34    }
35};
```

10.4 欧拉路径与欧拉回路

对存有向图邻接表 G 求欧拉路径/回路（每条边恰好访问一次）：先调用 `dfs(s)`，再 `reverse` 整个 `path`，即得到从起点到终点的顶点序列。该段为使用片段，需放在已定义好邻接表 G 、顶点数 n 与起点 s 的上下文中（`path`、`now` 在段内自行声明）。复杂度 $\mathcal{O}(n+m)$ ，起点选择规则见下方注意。

```

1  std::vector<int> path, now(n+1);
2  auto dfs = [&](this auto&& dfs, int u) -> void {
3      while (now[u] < (int)G[u].size()){
4          int v = G[u][now[u]];
5          now[u]++;
6          dfs(v);
7      }
8      path.push_back(u);
9  };
10 dfs(s); // 若为回路则任意点出发，否则选择 out[i]-in[i]=1 的点出发
11 std::reverse(path.begin(), path.end());

```

注意：回路可从任意点出发、路径需从 $out[i] - in[i] = 1$ 的点出发；本模板按有向图书写（每条边仅在其出点邻接表中由游标删除），无向图需同时删除反向边并以奇度顶点为起点，使用前应校验度数条件与弱连通性。

10.5 最短路

10.5.1 Dijkstra（堆优化，非负权）

$G[u]$ 为邻接表，元素为 (v, w) （下标 $1..n$ ，可带负权的场景见下方 Johnson）。`dijkstra(s)` 返回 s 到各点的距离 `dis`，不可达为 `INF`。

复杂度 $\mathcal{O}((n+m)\log n)$ 。

```

1  using i64 = long long;
2  const i64 INF = (i64)4e18;
3  auto dijkstra = [&](int s) -> std::vector<i64> {
4      std::vector<i64> dis(n + 1, INF);
5      std::priority_queue<std::pair<i64, int>,
6          ⇐ std::vector<std::pair<i64, int>>, std::greater<>> pq;
7      dis[s] = 0;

```

```

7   pq.push({0, s});
8   while (!pq.empty()){
9       auto [d, u] = pq.top();
10      pq.pop();
11      if (d != dis[u]) continue;
12      for (auto [v, w] : G[u]){
13          if (d + w < dis[v]){
14              dis[v] = d + w;
15              pq.push({dis[v], v});
16          }
17      }
18  }
19  return dis;
20 };

```

10.5.2 Johnson 全源最短路（允许负权、无负环）

单点 Dijkstra 无法处理负权边。Johnson 先求一遍“势能” h ，把每条边 (u, v, w) 改写为 $w + h_u - h_v \geq 0$ 后从每个点跑 Dijkstra，最后还原真实距离。要求无负环（存在时 `work` 返回 `false`）；点数 n 不大且边稀疏时优于 Floyd。

$\text{dis}[u][v]$ 为 $u \rightarrow v$ 的最短路，不可达为 `INF`。

复杂度 $\mathcal{O}(nm + nm \log n)$ 。

```

1  using i64 = long long;
2  struct Johnson {
3      int n;
4      std::vector<std::vector<std::pair<int, i64>>> G;
5      std::vector<std::vector<i64>> dis;
6      Johnson(int n) : n(n), G(n + 1), dis(n + 1,
        ↪ std::vector<i64>(n + 1, (i64)4e18)) {}
7      void addEdge(int u, int v, i64 w){ G[u].push_back({v,
        ↪ w}); }
8      bool work(){
9          const i64 INF = (i64)4e18;
10         std::vector<i64> h(n + 1, 0), cnt(n + 1, 0);
11         std::vector<char> inq(n + 1, 1);
12         std::queue<int> q;
13         for (int i = 1; i <= n; i++) q.push(i);

```

```

14     while (!q.empty()){
15         int u = q.front(); q.pop(); inq[u] = 0;
16         for (auto [v, w] : G[u]){
17             if (h[v] > h[u] + w){
18                 h[v] = h[u] + w;
19                 if (!inq[v]){
20                     if (++cnt[v] > n) return false;
21                     q.push(v); inq[v] = 1;
22                 }
23             }
24         }
25     }
26     for (int u = 1; u <= n; u++){
27         dis[u][u] = 0;
28         std::priority_queue<std::pair<i64, int>,
29             ⇨ std::vector<std::pair<i64, int>>,
30             ⇨ std::greater<>> pq;
31         pq.push({0, u});
32         while (!pq.empty()){
33             auto [d, x] = pq.top(); pq.pop();
34             if (d != dis[u][x]) continue;
35             for (auto [v, w] : G[x]){
36                 i64 nd = d + w + h[x] - h[v];
37                 if (nd < dis[u][v]){
38                     dis[u][v] = nd;
39                     pq.push({nd, v});
40                 }
41             }
42         }
43         for (int v = 1; v <= n; v++)
44             if (dis[u][v] < INF) dis[u][v] += h[v] -
45                 ⇨ h[u];
46     }
47     return true;
48 }
49 };

```

10.6 二分图最大权匹配 (KM)

KM 求两侧各 n 点的二分图最大权（完美）匹配，要求存在由实边组成的完美匹配。权矩阵 w 按下标 $1..n$ 填：`addEdge(u,v,c)` 在 u 与 v 间建权 c 的边，重边取较大者；未建边保持初始化极小值 $-\infty$ （约 -4×10^{18} ），故负权实边也能正常加入，最优完美匹配不会用到极小值边。两侧点数不等时，先给较小一侧补上数量差个权 0 的虚拟点（相应 0 边照常 `addEdge`），补到同阶后结果即为两侧各自的“最大权匹配”。求最小权时把所有权取负后调用。

`work()` 返回最大权和；结束后 `match[j]` 为右侧 j 匹配到的左侧点。

复杂度 $\mathcal{O}(n^3)$ 。

```

1  using i64 = long long;
2  struct KM {
3      int n;
4      std::vector<std::vector<i64>> w;
5      std::vector<i64> lx, ly, slk;
6      std::vector<int> match, pre;
7      std::vector<char> vis;
8      KM(int n) : n(n), w(n + 1, std::vector<i64>(n + 1,
9          ↪ -(i64)4e18)),
10             lx(n + 1), ly(n + 1), slk(n + 1),
11             match(n + 1), pre(n + 1), vis(n + 1) {}
12  void addEdge(int u, int v, i64 c){ w[u][v] =
13      ↪ std::max(w[u][v], c); }
14  void augment(int root){
15      const i64 INF = (i64)4e18;
16      std::fill(vis.begin(), vis.end(), 0);
17      std::fill(slk.begin(), slk.end(), INF);
18      int y = 0, yy = 0;
19      match[0] = root;
20      do {
21          int a = match[y];
22          i64 d = INF;
23          vis[y] = 1;
24          for (int b = 1; b <= n; b++){
25              if (vis[b]) continue;
26              i64 g = lx[a] + ly[b] - w[a][b];
27              if (g < slk[b]){ slk[b] = g; pre[b] = y; }

```

```

26         if (slk[b] < d){ d = slk[b]; yy = b; }
27     }
28     for (int b = 0; b <= n; b++){
29         if (vis[b]){ lx[match[b]] -= d; ly[b] += d; }
30         else slk[b] -= d;
31     }
32     y = yy;
33 } while (match[y]);
34 while (y){
35     match[y] = match[pre[y]];
36     y = pre[y];
37 }
38 }
39 i64 work(){
40     const i64 INF = (i64)4e18;
41     for (int i = 1; i <= n; i++){
42         lx[i] = -INF;
43         for (int j = 1; j <= n; j++) lx[i] =
44             ↪ std::max(lx[i], w[i][j]);
45         ly[i] = 0;
46     }
47     std::fill(match.begin(), match.end(), 0);
48     for (int i = 1; i <= n; i++) augment(i);
49     i64 ans = 0;
50     for (int j = 1; j <= n; j++) ans += w[match[j]][j];
51     return ans;
52 };

```

10.7 网络流

10.7.1 网络最大流

Edmonds-Karp 算法

每次用 BFS 找一条增广路并增广，复杂度 $\mathcal{O}(nm^2)$ 。

```
1 using i64 = long long;
```

```
2 constexpr i64 INF = 1e18;
3 struct EdmondsKarp {
4     int n;
5     struct Edge {
6         int to = 0, nxt = 0;
7         i64 f = 0, c = 0;
8     };
9     std::vector<int> head, fr;
10    std::vector<Edge> edge;
11    EdmondsKarp(int _n):n(_n),head(n+1),edge(2),fr(n+1){}
12    void addEdge(int u,int v,i64 f,i64 c){
13        edge.push_back({v,head[u],f,c});
14        head[u] = edge.size() - 1;
15    }
16    void connect(int u,int v,i64 w){
17        addEdge(u,v,w,w);
18        addEdge(v,u,0,w);
19    }
20    i64 work(int s,int t){
21        i64 ans = 0;
22        while (1) {
23            std::queue<int> que;
24            std::fill(fr.begin(),fr.end(),-1);
25            que.push(s);
26            fr[s] = 0;
27            while (!que.empty()){
28                int u = que.front();
29                que.pop();
30                for (int e = head[u];e;e = edge[e].nxt){
31                    int v = edge[e].to;
32                    i64 w = edge[e].f;
33                    if (fr[v] != -1 || w == 0) continue;
34                    fr[v] = e^1;
35                    que.push(v);
36                }
37            }
38            if (fr[t] == -1) break;
39            i64 mn = INF;
```

```

40         for (int u = t; u != s; u = edge[fr[u]].to){
41             mn = std::min(mn, edge[fr[u] ^ 1].f);
42         }
43         for (int u = t; u != s; u = edge[fr[u]].to){
44             edge[fr[u]].f += mn;
45             edge[fr[u] ^ 1].f -= mn;
46         }
47         ans += mn;
48     }
49     return ans;
50 }
51 };

```

Dinic 算法

分层图 + 多路增广，复杂度 $O(n^2m)$ 。对于容量均为 1 的图总复杂度为 $O(m \cdot \min(m^{\frac{1}{2}}, n^{\frac{2}{3}}))$ ；二分图最大匹配（无权）为 $O(n^{\frac{1}{2}} \cdot m)$ 。

```

1  using i64 = long long;
2  constexpr i64 INF = 1e18;
3  struct Dinic {
4      int n;
5      std::vector<int> dis, now, head;
6      struct Edge {
7          int to = 0, nxt = 0;
8          i64 f = 0, c = 0;
9      };
10     std::vector<Edge> edge;
11     Dinic(int _n): n(_n), dis(_n+1), now(n+1), head(n+1), edge(2)
        ⇨ {}
12     void addEdge(int u, int v, i64 f, i64 c){
13         edge.push_back({v, head[u], f, c});
14         head[u] = edge.size() - 1;
15     }
16     void connect(int u, int v, i64 w){
17         addEdge(u, v, w, w);
18         addEdge(v, u, 0, w);
19     }

```

```
20     bool level_graph(int s,int t){
21         std::queue<int> que;
22         std::fill(dis.begin(),dis.end(),-1);
23         dis[s] = 0,que.push(s);
24         now[s] = head[s];
25         while (!que.empty()) {
26             auto u = que.front();
27             que.pop();
28             for (int e = head[u];e;e = edge[e].nxt){
29                 int v = edge[e].to;
30                 i64 w = edge[e].f;
31                 if (dis[v] ≠ -1 || w = 0) continue;
32                 dis[v] = dis[u] + 1;
33                 now[v] = head[v];
34                 que.push(v);
35                 if (v = t) return 1;
36             }
37         }
38         return 0;
39     }
40     i64 blocking_flows(int u,i64 cap,const int& t){
41         if (u = t) return cap;
42         i64 sum = 0;
43         for (int e = now[u];e;e = edge[e].nxt){
44             int v = edge[e].to;
45             i64 w = edge[e].f;
46             if (!cap) break;
47             now[u] = e;
48             if (dis[v] ≠ dis[u] + 1 || w <= 0) continue;
49             i64 k = blocking_flows(v,std::min(cap,w),t);
50             if (!k) dis[v] = -1;
51             edge[e].f -= k,edge[e^1].f += k;
52             sum += k,cap -= k;
53         }
54         return sum;
55     }
56     i64 work(int s,int t){
57         i64 sum = 0;
```

```

58         while (level_graph(s,t)){
59             sum += blocking_flows(s,INF,t);
60         }
61         return sum;
62     }
63 };

```

预留推进网络最大流 (HLPP)

在稠密图或 Dinic 容易退化的网络中计算最大流；使用最高标号选择、Gap 优化与全局重标号。

常用上界记作 $O(V^2\sqrt{E})$ ，空间 $O(V + E)$ 。

```

1  struct HLPP {
2      using i64 = long long;
3      struct Edge {
4          int to;
5          i64 c;
6          int rev;
7          Edge(int to, i64 c, int rev) : to(to), c(c),
            ↪ rev(rev) {}
8      };
9      int n, s, t;
10     int maxh, maxgaph, workcnt;
11     std::vector<std::vector<Edge>> vec;
12     std::vector<i64> ov;
13     std::vector<int> h, cur;
14     std::vector<int> ovList, ovNxt;
15     std::vector<int> gap, gapPrv, gapNxt;
16
17     HLPP(int n)
18         : n(n), maxh(0), maxgaph(0), workcnt(0), vec(n + 1),
19           ov(n + 1), h(n + 1), cur(n + 1), ovList(n + 1,
            ↪ -1),
20           ovNxt(n + 1, -1), gap(n + 1, -1), gapPrv(n + 1,
            ↪ -1),
21           gapNxt(n + 1, -1) {}
22

```

```

23     void connect(int u, int v, i64 c) {
24         vec[u].push_back(Edge(v, c, vec[v].size()));
25         vec[v].push_back(Edge(u, 0, vec[u].size() - 1));
26     }
27
28     i64 work(int s_, int t_) {
29         s = s_, t = t_;
30         globalRelabel();
31         for (auto &e : vec[s]) {
32             if (e.c) pushFlow(s, e, e.c);
33         }
34         for (; maxh >= 0; --maxh) {
35             while (~ovList[maxh]) {
36                 int x = ovList[maxh];
37                 ovList[maxh] = ovNxt[x];
38                 discharge(x);
39                 if (workcnt > (n << 2))
40                     globalRelabel();
41             }
42         }
43         return ov[t];
44     }
45
46 private:
47     void discharge(int x) {
48         int nh = n, sz = vec[x].size();
49         for (int i = cur[x]; i < sz; ++i) {
50             auto &e = vec[x][i];
51             if (e.c > 0) {
52                 if (h[x] == h[e.to] + 1) {
53                     pushFlow(x, e, std::min(ov[x], e.c));
54                     if (ov[x] == 0) {
55                         cur[x] = i;
56                         return;
57                     }
58                 } else {
59                     nh = std::min(nh, h[e.to] + 1);
60                 }

```

```

61         }
62     }
63     for (int i = 0; i < cur[x]; ++i) {
64         auto &e = vec[x][i];
65         if (e.c > 0)
66             nh = std::min(nh, h[e.to] + 1);
67     }
68     cur[x] = 0;
69     ++workcnt;
70     int oldh = h[x], head = gap[oldh];
71     if (~gapNxt[head]) {
72         setHeight(x, nh);
73     } else {
74         for (int i = oldh; i <= maxgaph; ++i) {
75             for (int j = gap[i]; ~j; ) {
76                 int nxt = gapNxt[j];
77                 h[j] = n;
78                 gapPrv[j] = gapNxt[j] = -1;
79                 j = nxt;
80             }
81             gap[i] = -1;
82         }
83         maxgaph = oldh - 1;
84     }
85 }
86
87 void globalRelabel() {
88     workcnt = maxh = maxgaph = 0;
89     std::fill(h.begin(), h.end(), n);
90     h[t] = 0;
91     std::fill(gapPrv.begin(), gapPrv.end(), -1);
92     std::fill(gapNxt.begin(), gapNxt.end(), -1);
93     std::fill(gap.begin(), gap.end(), -1);
94     std::fill(ovList.begin(), ovList.end(), -1);
95     std::fill(cur.begin(), cur.end(), 0);
96     std::queue<int> q;
97     q.push(t);
98     while (!q.empty()) {

```

```

99         int x = q.front();
100        q.pop();
101        for (auto &e : vec[x]) {
102            if (h[e.to] == n &&
103                e.to != s &&
104                vec[e.to][e.rev].c > 0) {
105                setHeight(e.to, h[x] + 1);
106                q.push(e.to);
107            }
108        }
109    }
110 }

111
112 void setHeight(int x, int newh) {
113     if (~gapPrv[x]) {
114         if (gapPrv[x] == x) {
115             gap[h[x]] = gapNxt[x];
116             if (~gapNxt[x])
117                 gapPrv[gapNxt[x]] = gapNxt[x];
118         } else {
119             gapNxt[gapPrv[x]] = gapNxt[x];
120             if (~gapNxt[x])
121                 gapPrv[gapNxt[x]] = gapPrv[x];
122         }
123         gapPrv[x] = gapNxt[x] = -1;
124     }
125     h[x] = newh;
126     if (h[x] >= n)
127         return;
128     maxgaph = std::max(maxgaph, h[x]);
129     if (ov[x] > 0) {
130         maxh = std::max(maxh, h[x]);
131         ovNxt[x] = ovList[h[x]];
132         ovList[h[x]] = x;
133     }
134     gapNxt[x] = gap[h[x]];
135     if (~gapNxt[x])
136         gapPrv[gapNxt[x]] = x;

```

```

137     gap[h[x]] = gapPrv[x] = x;
138 }
139
140 void pushFlow(int from, Edge &e, i64 flow) {
141     if (!ov[e.to] && e.to ≠ t && h[e.to] < n) {
142         maxh = std::max(maxh, h[e.to]);
143         ovNxt[e.to] = ovList[h[e.to]];
144         ovList[h[e.to]] = e.to;
145     }
146     e.c -= flow;
147     vec[e.to][e.rev].c += flow;
148     ov[from] -= flow;
149     ov[e.to] += flow;
150 }
151 };

```

10.7.2 网络费用流

基于 EK 的 SSP 算法

每次用 SPFA 找最短路（按费用）增广，基于 EdmondsKarp。复杂度上界 $\mathcal{O}(f \cdot nm)$ ， f 为最大流值。

```

1  using i64 = long long;
2  using p64 = std::array<i64,2>;
3  constexpr i64 INF = 1e18;
4  struct SSP {
5      int n;
6      struct Edge {
7          int to = 0, nxt = 0;
8          i64 f = 0, c = 0, w = 0;
9      };
10     std::vector<int> head, fr;
11     std::vector<Edge> edge;
12     std::vector<i64> dis;
13     std::vector<bool> vis;
14     SSP(int _n):n(_n),head(n+1),edge(2),fr(n+1),dis(n+1),vis_
        ↪ (n+1){}

```

```

15     void addEdge(int u,int v,i64 f,i64 c,i64 w){
16         edge.push_back({v,head[u],f,c,w});
17         head[u] = edge.size() - 1;
18     }
19     void connect(int u,int v,i64 c,i64 w){
20         addEdge(u,v,c,c,w);
21         addEdge(v,u,0,c,-w);
22     }
23     p64 work(int s,int t){
24         p64 ans = {0,0};
25         while (1) {
26             std::queue<int> que;
27             que.push(s);
28             std::fill(fr.begin(),fr.end(),0);
29             std::fill(vis.begin(),vis.end(),0);
30             std::fill(dis.begin(),dis.end(),INF);
31             dis[s] = 0,vis[s] = 1;
32             while (!que.empty()) {
33                 int u = que.front();
34                 que.pop();
35                 vis[u] = 0;
36                 for (int e = head[u];e;e = edge[e].nxt){
37                     i64 f = edge[e].f,w = edge[e].w;
38                     int v = edge[e].to;
39                     if (f > 0 && dis[u] + w < dis[v]){
40                         dis[v] = dis[u] + w;
41                         fr[v] = e^1;
42                         if (!vis[v]){
43                             que.push(v);
44                             vis[v] = 1;
45                         }
46                     }
47                 }
48             }
49             if (dis[t] >= INF) break;
50             i64 mn = INF;
51             for (int u = t;u != s;u = edge[fr[u]].to){
52                 mn = std::min(mn,edge[fr[u] ^ 1].f);

```

```

53         }
54         for (int u = t; u != s; u = edge[fr[u]].to){
55             edge[fr[u]].f += mn;
56             edge[fr[u] ^ 1].f -= mn;
57         }
58         ans[0] += mn;
59         ans[1] += dis[t] * mn;
60     }
61     return ans;
62 }
63 };

```

基于 Dinic 的 SSP 算法 (Dijkstra 势能优化)

先用 SPFA 求初始势能 h ，再用势能将费用化为非负后跑 Dijkstra 找增广路，基于 Dinic。复杂度上界 $\mathcal{O}(f \cdot m \log n)$ ， f 为最大流值。

注意：以上两种算法均无法处理带负环费用的最小费用最大流。

```

1  using i64 = long long;
2  using p64 = std::array<i64,2>;
3  constexpr i64 INF = 1e18;
4  struct MinCostFlow {
5      int n;
6      struct Edge {
7          int to = 0, nxt = 0;
8          i64 f = 0, c = 0, w = 0;
9      };
10     std::vector<int> head, now;
11     std::vector<Edge> edge;
12     std::vector<i64> dis, h;
13     std::vector<bool> vis;
14     MinCostFlow (int _n):n(_n),head(n+1),edge(2),now(n+1),dis
        ↪ s(n+1),h(n+1),vis(n+1){}
15     void addEdge(int u,int v,i64 f,i64 c,i64 w){
16         edge.push_back({v,head[u],f,c,w});
17         head[u] = edge.size() - 1;
18     }
19     void connect(int u,int v,i64 c,i64 w){

```

```
20     addEdge(u,v,c,c,w);
21     addEdge(v,u,0,c,-w);
22 }
23 void SPFA(int s,int t){
24     std::fill(h.begin(),h.end(),INF);
25     std::fill(vis.begin(),vis.end(),0);
26     std::queue<int> que;
27     que.push(s);
28     h[s] = 0,vis[s] = 1;
29     while (!que.empty()){
30         int u = que.front();
31         que.pop();
32         vis[u] = 0;
33         for (int e = head[u];e;e = edge[e].nxt){
34             i64 f = edge[e].f,w = edge[e].w;
35             int v = edge[e].to;
36             if (f > 0 && h[u] + w < h[v]){
37                 h[v] = h[u] + w;
38                 if (!vis[v]) {
39                     que.push(v);
40                     vis[v] = 1;
41                 }
42             }
43         }
44     }
45 }
46 bool dijkstra(int s,int t){
47     std::fill(dis.begin(),dis.end(),INF);
48     std::fill(vis.begin(),vis.end(),0);
49     std::priority_queue<p64> pq;
50     pq.push({0,s});
51     dis[s] = 0;
52     while (!pq.empty()){
53         auto [dist,u] = pq.top();
54         pq.pop();
55         if (vis[u]) continue;
56         vis[u] = 1;
57         for (int e = head[u];e;e = edge[e].nxt){
```

```

58         int v = edge[e].to;
59         i64 w = edge[e].w, f = edge[e].f;
60         if (f > 0 && dis[u] + w + h[u] - h[v] <
        ⇨ dis[v]){
61             dis[v] = dis[u] + w + h[u] - h[v];
62             pq.push({-dis[v], v});
63         }
64     }
65 }
66 return vis[t];
67 }
68 i64 dfs(int u, const int& t, i64 flow, i64 &cost){
69     if (u == t) return flow;
70     i64 sum = 0;
71     vis[u] = 1;
72     for (int e = now[u]; e; e = edge[e].nxt){
73         now[u] = e;
74         int v = edge[e].to;
75         i64 f = edge[e].f, w = edge[e].w;
76         if (f > 0 && !vis[v] && h[v] == h[u] + w) {
77             i64 f2 = dfs(v, t, std::min(f, flow), cost);
78             edge[e].f -= f2;
79             edge[e^1].f += f2;
80             sum += f2;
81             flow -= f2;
82             cost += f2 * w;
83             if (flow == 0) break;
84         }
85     }
86     vis[u] = 0;
87     return sum;
88 }
89 p64 work(int s, int t){
90     p64 ans = {0, 0};
91     SPFA(s, t);
92     while (dijkstra(s, t)){
93         std::fill(vis.begin(), vis.end(), 0);
94         for (int i = 0; i <= n; i++){

```

```

95         h[i] += dis[i];
96         now[i] = head[i];
97     }
98     ans[0] += dfs(s,t,INF,ans[1]);
99 }
100 return ans;
101 }
102 };

```

10.7.3 上下界网络流

上下界可行流/最大流/最小流

基于 Dinic（见 MaxFlow.tex，其中定义了 `i64` 与 `INF`）。`connect(u,v,l,r)`：添加边 $u \rightarrow v$ ，流量须落在区间 $[l,r]$ 内。`getFlow()` 判断是否存在满足所有边上下界的可行流（不带源汇即判断是否存在循环流）；`maxflow(s,t)`、`minflow(s,t)` 返回满足边上下界的 $s \rightarrow t$ 最大/最小流量，不存在可行流时返回 -1 。该方法求得的可行流值非负，即需要存在一条流量值 ≥ 0 的可行流。`maxflow` 与 `minflow` 各在可行流判定后多做一次增广，总复杂度与两次 Dinic 同阶， $\mathcal{O}(n^2m)$ 。

```

1  template<typename Flow>
2  struct BoundFlow {
3      Flow G;
4      std::vector<i64> w,f;
5      int n,m = 0;
6      BoundFlow (int _n) : n(_n),G(_n + 2),w(_n+1),f(1) {}
7      void connect(int u,int v,i64 l,i64 r) {
8          m++;
9          G.connect(u,v,r-l);
10         f.push_back(l);
11         w[u] -= l;
12         w[v] += l;
13     }
14     bool getFlow(int s = 0,int t = 0) {
15         int S = n+1,T = n+2;
16         for (int i = 1;i <= n;i++){
17             if (w[i] > 0) {
18                 G.connect(S,i,abs(w[i]));

```

```

19         } else if (w[i] < 0) {
20             G.connect(i,T,abs(w[i]));
21         }
22     }
23     int M = G.edge.size();
24     if (s && t) G.connect(t,s,INF);
25     G.work(S,T);
26     for (int i = 2;i <= m*2;i += 2){
27         f[i/2] += G.edge[i].c - G.edge[i].f;
28     }
29     for (int i = m*2+2;i < M;i += 2){
30         if (G.edge[i].f != 0 || G.edge[i^1].f !=
31             ⇨ G.edge[i^1].c) return 0;
32     }
33     return 1;
34 }
35 i64 maxflow(int s,int t){
36     if (!getFlow(s,t)) return -1;
37     int rid = G.head[t];
38     i64 sum = G.edge[rid^1].f;
39     G.edge[rid].f = G.edge[rid^1].f = 0;
40     G.edge[rid].c = G.edge[rid^1].c = 0;
41     sum += G.work(s,t);
42     return sum;
43 }
44 i64 minflow(int s,int t){
45     if (!getFlow(s,t)) return -1;
46     int rid = G.head[t];
47     i64 sum = G.edge[rid^1].f;
48     G.edge[rid].f = G.edge[rid^1].f = 0;
49     G.edge[rid].c = G.edge[rid^1].c = 0;
50     sum -= G.work(t,s);
51     return sum;
52 }
};

```

10.7.4 最小割树 (Gomory-Hu)

建出 n 个点的树，使树上任意两点间路径的最小边权等于原图这两点间的最小割；之后两点最小割即树上路径最小边权。构造前先对原图加边：connect(u, v, c) 加容量 c 的无向边。前置依赖：MinCut 必须用 MaxFlow.tex 的 Dinic（代码直接读取其 edge/dis 内部状态，Flow.work 一次只求一个源汇的最小割），并需一并抄取 using i64 = long long;; 结点编号为 $1..n$ 。单点建树约 n 次最大流，总复杂度 $\mathcal{O}(n^3m)$ 。

```

1  template<typename MinCut>
2  struct GomoryHuTree {
3      int n;
4      MinCut Flow;
5      std::vector<std::vector<std::array<long long,2>>> G;
6      std::vector<int> vdx;
7      GomoryHuTree(int _n):n(_n),Flow(n),G(n+1),vdx(n+1){
8          std::iota(vdx.begin(),vdx.end(),0);
9      }
10     void connect(int u,int v,i64 c){
11         Flow.connect(u,v,c);
12         Flow.connect(v,u,c);
13     }
14     void build(int l,int r){
15         if (l >= r) return;
16         for (int e = 2;e < Flow.edge.size();e += 2){
17             Flow.edge[e].f = Flow.edge[e].c;
18             Flow.edge[e^1].f = 0;
19         }
20         i64 cut = Flow.work(vdx[l],vdx[r]);
21         G[vdx[r]].push_back({vdx[l],cut});
22         G[vdx[l]].push_back({vdx[r],cut});
23         std::vector<int> v1,v2;
24         for (int i = l;i <= r;i++){
25             int u = vdx[i];
26             if (Flow.dis[u] != -1) v1.push_back(u);
27             else v2.push_back(u);
28         }
29         const int mid = l + v1.size();
30         for (int i = l;i < mid;i++){
31             vdx[i] = v1[i - l];

```

```

32     }
33     for (int i = mid; i <= r; i++){
34         vdx[i] = v2[i - mid];
35     }
36     build(l, mid-1);
37     build(mid, r);
38 }
39 };

```

10.8 Matrix-Tree 定理

需要 Z （模意义整数，见杂项章节）与高斯消元求行列式 `Guass::det`。

10.8.1 生成树计数

无向图生成树个数 = 拉普拉斯矩阵（度数矩阵 - 邻接矩阵）去掉任一行一列后的行列式。重边按权值累加。

```

1 namespace MatrixTree {
2     using MInt = Z;
3     using std::vector;
4     using std::array;
5     MInt undirected(int n, vector<array<int, 3>> E){
6         vector K(n, vector(n, MInt(0)));
7         for (auto [x, y, w] : E){
8             K[x][y] -= w;
9             K[y][x] -= w;
10            K[x][x] += w;
11            K[y][y] += w;
12        }
13        for (int i = 0; i < n; i++){
14            K[i].pop_back();
15        }
16        K.pop_back();
17        return Guass::det(K);
18    }
19    // 该版本为外向树版本，如果为内向树则需要边权取反
20    MInt directed(int n, int rt, vector<array<int, 3>> E){

```

```

21     vector K(n,vector(n,MInt(0)));
22     for (auto [x,y,w] : E){
23         K[x][y] -= w;
24         K[y][y] += w;
25     }
26     for (int i = 0;i < n;i++){
27         K[i].erase(K[i].begin()+rt,K[i].begin()+rt+1);
28     }
29     K.erase(K.begin()+rt,K.begin()+rt+1);
30     return Guass::det(K);
31 }
32 };

```

10.8.2 BEST 定理

若有向强连通图中存在欧拉回路，那么它的欧拉回路个数为 $ec(G) = t^{root}(G,k) \cdot \prod_{i=1}^n (\deg(i) - 1)!$ ，其中 k 可以取 $1 \sim n$ 的任意正整数。

```

1 namespace MatrixTree {
2     using std::vector;
3     using std::array;
4     MInt BEST(int n,vector<array<int,3>> E){
5         if (E.empty()) return 1;
6         vector<int> in(n),out(n);
7         for (auto [x,y,w] : E){
8             out[x] += w;
9             in[y] += w;
10        }
11        for (int i = 0;i < n;i++) {
12            if (in[i] != out[i]) return 0;
13        }
14        MInt prod = 1;
15        int mx = 0;
16        for (int i = 0;i < n;i++) mx = std::max(mx,in[i]);
17        vector<MInt> fact(mx+1,1);
18        for (int i = 1;i <= mx;i++) fact[i] = fact[i-1] * i;
19        for (int i = 0;i < n;i++) if (in[i]) prod *=
            ↪ fact[in[i]-1];

```

```
20         return directed(n,0,E) * prod;
21     }
22 };
```

字符串

11.1 KMP (border 数组)

用于单模式串匹配。构造时传入模式串 `base`，之后调用 `operator()(str)` 得到它在 `str` 中的所有出现位置，复杂度 $\mathcal{O}(n + m)$ 。

`nxt[i]` 为 `base` 每个前缀的最长真 border（border 即既作前缀又作后缀的子串）长度；`operator()` 返回模式串在文本中的所有出现起点下标，重叠出现同样统计。

```

1  struct KMP{
2      std::vector<int> nxt;
3      std::string match;
4      KMP(const std::string
        ↪ &base):nxt(base.size()),match(base){
5          for(int p=0,itr=1;itr < base.size();){
6              if (base[itr] == base[p]) nxt[itr++] = ++p;
7              else {
8                  if (!p) nxt[itr++] = 0;
9                  else p=nxt[p-1];
10             }
11         }
12     }
13     std::vector<int> operator()(const std::string &str){ //
        ↪ 获得每个子串的位置数组
14         std::vector<int> ans;
15         for(int p = 0,itr = 0;itr < str.size();){
16             if (match[p] == str[itr]) p++,itr++;
17             else {
18                 if (!p) itr++;

```

```

19         else p = nxt[p-1];
20     }
21     if (p == match.size()){
22         ans.push_back(itr-match.size());
23         p = nxt[p-1];
24     }
25 }
26 return ans;
27 }
28 };

```

11.2 Manacher

用于求字符串的所有回文半径与最长回文子串，复杂度 $\mathcal{O}(n)$ 。

work(str) 返回 **ans**：其下标对应插入分隔符后的变换串（原位置 i 的字符在变换下标 $2i+2$ ），每项即该中心的最长回文长度， $\max(\text{ans})$ 即最长回文子串长度。

```

1 namespace Manacher{
2     using Container=std::string;
3     void work(Container &str,std::vector<int> &ans){
4         std::vector<int> ch(str.size()*2+3,-1);
5         ans = std::vector<int>(str.size()*2+3,0);
6         ch[0] = 448268,ch[ch.size()-1] = -ch[0];
7         for (int i = 0; i < str.size(); ++i) ch[(i+1)*2] =
8             ↪ str[i];
9         int mid = 1,r = 1;
10        for (int i = 1; i < ch.size() - 1; ++i) {
11            ans[i] = std::min(ans[mid*2-i],r-i);
12            while(ch[i+ans[i]+1]==ch[i-ans[i]-1]) ans[i]++;
13            if (i + ans[i] > r){
14                mid = i;
15                r = i + ans[i];
16            }
17        }
18        std::vector<int> work(Container &str){
19            std::vector<int> ans;

```

```

20         return work(str,ans),ans;
21     }
22 };

```

11.3 Z 函数 (exKMP)

用于求串与其每个后缀的最长公共前缀 (LCP)，也可作单模式串匹配 (exKMP)，复杂度 $\mathcal{O}(n)$ 。

$Z[i]$ 为 raw 与后缀 $raw[i..]$ 的 LCP, $Z[0]=n$; $operator()(str)$ 返回 $P[i]$ 即 $str[i..]$ 与 raw 的 LCP, 取值为 m 表示模式串出现在 $str[i]$ 处。

```

1  struct ZFun{
2      std::string raw;
3      std::vector<int> Z;
4      ZFun(const std::string &S):raw(S),Z(S.size()){
5          const int n = S.size();
6          for (int i = 1,l = 0,r = 0; i < n; ++i){
7              if (i < r){
8                  if (Z[i-l] < r-i+1) Z[i] = Z[i-l];
9                  else Z[i] = r-i+1;
10             }
11             while(i+Z[i] < n && S[i+Z[i]] == S[Z[i]]) Z[i]++;
12             if (i + Z[i] - 1 > r) l = i,r = i+Z[i]-1;
13         }
14         Z[0] = n;
15     }
16     std::vector<int> operator()(const std::string &S){
17         const int n = S.size(),m = raw.size();
18         std::vector<int> P(n);
19         for (int i = 0,l = 0,r = l;i < n; i++){
20             if (i < r){
21                 if (Z[i-l] < r-i+1) P[i] = Z[i-l];
22                 else P[i] = r-i+1;
23             }
24             while(i+P[i] < n && P[i] < m && S[i+P[i]] ==
25                   ↪ raw[P[i]]) P[i]++;
26             if (i + P[i] - 1 > r) l = i,r = i+P[i]-1;
27         }
28     }
29 };

```

```

27         return P;
28     }
29 };

```

11.4 Lyndon 分解

用于把字符串分解为字典序不增的 Lyndon 词序列，是求最小循环节、最小表示（旋转同构串的字典序最小者）类问题的常用工具。每段是 Lyndon 词（严格字典序小于其所有真后缀的串）。

`lyndon(str)` 返回每个 Lyndon 词的结束位置（左闭右开），相邻右端点之差即该词长度。

```

1 namespace Lyndon{
2     std::vector<int> lyndon(const std::string &str){
3         std::vector<int> end;
4         for (int i = 0,j,k;i < str.size();){
5             for(j = i,k = i+1;k < str.size() && str[k] >=
6                 ⇨ str[j];k++)
7                 if (str[j] == str[k]) j++;
8                 else j = i;
9                 while (i <= j) end.push_back(i+k-j),i += k-j;
10            }
11        return end;
12    }
13 }

```

11.5 后缀数组

在 $\mathcal{O}(n \log n)$ 复杂度内求出 `sa,rk,h` 数组，用于后缀排序与 LCP 相关统计。

`sa[i]` 为第 i 小的后缀的起点，`rk[i]` 为后缀 i 的排名，`h[i]` 为 `sa[i-1]` 与 `sa[i]` 的 LCP；任意两后缀的 LCP 是 `h` 对应区间的 `min`（需 RMQ）。

```

1 struct SuffixArray {
2     int n,m;
3     std::vector<int> rk,sa,h,bkt;
4     SuffixArray(const std::string& s):n(s.size()),m(std::max_
5         ⇨ (n,128)),rk(n,0),sa(n,0),h(n,0),bkt(m,0){

```

```

5     for (int i = 0; i < n; i++) bkt[s[i]]++;
6     for (int i = 1; i < 128; i++) bkt[i] += bkt[i-1];
7     for (int i = n-1; i >= 0; i--) sa[--bkt[s[i]]] = i;
8     rk[sa[0]] = 0;
9     std::vector<int> id(n);
10    int c = 0;
11    for (int i = 1; i < n; i++){
12        int x = sa[i-1], y = sa[i];
13        if (s[x] != s[y]) bkt[c++] = i;
14        rk[y] = c;
15    }
16    bkt[c++] = n;
17    for (int k = 1; k < n && c < n; k <= 1){
18        for (int i = 0; i < k; i++) id[i] = n - k + i;
19        int p = k;
20        for (int i = 0; i < n; i++) {
21            int x = sa[i] - k;
22            if (x >= 0) {
23                id[p++] = x;
24            }
25        }
26        for (int i = n-1; i >= 0; i--){
27            int x = id[i];
28            sa[--bkt[rk[x]]] = x;
29        }
30        c = 0;
31        id[sa[0]] = 0;
32        for (int i = 1; i < n; i++){
33            int x = sa[i-1], y = sa[i];
34            if (rk[x] != rk[y] || x == n-k || rk[x+k]
35                ↪ != rk[y+k]) {
36                bkt[c++] = i;
37            }
38            id[y] = c;
39        }
40        bkt[c++] = n;
41        std::swap(id, rk);
42    }

```

```

42     for (int i = 0; i < n; i++) rk[sa[i]] = i;
43     for (int i = 0, k = 0; i < n; i++){
44         if (rk[i] == 0) continue;
45         if (k) k--;
46         int j = sa[rk[i] - 1];
47         while (j + k < n && i + k < n && s[i+k] ==
48             ↪ s[j+k]) k++;
49         h[rk[i]] = k;
50     }
51     bkt.clear();
52 }
53 int size() const {
54     return n;
55 }
56 };

```

11.6 AC 自动机 (Aho-Corasick)

用于多模式串同时匹配：先逐个插入所有模式串，再 `build()` 建立自动机，之后在文本串上沿转移走一遍即可完成匹配，复杂度 $\mathcal{O}(\sum |\text{模式}| + |\text{文本}|)$ 。

`operator>>(s)` 插入一个模式串并返回其终止结点编号；`fail` 为失配指针、`in` 为 `fail` 树入度。精确统计每个模式串出现次数时，把文本串经过的结点计数沿 `fail` 树按 `in` 度做拓扑累加即可。

```

1  template<int S, int E>
2  struct AhoCorasick{
3      AhoCorasick(){
4          nodes.resize(1);
5      }
6      static constexpr int charSiz = E - S + 1;
7      struct ACNode{
8          int fa = 0, fail = 0, in = 0;
9          bool built = 0;
10         std::vector<int> nxt = std::vector<int>(charSiz);
11         int& operator[](int i){return nxt[i];}
12     };
13     std::vector<ACNode> nodes;

```

```
14     constexpr int ch(int raw){
15         return raw - S;
16     }
17     constexpr int newNode(int fa = 0){
18         nodes.push_back(ACNode());
19         nodes.back().fa = fa;
20         return nodes.size() - 1;
21     }
22     int operator>>(const std::string& s){
23         assert(!nodes[0].built);
24         int cur = 0;
25         for (int i = 0; i < s.size(); i++){
26             int chd = ch(s[i]);
27             if (!nodes[cur][chd]) nodes[cur][chd] =
28                 ↪ newNode(cur);
29             cur = nodes[cur][chd];
30         }
31         return cur;
32     }
33     int& fail(int u){
34         return nodes[u].fail;
35     }
36     void build(){
37         if (nodes.front().built) return;
38         nodes.front().built = 1;
39         std::queue<int> bfs;
40         for (int v : nodes[0].nxt){
41             if (v) {
42                 bfs.push(v);
43                 nodes[v].built = 1;
44             }
45         }
46         while(bfs.size()){
47             int u = bfs.front();
48             bfs.pop();
49             for (int i = 0; i < charSiz; i++){
50                 const int &chd = nodes[u][i];
51                 if (chd) {
```

```

51         if (!nodes[chd].built){
52             nodes[chd].built = 1;
53             bfs.push(chd);
54             fail(chd) = nodes[fail(u)][i];
55             nodes[fail(chd)].in ++;
56         }
57     } else {
58         nodes[u][i] = nodes[fail(u)][i];
59     }
60 }
61 }
62 }
63 ACNode& operator[](int i){
64     return nodes[i];
65 }
66 };
67 using ACAM = AhoCorasick<'a','z'>;

```

11.7 广义后缀自动机 (GSAM)

用于多个模式串共享的广义后缀自动机，支持多串的子串统计类问题（本质不同子串数、出现次数等）。`operator>>` 依次插入各模式串。

结点转移为 `ch`、后缀链接为 `link`、`longest` 为该状态最长串长、`cnt` 为该状态的结束次数；本质不同子串总数为 $\sum_i (\text{longest}[i] - \text{longest}[\text{link}[i]])$ 。转移表按固定字符集开定长数组，默认支持 'a'-'z'；若字符集不同或很大（约 50 以上），需自行把 `ch` 换成按需存储的稀疏表。

```

1  struct SAMInfo{
2      int cnt = 0;
3      std::vector<int> ch;
4      int link = 0, longest = 0;
5      int& operator[](int idx){return ch[idx];}
6  };
7
8  // General SuffixAutomaton
9  template<typename Info, int S, int E>
10 struct SuffixAutomaton{
11     std::vector<Info> nodes;

```

```

12     int ed = 1;
13     constexpr SuffixAutomaton(){
14         newNode(); // Void Node
15         newNode(); // Empty String Node -> ""
16     }
17     constexpr int newNode(){
18         int idx = nodes.size();
19         nodes.push_back(Info());
20         nodes.back().ch.resize(E-S+1);
21         return idx;
22     }
23     constexpr void extend(int c){
24         if (nodes[ed][c] && nodes[ed].longest + 1 ==
25             ↪ nodes[nodes[ed][c]].longest){
26             ed = nodes[ed][c], nodes[ed].cnt ++;
27             return;
28         }
29         int p = newNode(), lst = ed;
30         nodes[p].longest = nodes[ed].longest +
31             ↪ 1, nodes[p].cnt = 1;
32         for (; ed && !nodes[ed][c]; ed = nodes[ed].link)
33             ↪ nodes[ed][c] = p;
34         if (!ed) nodes[p].link = 1;
35         else {
36             int q = nodes[ed][c];
37             if (nodes[q].longest == nodes[ed].longest + 1)
38                 ↪ nodes[p].link = q; // Legal link
39             else {
40                 int x;
41                 if (lst == ed) x = p; // Split self
42                 else x = newNode(), nodes[p].link = x; // New
43                     ↪ Link
44                 nodes[x].ch = nodes[q].ch, nodes[x].link =
45                     ↪ nodes[q].link;
46                 nodes[x].longest = nodes[ed].longest + 1;
47                 nodes[q].link = x;
48                 for (; ed && nodes[ed][c] == q; ed =
49                     ↪ nodes[ed].link) nodes[ed][c] = x;

```

```

43         }
44     }
45     ed = p; // update ed
46 }
47 constexpr void operator>>(const std::string &str){
48     ed = 1;
49     for (auto &c : str) extend(c-S);
50 }
51 constexpr Info& operator[](int idx){return nodes[idx];}
52 };
53 using GSAM = SuffixAutomaton<SAMInfo, 'a', 'z'>;

```

11.8 回文树 (PAM / Eertree)

用于求字符串的全部本质不同回文子串及其相关统计。除两根外每个结点对应一个本质不同回文，**len** 为其长度、**fail** 为其最长回文真后缀；**extend(c)** 在线加入一个字符并返回以新字符结尾的回文后缀个数（**cnt** 沿 **fail** 链累计）。

注意 **cnt** 不是出现次数，统计各回文出现次数需另行沿 **fail** 树拓扑累加。

```

1  struct PAMNode{
2      int fail = 1, len = 0, cnt = 0;
3      std::vector<int> ch = std::vector<int>(26);
4      int& operator[](int idx){return ch[idx];}
5  };
6  struct PalindromicAutomaton{
7      using Info = PAMNode;
8      std::vector<Info> nodes;
9      int lst = 1;
10     std::string raw;
11     PalindromicAutomaton():nodes(2){
12         nodes[0].fail = 1, nodes[1].len = -1;
13     }
14     int newNode(){
15         nodes.push_back(Info());
16         return nodes.size()-1;
17     }
18     int matchFail(int u, int idx){

```

```

19     while(idx - nodes[u].len - 1 < 0 || raw[idx] !=
      ↪ raw[idx-nodes[u].len-1]) u = nodes[u].fail;
20     return u;
21 }
22 // 返回插入该字符后添加的回文子串数量
23 int extend(int c){
24     raw += c + 'a';
25     int p = matchFail(lst,raw.size()-1);
26     if (!nodes[p][c]){
27         int x = newNode();
28         nodes[x].fail = nodes[matchFail(nodes[p].fail,raw
      ↪ w.size()-1)][c];
29         nodes[p][c] = x;
30         nodes[x].len = nodes[p].len + 2;
31         nodes[x].cnt = nodes[nodes[x].fail].cnt + 1;
32     }
33     lst = nodes[p][c];
34     return nodes[lst].cnt;
35 }
36 };
37 using PAM = PalindromicAutomaton;

```

11.9 Thue-Morse 序列

用于构造字符串哈希反例，hack 自然溢出 (2^{64}) 或 2^x 取模的字符串哈希，与基数无关。

ThueMorse(MOD) 返回长度 2^p 的串；MOD 需为 0 或 2 的幂（否则断言失败），冲突长度为 2^{p-1} 。

```

1 // 用于 hack 自然溢出或者  $2^x$  作为模数
2 std::string ThueMorse(unsigned long long MOD = 0){
3     assert(__builtin_popcountll(MOD) == 1 || MOD == 0);
4     int p = 12;
5     int len = 1 << p; // 冲突长度为 ( $2^{(p-1)}$ )
6     std::string str;
7     for (int i = 0; i < len; i++) str +=
      ↪ 'a'+(__builtin_popcount(i)&1);
8     return str;

```

9

}

杂项

12.1 STL 常用库

12.1.1 int128

`i128`: `__int128` 的别名, 支持四则、比较与取模; 本段在 `std` 中补上 `to_string`、`stoi128` (十进制串转值, 可含负号) 及流式 IO, 故可对 `i128` 直接 `cin/cout`。

```

1  #include <bits/stdc++.h>
2  using i128 = __int128;
3  namespace std {
4      string to_string(i128 x) {
5          if (x == 0) return "0";
6          string s; bool sign = x < 0;
7          if (sign) x = -x;
8          while (x) s += x % 10 + '0', x /= 10;
9          if (sign) s += "-";
10         reverse(s.begin(), s.end());
11         return s;
12     }
13     i128 stoi128(const string &str){
14         i128 x = 0; bool sign = 0;
15         for(int i(0); i < str.size(); i++)
16             if (str[i] == '-') sign = 1;
17             else x *= 10, x += str[i] - '0';
18         return sign ? -x : x;
19     }
20     i128 abs(const i128 x){ return x < 0 ? -x : x; }
21     istream& operator>>(istream& is, i128 &x){

```

```
22     std::string s;
23     is >> s,x = stoi128(s);
24     return is;
25 }
26 ostream& operator<<(ostream& os,const i128 x){
27     if (x == 0) return os << 0;
28     os << to_string(x);
29     return os;
30 }
31 }
```

12.1.2 pb_ds 常用容器

pb_ds (__gnu_pbds) 属 GCC 扩展 (MinGW / Linux 均支持)，提供平衡树、配对堆、哈希表等容器。常用容器与头文件见下表，操作代码见各小节。

容器	底层结构	头文件	特点
tree	红黑树	assoc_container.hpp	
tree_policy.hpp	第 k 小 / 排名，单次 $\mathcal{O}(\log n)$		
priority_queue	配对堆	priority_queue.hpp	push 返回点迭代器，join 均摊 $\mathcal{O}(1)$
gp_hash_table	开放寻址	assoc_container.hpp	均摊 $\mathcal{O}(1)$ ，常数小于 unordered_map

有序统计树：tree 要求键值唯一，用 pair 第二维存唯一编号实现 multiset（重复元素），删除重复值用 lower_bound 定位单个元素。

操作	写法	复杂度
插入一个 x	tr.insert({x, ++id})	$\mathcal{O}(\log n)$
删除一个 x	tr.erase(tr.lower_bound({x, 0}))	$\mathcal{O}(\log n)$
第 k 小	tr.find_by_order(k)→first	$\mathcal{O}(\log n)$
小于 x 的个数	tr.order_of_key({x, 0})	$\mathcal{O}(\log n)$
值 $\geq k$ 移出	T.clear(); tr.split({k, 0}, T)	$\mathcal{O}(\log n)$
合并	tr.join(T)	$\mathcal{O}(\log^2 n)$

split 将值 $\geq k$ 的元素移入 T ；join 要求 tr 中所有键 $< T$ 中所有键，否则抛异常。

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  using pii = std::pair<int,int>;
5  typedef tree<pii, null_type, std::less<pii>, rb_tree_tag,
6             tree_order_statistics_node_update> OST;
7  OST tr;
8  int id = 0;
9  void insert(int x){ tr.insert({x, ++id}); } //
   ↪ 插入一个 x
10 void erase(int x){ tr.erase(tr.lower_bound({x, 0})); } //
   ↪ 删除一个 x
11 int kth(int k){ return tr.find_by_order(k)->first; } // 第
   ↪ k 小(0-indexed)
12 int cnt(int x){ return tr.order_of_key({x, 0}); } //
   ↪ 严格小于 x 的个数
13 void split(int k, OST &T){ T.clear(); tr.split({k, 0}, T); }
   ↪ // 值 >= k 移入 T
14 void join(OST &T){ tr.join(T); } // 要求
   ↪ tr 全键 < T 全键

```

配对堆：push 返回点迭代器，可 $\mathcal{O}(\log n)$ 修改堆内某点、均摊 $\mathcal{O}(1)$ 合并两堆，常用于 Dijkstra 与堆的合并。

操作	写法	复杂度
入堆	<code>auto it = pq.push({dis[v], v})</code>	$\mathcal{O}(\log n)$
修改键值	<code>pq.modify(it, {dis[v] + w, v})</code>	$\mathcal{O}(\log n)$
合并	<code>pq.join(other)</code>	均摊 $\mathcal{O}(1)$
取堆顶	<code>pq.top()</code>	$\mathcal{O}(1)$
弹堆顶	<code>pq.pop()</code>	$\mathcal{O}(\log n)$

```

1  #include <ext/pb_ds/priority_queue.hpp>
2  using namespace __gnu_pbds;
3  using Node = std::pair<int,int>; // (dist, v)
4  __gnu_pbds::priority_queue<Node, std::greater<Node>,
5                             pairing_heap_tag> pq;
6  auto it = pq.push({dis[v], v}); // 入堆并记录点迭代器
7  pq.modify(it, {dis[v] + w, v}); // 修改堆中某点的键值

```

```

8  pq.join(other); // 合并另一个堆到本堆
9  auto [d, u] = pq.top(); // 取堆顶
10 pq.pop(); // 弹堆顶

```

哈希表：均摊 $O(1)$ ，常数比 `unordered_map` 小；可自定义哈希函数避免被构造数据卡掉。

操作	写法	复杂度
插入 / 查询	<code>mp[x] = 1</code>	均摊 $O(1)$
查找	<code>mp.find(x) != mp.end()</code>	均摊 $O(1)$
自定义哈希	<code>gp_hash_table<int,int, custom_hash></code>	抗哈希冲突

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  using namespace __gnu_pbds;
3  struct custom_hash { // splitmix64 抗哈希冲突
4      static uint64_t splitmix64(uint64_t x){
5          x += 0x9e3779b97f4a7c15;
6          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
7          x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
8          return x ^ (x >> 31);
9      }
10     size_t operator()(uint64_t x) const {
11         static const uint64_t FIXED_RANDOM =
12             std::chrono::steady_clock::now().time_since_epoch()
13             ↪ h().count();
14         return splitmix64(x + FIXED_RANDOM);
15     }
16 };
17 gp_hash_table<int,int> mp; // 普通哈希表
18 gp_hash_table<int,int,custom_hash> mp2; // 带自定义哈希
19 mp[x] = 1;

```

12.1.3 可并堆（左偏树）

需要合并两个堆时，`pb_ds` 的配对堆（见上节）最省事；手写左偏树适用于需要自定义合并语义或环境无 `pb_ds` 的场合。以小根堆为例，`merge` 单次 $O(\log n)$ 。

操作	写法	复杂度
合并两堆	<code>int rt = merge(x, y)</code>	$\mathcal{O}(\log n)$
插入	<code>rt = push(rt, v)</code>	$\mathcal{O}(\log n)$
取堆顶	<code>top(rt)</code>	$\mathcal{O}(1)$
弹堆顶	<code>rt = pop(rt)</code>	$\mathcal{O}(\log n)$

```

1 // 可并堆（左偏树）小根堆；merge  $\mathcal{O}(\log n)$ 。
2 // 待合并的两堆必须互不相交（同一结点不能属于两堆）。
3 constexpr int N = 1e5 + 5;
4 std::array<int, N> val{}, ls{}, rs{}, d{}; // d[x] =
    ↳ 到空结点的最短距离（d[0] = 0）
5 int tot = 0;
6 int newnode(int v){ val[++tot] = v; d[tot] = 1; return tot; }
7 int merge(int x, int y){
8     if (!x || !y) return x ? x : y;
9     if (val[x] > val[y]) std::swap(x, y); // 小根堆：根取较小者
10    rs[x] = merge(rs[x], y);
11    if (d[ls[x]] < d[rs[x]]) std::swap(ls[x], rs[x]); //
        ↳ 维护左偏：左子 d 更大
12    d[x] = d[rs[x]] + 1;
13    return x;
14 }
15 int push(int rt, int v){ return merge(rt, newnode(v)); } //
    ↳ 插入：新结点自成一堆后合并
16 int top(int rt){ return val[rt]; }
17 int pop(int rt){ return merge(ls[rt], rs[rt]); } //
    ↳ 弹堆顶，左右子树合并为新堆

```

12.2 取模类

12.2.1 取模类（静态模数）

`ModInt<MOD>`: `mod MOD` 的整数，重载加减乘除、 $^$ （乘方）、比较、`++/--` 与输入输出，运算后自动取模。`Z = ModInt<998244353>` 为常用别名。要求 `MOD` 为质数（除法经费马小定理求逆元），对 0 求逆会触发 `assert`；`x()` 返回当前数值。

```

1 template <int MOD>
2 struct ModInt{

```

```

3   long long val = 0;
4   ModInt(long long x = 0):val(norm(x)){}
5   ModInt(const std::string& s){
6       for (const int& ch : s){
7           val = val * 10 + ch - '0';
8           val = norm(val);
9       }
10  }
11  long long operator()() const { return val; }
12  constexpr ModInt operator-() const {
13      return ModInt(MOD - val);
14  }
15  constexpr ModInt operator~() const {
16      assert(val != 0);
17      return Pow(val,MOD-2);
18  }
19  constexpr ModInt& operator+=(const ModInt& x){
20      return val += x(),val = norm(val),*this;
21  }
22  constexpr ModInt& operator-=(const ModInt& x){
23      return val -= x(),val = norm(val),*this;
24  }
25  constexpr ModInt& operator*=(const ModInt& x){
26      return val *= x(),val = norm(val),*this;
27  }
28  constexpr ModInt& operator/=(const ModInt& x){
29      return val *= (~x>(),val = norm(val),*this;
30  }
31  constexpr ModInt& operator^=(long long y){
32      return val = Pow(*this,y>(),*this;
33  }
34  constexpr bool operator==(const ModInt& x) const {
35      return val == x();
36  }
37  constexpr ModInt& operator++(){
38      return val ++,val = norm(val),*this;
39  }
40  constexpr ModInt operator++(int){

```

```

41     ModInt v = *this;
42     return val ++, val = norm(val), v;
43 }
44 constexpr ModInt& operator--(){
45     return val --, val = norm(val), *this;
46 }
47 constexpr ModInt operator--(int){
48     ModInt v = *this;
49     return val --, val = norm(val), v;
50 }
51 constexpr friend ModInt operator+(ModInt x, const ModInt&
    ↪ y){
52     return x += y;
53 }
54 constexpr friend ModInt operator-(ModInt x, const ModInt&
    ↪ y){
55     return x -= y;
56 }
57 constexpr friend ModInt operator*(ModInt x, const ModInt&
    ↪ y){
58     return x *= y;
59 }
60 constexpr friend ModInt operator/(ModInt x, const ModInt&
    ↪ y){
61     return x /= y;
62 }
63 constexpr friend ModInt operator^(ModInt x, long long y){
64     return x ^= y;
65 }
66 constexpr friend std::ostream& operator<<(std::ostream&
    ↪ os, const ModInt& x){
67     return os << x();
68 }
69 constexpr friend std::istream& operator>>(std::istream&
    ↪ is, ModInt &x){
70     is >> x.val, x.val = norm(x.val);
71     return is;
72 }

```

```

73     constexpr static long long norm(const long long &x){
74         return x < 0 ? (x % MOD + MOD) % MOD : x % MOD;
75     }
76     constexpr static ModInt Pow(ModInt x, long long y){
77         ModInt ans = 1;
78         for (; y; y >>= 1, x *= x)
79             if (y & 1) ans *= x;
80         return ans;
81     }
82 };
83 using Z = ModInt<998244353>;
84

```

12.2.2 取模类 (全局模数)

arbModInt: 模数可运行时指定, 使用前调用 `arbModInt::setMod(p)` (默认 998244353); 其余用法与 `ModInt` 一致。适合模数由输入给定、或需在运行中切换模数的题 (如原根、Cipolla)。

```

1  static int MOD = 998244353;
2  struct arbModInt{
3      long long val = 0;
4      arbModInt(long long x = 0):val(norm(x)){}
5      arbModInt(const arbModInt& x):val(x.val){}
6      arbModInt(const std::string& s){
7          for (const int& ch : s){
8              val = val * 10 + ch - '0';
9              val = norm(val);
10         }
11     }
12     constexpr static void setMod(int p){
13         MOD = p;
14     }
15     long long operator()() const { return val; }
16     constexpr arbModInt operator-() const {
17         return arbModInt(MOD - val);
18     }
19     constexpr arbModInt operator~() const {

```

```

20     assert(val  $\neq$  0);
21     return Pow(val,MOD-2);
22 }
23 constexpr arbModInt& operator+=(const arbModInt& x){
24     return val += x(),val = norm(val),*this;
25 }
26 constexpr arbModInt& operator-=(const arbModInt& x){
27     return val -= x(),val = norm(val),*this;
28 }
29 constexpr arbModInt& operator*=(const arbModInt& x){
30     return val *= x(),val = norm(val),*this;
31 }
32 constexpr arbModInt& operator/=(const arbModInt& x){
33     return val *= (~x)(),val = norm(val),*this;
34 }
35 constexpr arbModInt& operator^=(long long y){
36     return val = Pow(*this,y)(),*this;
37 }
38 constexpr bool operator==(const arbModInt& x) const {
39     return val == x();
40 }
41 constexpr arbModInt& operator++(){
42     return val ++,val = norm(val),*this;
43 }
44 constexpr arbModInt operator++(int){
45     arbModInt v = *this;
46     return val ++,val = norm(val),v;
47 }
48 constexpr arbModInt& operator--(){
49     return val --,val = norm(val),*this;
50 }
51 constexpr arbModInt operator--(int){
52     arbModInt v = *this;
53     return val --,val = norm(val),v;
54 }
55 constexpr friend arbModInt operator+(arbModInt x,const
    ↪ arbModInt& y){
56     return x += y;

```

```
57     }
58     constexpr friend arbModInt operator-(arbModInt x,const
    ↪   arbModInt& y){
59         return x -= y;
60     }
61     constexpr friend arbModInt operator*(arbModInt x,const
    ↪   arbModInt& y){
62         return x *= y;
63     }
64     constexpr friend arbModInt operator/(arbModInt x,const
    ↪   arbModInt& y){
65         return x /= y;
66     }
67     constexpr friend arbModInt operator^(arbModInt x,const
    ↪   long long& y){
68         return x ^= y;
69     }
70     constexpr friend std::ostream& operator<<(std::ostream&
    ↪   os,const arbModInt& x){
71         return os << x();
72     }
73     constexpr friend std::istream& operator>>(std::istream&
    ↪   is,arbModInt &x){
74         is >> x.val, x.val = norm(x.val);
75         return is;
76     }
77     constexpr static long long norm(long long x){
78         return x < 0 ? (x % MOD + MOD) % MOD : x % MOD;
79     }
80     constexpr static arbModInt Pow(arbModInt x,long long y){
81         arbModInt ans = 1;
82         for (;y; y >>= 1,x *= x)
83             if (y & 1) ans *= x;
84         return ans;
85     }
86 };
```

12.3 复数

`Complex<T>`: 自定义复数, 重载四则与 `Pow` (快速幂), 常数比 `std::complex` 小; T 常取 `double`。常用于 FFT、二次剩余等场合。

```
1  template<typename T>
2  struct Complex{
3      T real,imag;
4      Complex(T x = 0,T y = 0):real(x),imag(y){}
5      constexpr Complex& operator+=(const Complex &x){
6          real += x.real,imag += x.imag;
7          return *this;
8      }
9      constexpr Complex& operator-=(const Complex &x){
10         real -= x.real,imag -= x.imag;
11         return *this;
12     }
13     constexpr Complex& operator*=(const Complex &x){
14         T rl = real * x.real - imag * x.imag;
15         imag = x.imag * real + x.real * imag;
16         real = rl;
17         return *this;
18     }
19     constexpr Complex& operator/=(const Complex &x){
20         T div = x.real * x.real + x.imag * x.imag;
21         T rl = (real*x.real+imag*x.imag) / div;
22         imag = (x.real * imag - real * x.imag) / div;
23         real = rl;
24         return *this;
25     }
26     constexpr friend Complex operator+(const Complex&
27     ↪ x,const Complex& y){
28         Complex r = x;
29         return (r += y);
30     }
31     constexpr friend Complex operator-(const Complex&
32     ↪ x,const Complex& y){
33         Complex r = x;
```

```

32     return (r -= y);
33 }
34 constexpr friend Complex operator*(const Complex&
    ↪ x,const Complex& y){
35     Complex r = x;
36     return (r *= y);
37 }
38 constexpr friend Complex operator/(const Complex&
    ↪ x,const Complex& y){
39     Complex r = x;
40     return (r /= y);
41 }
42 constexpr friend std::ostream& operator<<(std::ostream&
    ↪ os,const Complex& x){
43     return os << x.real << "+" << x.imag << "i";
44 }
45 constexpr friend std::istream& operator>>(std::istream&
    ↪ is,Complex& x){
46     return is >> x.real >> x.imag;
47 }
48 constexpr static Complex Pow(Complex x,long long p){
49     Complex res = {1,0};
50     for (;p;p >>= 1,x *= x){
51         if (p&1) res *= x;
52     }
53     return res;
54 }
55 };

```

12.4 高精度

12.4.1 高精度整数

BigInt: 高精度整数，用十进制串 **BigInt(s)** 或 **long long BigInt(x)** 构造，支持负数与四则运算、%、比较、<<（乘 10^k ）及流式 IO，**to_string()** 转回十进制串；**a / b**、**a % b** 的符号规则同 C++ 整数。

加减复杂度 $\mathcal{O}(n)$ ；乘法与除法均为 $\mathcal{O}(n^2)$ （ n 为十进制位数）。

```
1 struct BigInt{
2     std::vector<int> num;
3     int sign = 1, size = 0;
4     BigInt(std::string raw){
5         while (raw.size()) {
6             int ch = raw.back(); raw.pop_back();
7             assert(ch == '-' || std::isdigit(ch));
8             if (ch == '-') sign = -1;
9             else num.push_back(ch - '0');
10        }
11        refresh();
12    }
13    BigInt(long long raw = 0){
14        if (raw < 0) sign = -1, raw = -raw;
15        while (raw) num.push_back(raw%10), raw /= 10;
16        refresh();
17    }
18    BigInt& operator=(long long x){
19        return *this = BigInt(x);
20    }
21    std::string to_string(){
22        refresh();
23        if (num.empty()) return "0";
24        std::string str = "";
25        if (sign == -1) str += "-";
26        for (int i = size-1; i >= 0; i--)
27            str += num[i]+'0';
28        return str;
29    }
30    int& operator[](int index){ // 访问超过 size 的内容会自动扩容
31        assert(index >= 0); // 防止越界
32        if (index >= num.size()) num.resize(index+1, 0); // 扩容
33        return num[index];
34    }
35    void refresh(){ // 刷新当前位数，删除无用前导 0
36        while (num.size() && !num.back()) num.pop_back();
37        size = num.size();
```

```

38         if (!size) sign = 1;
39     }
40     friend bool operator<(BigInt& x,BigInt& y){
41         x.refresh(),y.refresh();
42         if (x.sign != y.sign) return x.sign < y.sign;
43         bool inv = x.sign == -1;
44         if (x.size != y.size) return (x.size < y.size) ^ inv;
45         for (int i = x.size - 1;i >= 0;i--)
46             if (x[i] != y[i]) return (x[i] < y[i]) ^ inv;
47         return false;
48     }
49     static BigInt unsigned_add(BigInt x,BigInt y){ //
50         ↪ 无符号加法
51         x.refresh(),y.refresh();
52         int sz = std::max(x.size,y.size);
53         BigInt z = 0LL;
54         for (int i = 0; i < sz; i++)
55             z[i] += x[i] + y[i],z[i+1] += z[i]/10, z[i] %=
56                 ↪ 10;
57         z.refresh();
58         return z;
59     }
60     static BigInt unsigned_remove(BigInt x,BigInt y){ //
61         ↪ 无符号减法
62         x.sign = 1,y.sign = 1;
63         if (x < y) std::swap(x,y);
64         int sz = x.size;
65         for (int i = 0; i < sz; i++) {
66             if (i < y.size) x[i] -= y[i];
67             if (x[i] < 0) x[i] += 10,x[i+1]--;
68         }
69         x.refresh();
70         return x;
71     }
72     friend BigInt operator+(BigInt x,BigInt y){
73         if (y.sign == -1) return y.sign = 1,x-y;
74         if (x.sign == -1) return x.sign = 1,y-x;
75         return unsigned_add(x,y);

```

```
73     }
74     friend BigInt operator-(BigInt x, BigInt y){
75         BigInt z;
76         if (y.sign == -1) return y.sign = 1, x+y;
77         if (x.sign == -1) return z =
78             ↪ unsigned_add(x, y), z.sign = -1, z;
79         z = unsigned_remove(x, y), z.sign = (x < y ? -1 : 1);
80         return z;
81     }
82     friend BigInt operator*(BigInt x, BigInt y){
83         BigInt z;
84         x.refresh(), y.refresh();
85         for (int i = 0; i < x.size; ++i)
86             for (int j = 0; j < y.size; ++j)
87                 z[i+j] += x[i]*y[j], z[i+j+1] +=
88                     ↪ z[i+j]/10, z[i+j] %= 10;
89         z.sign = x.sign * y.sign, z.refresh();
90         return z;
91     }
92     friend BigInt operator/(BigInt x, BigInt y){
93         BigInt z;
94         x.refresh(), y.refresh();
95         assert(y.size > 0);
96         int sg = x.sign * y.sign;
97         x.sign = y.sign = 1;
98         int sz = x.size - y.size;
99         for (int i = sz; i >= 0; i--){
100             BigInt t = y << i;
101             while (!(x < t)) z[i]++, x -= t;
102         }
103         z.sign = sg, z.refresh();
104         return z;
105     }
106     friend BigInt operator%(BigInt x, BigInt y){
107         x.refresh(), y.refresh();
108         assert(y.size > 0);
109         int sg = x.sign;
110         x.sign = y.sign = 1;
```

```

109     for (int i = x.size - y.size; i >= 0; i--){
110         BigInt t = y << i;
111         while (!(x < t)) x -= t;
112     }
113     x.sign = sg, x.refresh();
114     return x;
115 }
116 friend BigInt operator<<(BigInt x, int y){ // x * 10^y
117     std::string raw = x.to_string();
118     for (; y > 0; y--) raw += "0";
119     return BigInt(raw);
120 }
121 BigInt& operator += (BigInt y){
122     return *this = *this + y;
123 }
124 BigInt& operator -= (BigInt y){
125     return *this = *this - y;
126 }
127 BigInt& operator *= (BigInt y){
128     return *this = *this * y;
129 }
130 BigInt& operator /= (BigInt y){
131     return *this = *this / y;
132 }
133 BigInt& operator %= (BigInt y){
134     return *this = *this % y;
135 }
136 friend std::ostream& operator<<(std::ostream& os, BigInt
    ↪ x){
137     return os << x.to_string();
138 }
139 friend std::istream& operator>>(std::istream& is, BigInt
    ↪ &x){
140     std::string str;
141     is >> str, x = BigInt(str);
142     return is;
143 }

```

144 };

12.5 分数

Fraction(P,Q): 有理数 P/Q , 构造即约分到最简并记录符号; 支持四则运算、比较 ($\leq, >$, 需 C++20) 与流式 IO。每次运算后约简, 复杂度 $\mathcal{O}(\log |P|)$ 。

```

1  struct Fraction{
2      long long p=1,q=1;
3      int sign = 1;
4      Fraction(long long P=0,long long Q=1){
5          int s = 1;
6          if (Q < 0) s = -1,Q = -Q;
7          if (P) s *= P/std::abs(P),p=std::abs(P);
8          else p = 0,s = 1;
9          sign = s,q = Q;
10         update();
11     }
12     void update(){
13         assert(q != 0);
14         if (!p) sign = 1; // 零值统一为正号, 避免出现 -0 (如 0 *
           ↪ 负数的结果为 0 时符号残留)
15         long long cd = std::gcd(p,q);
16         p /= cd,q /= cd;
17     }
18     void common(long long divisor){
19         long long times = divisor / std::gcd(divisor,q);
20         p *= times,q *= times;
21     }
22     void common(Fraction &f){
23         common(f.q);
24     }
25     friend std::ostream& operator<<(std::ostream
           ↪ &os,Fraction f){
26         if (f.sign < 0) os << '-';
27         if (f.q != 1) os << f.p << "/" << f.q;
28         else os << f.p;

```

```
29         return os;
30     }
31     Fraction& operator+=(Fraction y){
32         common(y),y.common(*this);
33         p *= sign,p += y.p * y.sign;
34         if (p) sign = p / std::abs(p),p = std::abs(p);
35         else p = 0,sign = 1;
36         update();
37         return *this;
38     }
39     friend Fraction operator+(Fraction x,Fraction y){
40         Fraction ans = x;
41         return ans += y;
42     }
43     Fraction& operator-=(Fraction y){
44         y.sign *= -1;
45         this->operator+=(y);
46         return *this;
47     }
48     friend Fraction operator-(Fraction x,Fraction y){
49         Fraction ans = x;
50         return ans -= y;
51     }
52     Fraction& operator*=(Fraction y){
53         p *= y.p,q *= y.q;
54         sign *= y.sign;
55         update();
56         return *this;
57     }
58     friend Fraction operator*(Fraction x,Fraction y){
59         Fraction ans = x;
60         return ans *= y;
61     }
62     Fraction& operator/=(Fraction y){
63         std::swap(y.p,y.q);
64         return this->operator*=(y);
65     }
66     friend Fraction operator/(Fraction x,Fraction y){
```

```

67     Fraction ans = x;
68     return ans != y;
69 }
70 // require C++20
71 friend auto operator<=>(Fraction x, Fraction y){
72     x.common(y), y.common(x);
73     return x.p * x.sign <=> y.p * y.sign;
74 }
75 bool operator==(Fraction b) const {
76     Fraction a = *this;
77     a.update(), b.update();
78     return (a.sign == b.sign && a.p == b.p && a.q ==
79             ↪ b.q);
80 };

```

12.6 时空优化

12.6.1 读入优化

`read<T>()`: 读取一个值。`read<int>()`、`read<long long>()` 走 `getchar` 快读, `read<std::string>()` 读取空白分隔的串, 其余类型回落 `cin` (快读与 `cin` 不要混用)。

```

1 namespace QuickStream {
2     template<typename T>
3     T read() {
4         T x;
5         std::cin >> x;
6         return x;
7     }
8
9     template<typename T>
10    inline T read_int() {
11        T x = 0;
12        bool f = 0;
13        int ch = getchar();
14        while (ch != EOF && (ch < '0' || ch > '9')) {

```

```
15         if (ch == '-') f = 1;
16         ch = getchar();
17     }
18     for (; ch >= '0' && ch <= '9'; ch = getchar())
19         x = x * 10 + (ch - '0');
20     return f ? -x : x;
21 }
22
23 template<>
24 int read<int>() {
25     return read_int<int>();
26 }
27
28 template<>
29 long long read<long long>() {
30     return read_int<long long>();
31 }
32
33 template<>
34 std::string read<std::string>() {
35     std::string s;
36     int ch = getchar();
37     while (ch != EOF && (ch == ' ' || ch == '\n' || ch
38 ↪ == '\r' || ch == '\t')) {
39         ch = getchar();
40     }
41     while (ch != EOF && ch != ' ' && ch != '\n' && ch !=
42 ↪ '\r' && ch != '\t') {
43         s += (char)ch;
44         ch = getchar();
45     }
46     return s;
47 }
48 using QuickStream::read;
```

12.6.2 指令集优化

启用 O3、循环展开与常见位运算指令集，对大常数运算有明显加速；要求评测机为 GCC 且支持相应指令（不支持时 `pragma` 会被忽略），部分 OJ 禁用。

```
1 #pragma GCC optimize("O3,unroll-loops,inline")
2 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```